Operating System Fundamentals

By Mohamed Gamal

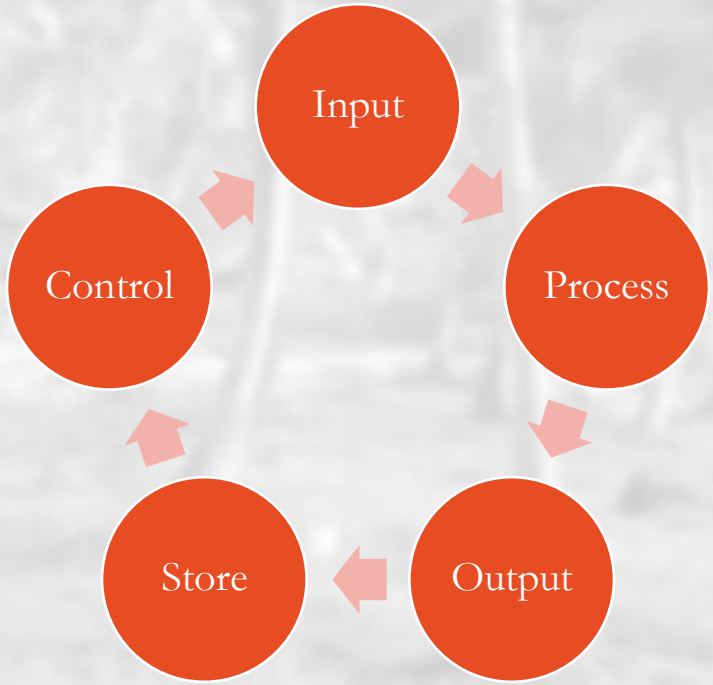
Agenda

The topics of today's session:

1. Operating System
2. Different Types of Systems
3. Computer System Operation
4. I/O Structure
5. Storage Structure
6. System Protection
7. OS Operations
8. Protection and Security
9. OS Services
10. System Structure
11. Processes and Threads
12. Synchronizations and Deadlocks
13. Open-Source OSes
14. Binary Logic
15. Command Line

Basic Computer Configuration

- The computer is an electronic machine that performs the following five basic operations:
  - Input
  - Process
  - Output
  - Store
  - Control



Operating System

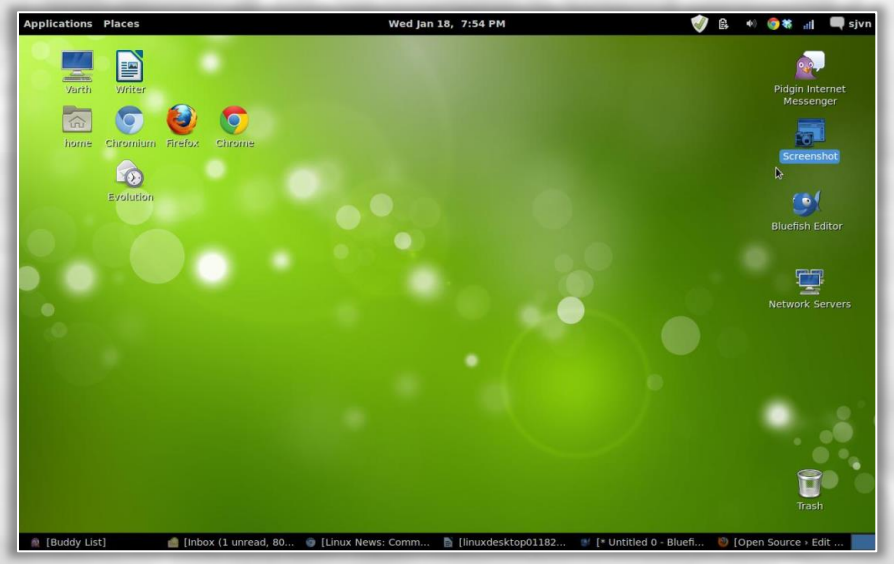
What's An Operating System (OS)?

- A program that acts as an intermediary between a user of a computer and the computer hardware.

- Operating System goals:
  - Execute user programs
  - Use the computer hardware in an efficient manner
  - Make user's life easier ;)

- Operating System examples:
  - Windows
  - Unix / Linux
  - MacOS

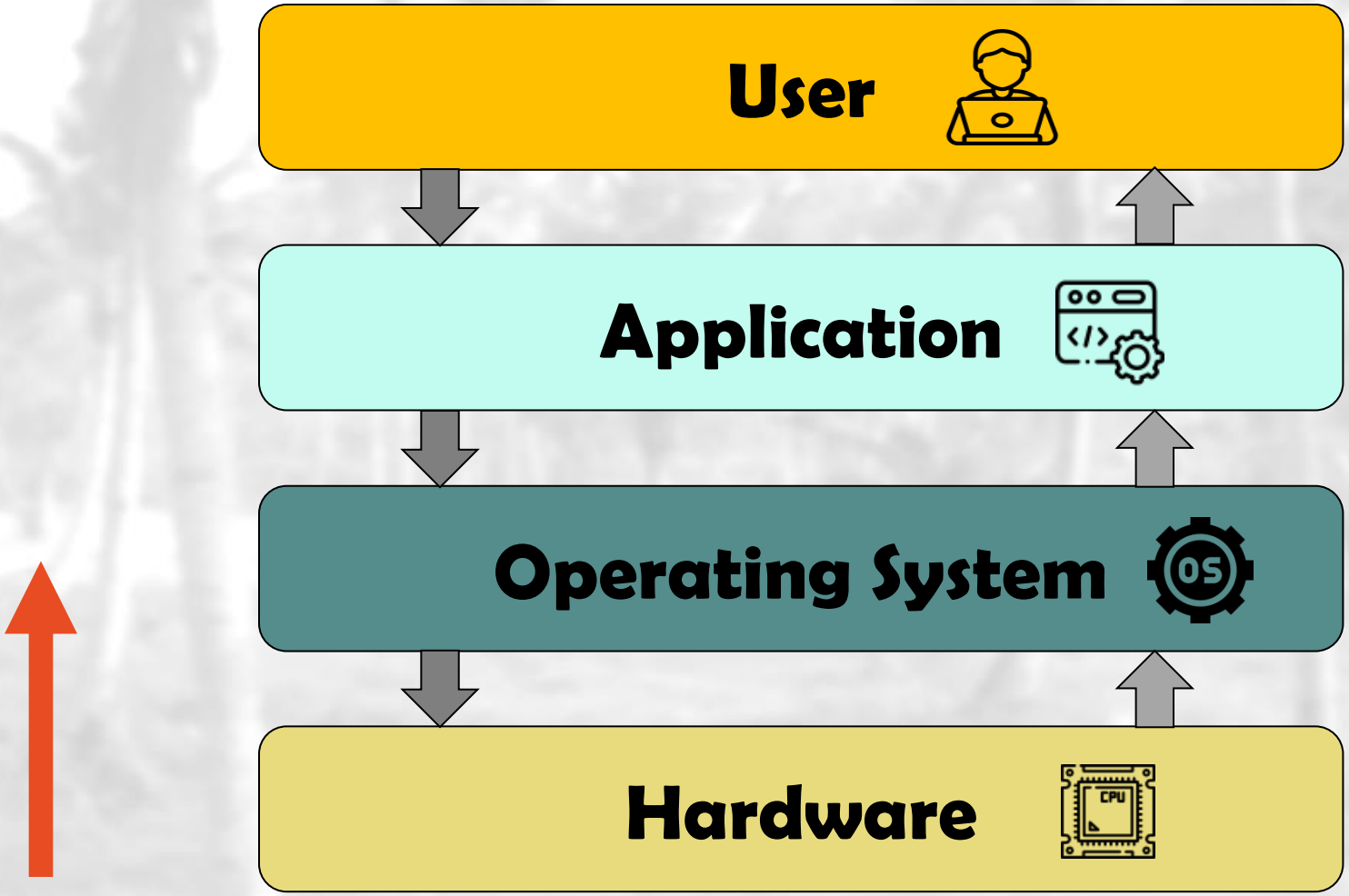
OSes User Interfaces



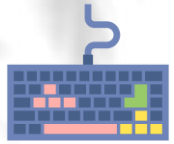
History of Operating Systems (Click)

- Operating systems were first *developed in the late 1950s* to manage tape storage.
- The General Motors Research Lab implemented the first OS in the early 1950s for their *IBM 701*
- In the late 1960s, the first version of the *Unix OS* was developed
- The first OS built by Microsoft was *DOS*, *built in 1981*.
- The present-day popular OS Windows first came to existence in 1985 when a GUI was created and paired with MS-DOS.

Computer System Components



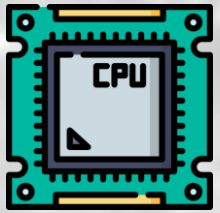
1) Computer Hardware



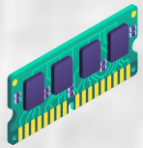
Input device

Output device

Central Processing Unit (CPU)



Main Memory (RAM)

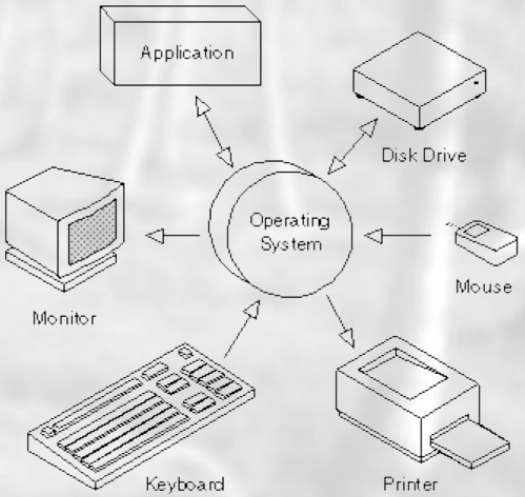


Secondary Storage

Anatomy of a Computer

2) Operating System

- OS controls and coordinates the use of hardware among various application programs for users
  - It manages and allocates the resources (e.g., CPU, Storage, …, etc.)
  - It controls execution of user programs and operations of I/O devices
- OS is a **Resource allocator** and **Control program**
- **Kernel** — the one program running at all times



3) Application Programs

- Video Games
- Web browsers
- Spread sheets
- Word processors
- Compilers
- …





3) Application vs. System Programs

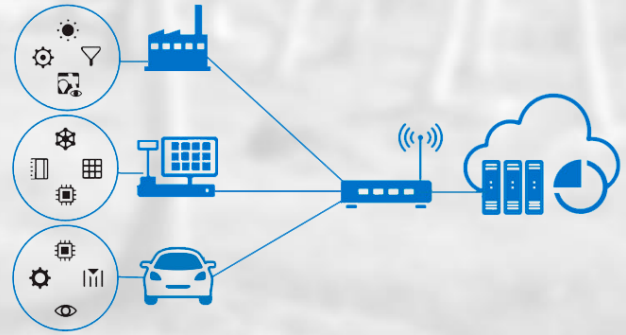
- System programs, also known as operating system utilities or system utilities, are software programs that are shipped with the OS and are an integral part of the operating system.
    - Status Information
    - File Modification
    - Drivers
    - …

4) Users

- People (e.g., normal users)
- Machines (e.g., servers, embedded systems … etc.)
- Other Computers (e.g., remote or networked devices … etc.)

Users

Machines



Other Computers

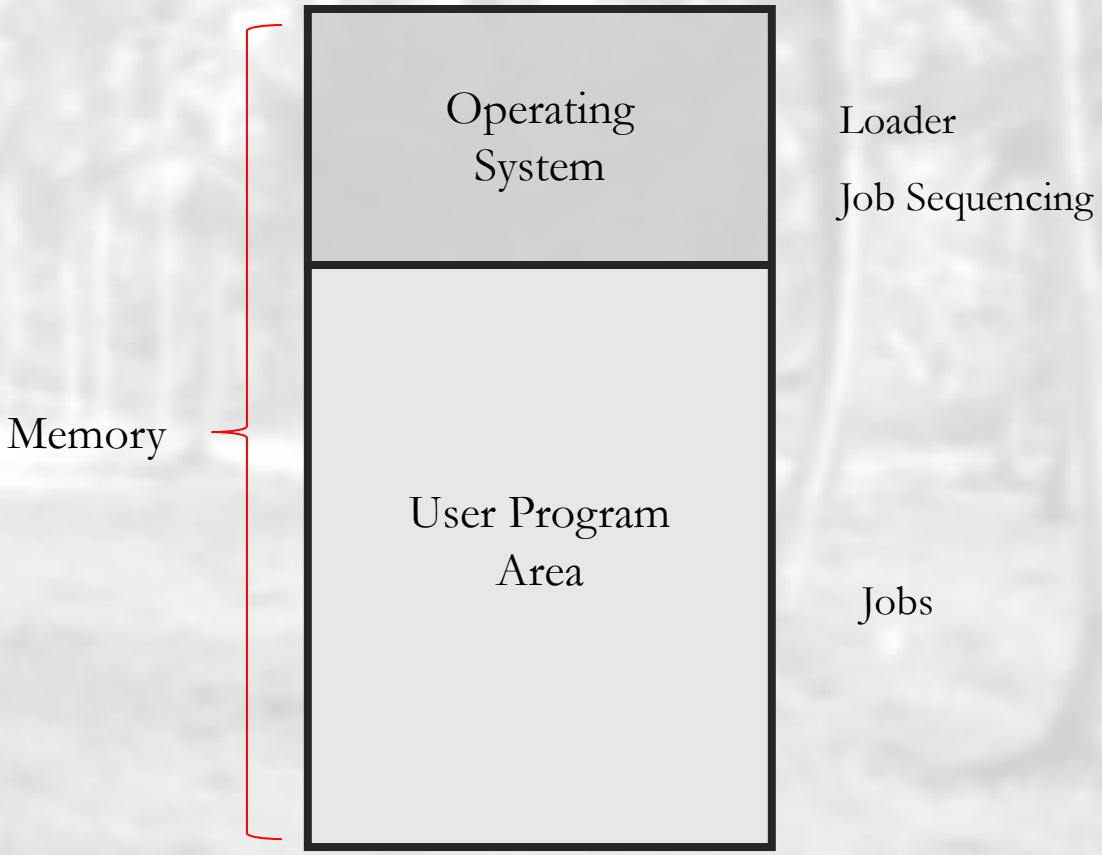
Mainframe Systems

Mainframe Systems

- A very large *powerful* and *expensive* computer capable of supporting hundreds, or even thousands, of users simultaneously.
- Reduce setup time by batching similar jobs.
- Automatic job sequencing
  - Automatically transfers control from one job to another.
  - First rudimentary operating system

Mainframe Systems (Cont.)

- Memory Layout for a simple batch system



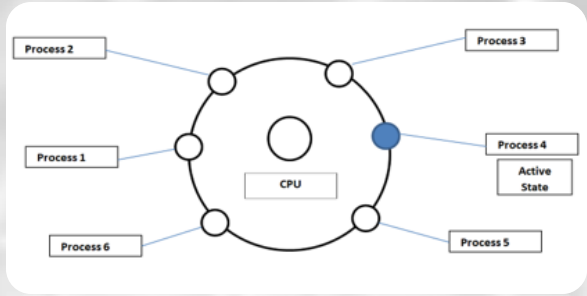
.bat files

Mainframe Systems (Cont.)

- Multi-programmed Batch Systems
  - Several jobs are kept in main memory at the same time, and the CPU is multiplexed among them
- OS features needed
  - I/O routine supplied by the system
- Memory management
  - The system must allocate the memory to several jobs
- CPU/Job scheduling
  - The system must choose among several jobs ready to run

Mainframe Systems (Cont.)

- Time-Sharing Systems (Interactive Computing)
  - The CPU is multiplexed among several jobs that are kept in memory and on disk
  - The CPU is allocated to a job only if the job is in memory
  - A job is swapped in and out of memory to the disk
  - On-line/Interactive communication between the user and the system is provided
    - When the operating system finishes the execution of one command, it seeks the next “control statement” from the user’s keyboard



Desktop Systems

Desktop Systems (Microcomputer)

- Personal Computers (PCs)
  - Dedicated to a single user

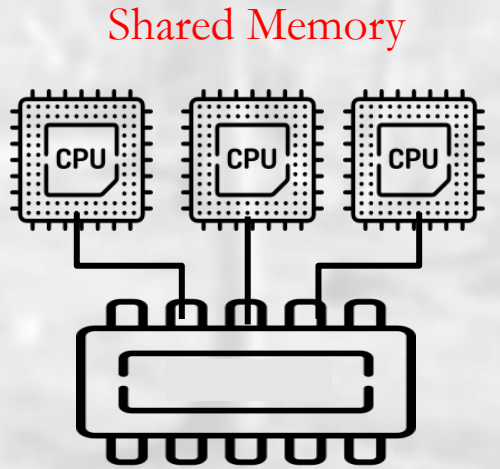
- I/O devices
  - Keyboards
  - Mouse
  - Display Monitors
  - Printers

- User convenience and responsiveness
  - May run several different types of operating systems
    - E.g., Windows, MacOS, Linux

Multiprocessor Systems

Parallel Systems

- Systems with more than one CPU in close communication
  - Also known as multiprocessor/multicore systems
- Tightly coupled system
  - processors share memory and a clock; communication usually takes place through the shared memory
- Advantages of parallel system:
  - Increased throughput
  - Economical
  - Increased reliability
  - graceful degradation



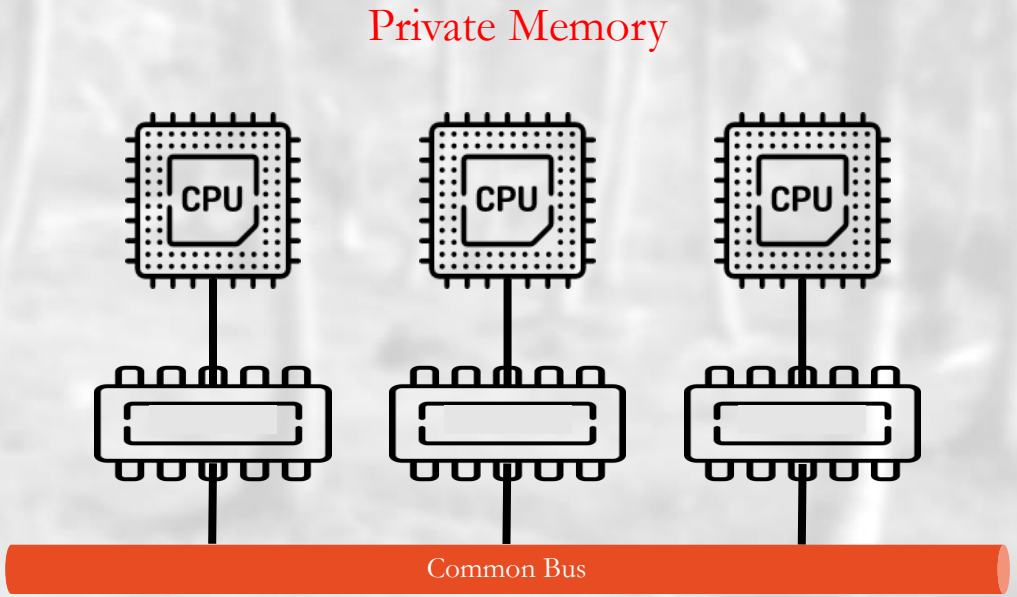
Fetch → Decode → Execute → Store


Task Manager
Performance
Run new task
Processes
Performance
App history
Startup apps
Users
Details
Services
Settings
CPU
41% 4.51 GHz
Memory
10.0/63.9 GB (16%)
Disk 0 (C:)
SSD
0%
Disk 1 (D:)
Removable
0%
Ethernet
Ethernet
S: 0 R: 0 Kbps
Ethernet
VMware Network Adapter VMnet1
S: 0 R: 0 Kbps
Ethernet
VMware Network Adapter VMnet8
S: 0 R: 0 Kbps
GPU 0
NVIDIA GeForce RTX 4060 Ti
47% (49 °C)
CPU
AMD Ryzen 7 5800X 8-Core Processor
% Utilization
100%
60 seconds
0
Utilization
41%
Speed
4.51 GHz
Processes
213
Threads
3741
Handles
107645
Up time
0:02:16:22
Base speed: 3.80 GHz
Sockets: 1
Cores: 8
Logical processors: 16
Virtualization: Enabled
L1 cache: 512 KB
L2 cache: 4.0 MB
L3 cache: 32.0 MB

Distributed Systems

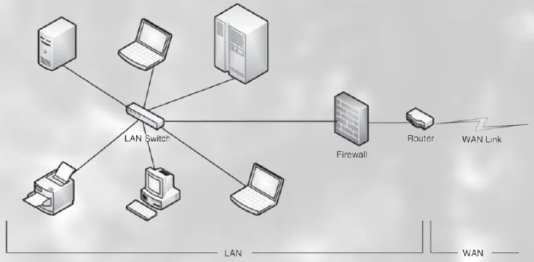
Distributed Systems

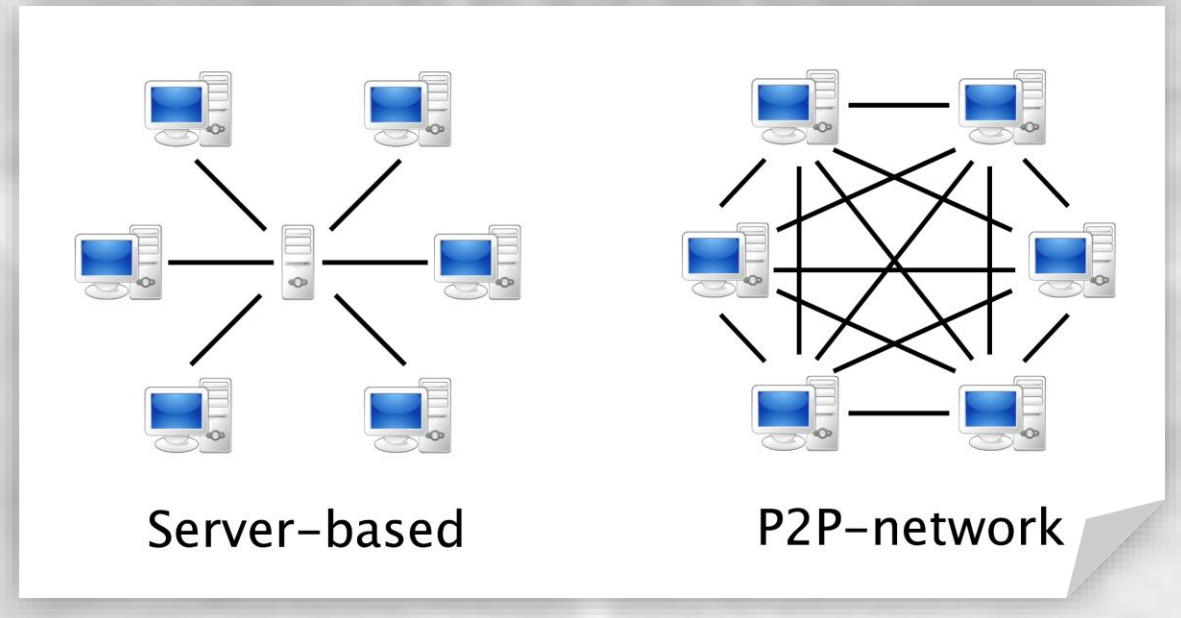
- Distribute the computation among several physical processors
- Loosely coupled system
  - Each processor has its own local memory
  - processors communicate with one another through various communications lines, such as high-speed buses or telephone lines
- Advantages of distributed systems
  - Resources Sharing
  - Computation speed up
  - Reliability
- Google, YouTube … etc.

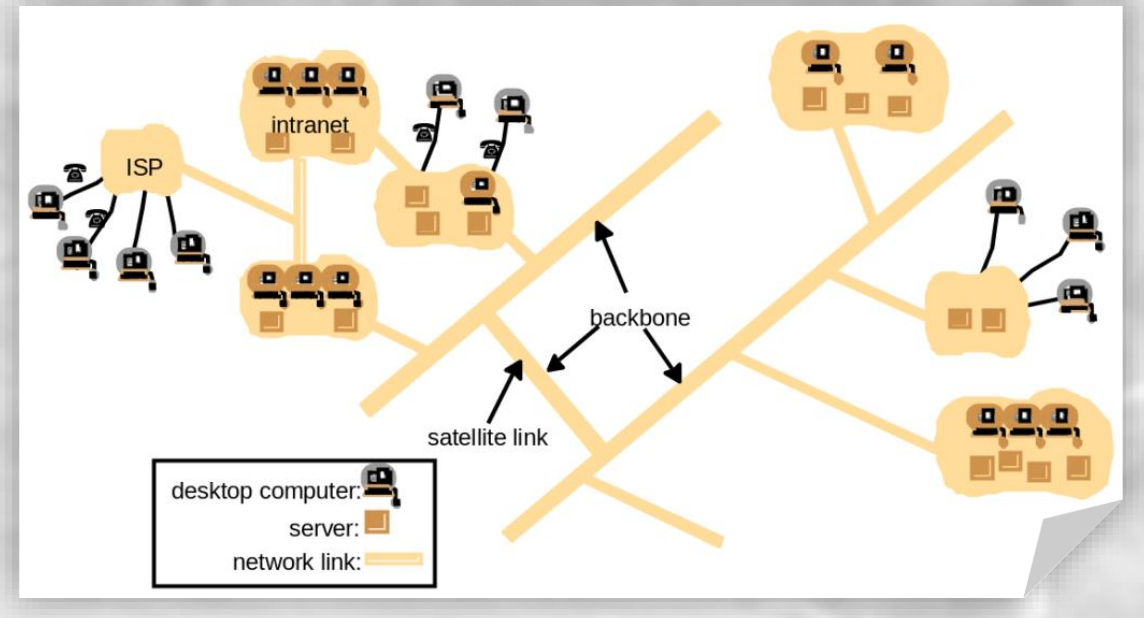


Distributed Systems (Cont.)

- Requires networking infrastructure
- Local area networks (LAN) or Wide area networks (WAN)
- May be either client-server or peer-to-peer systems





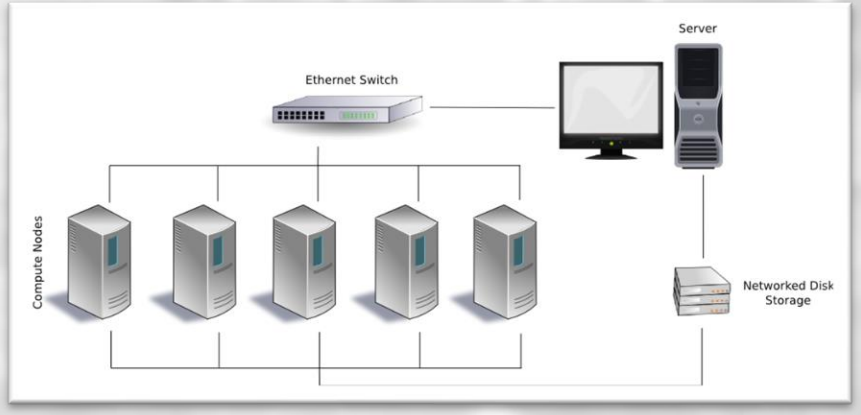


Clustered Systems

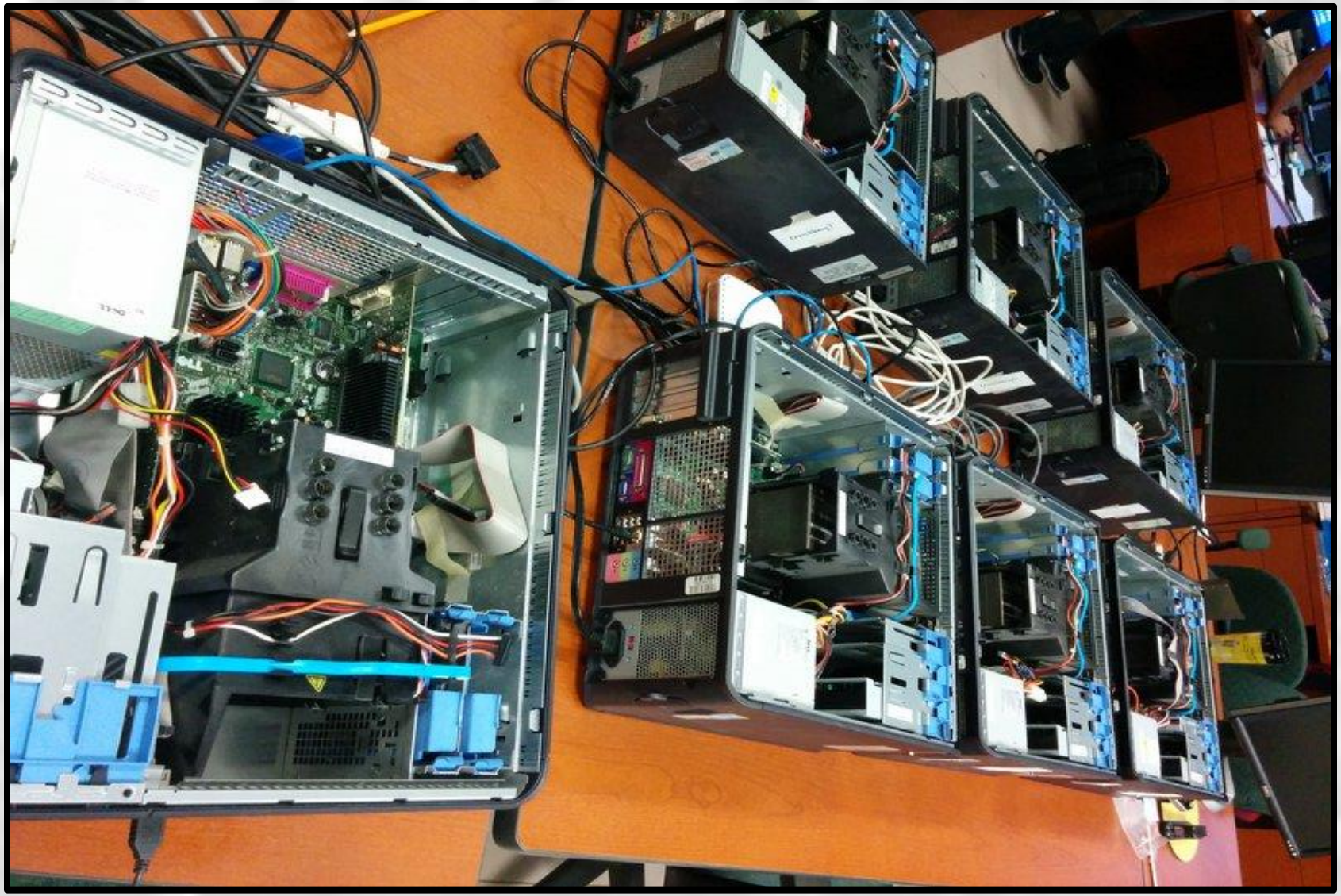
Clustered Systems

- Each cluster is a collection of computing nodes controlled and accessed by a single master node.
- Essentially a group of high-end systems connected through a LAN.
- Homogeneous, same OS, near-identical hardware.
- <u>Advantages:</u>
  - Provides high reliability and performance.
  - Cheaper to build a supercomputer by putting together many simple computers, rather than buying a high-performance one.

Beowulf clusters



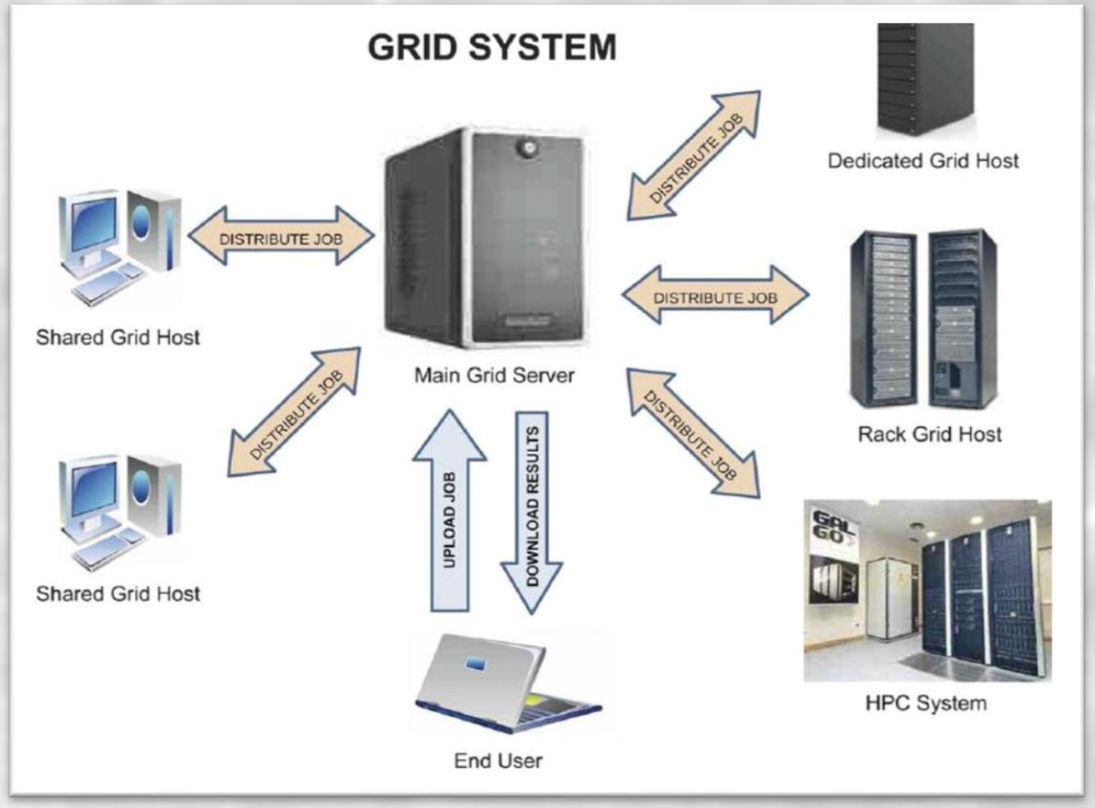
Clustered Systems – Example



Grid Systems

Grid Systems

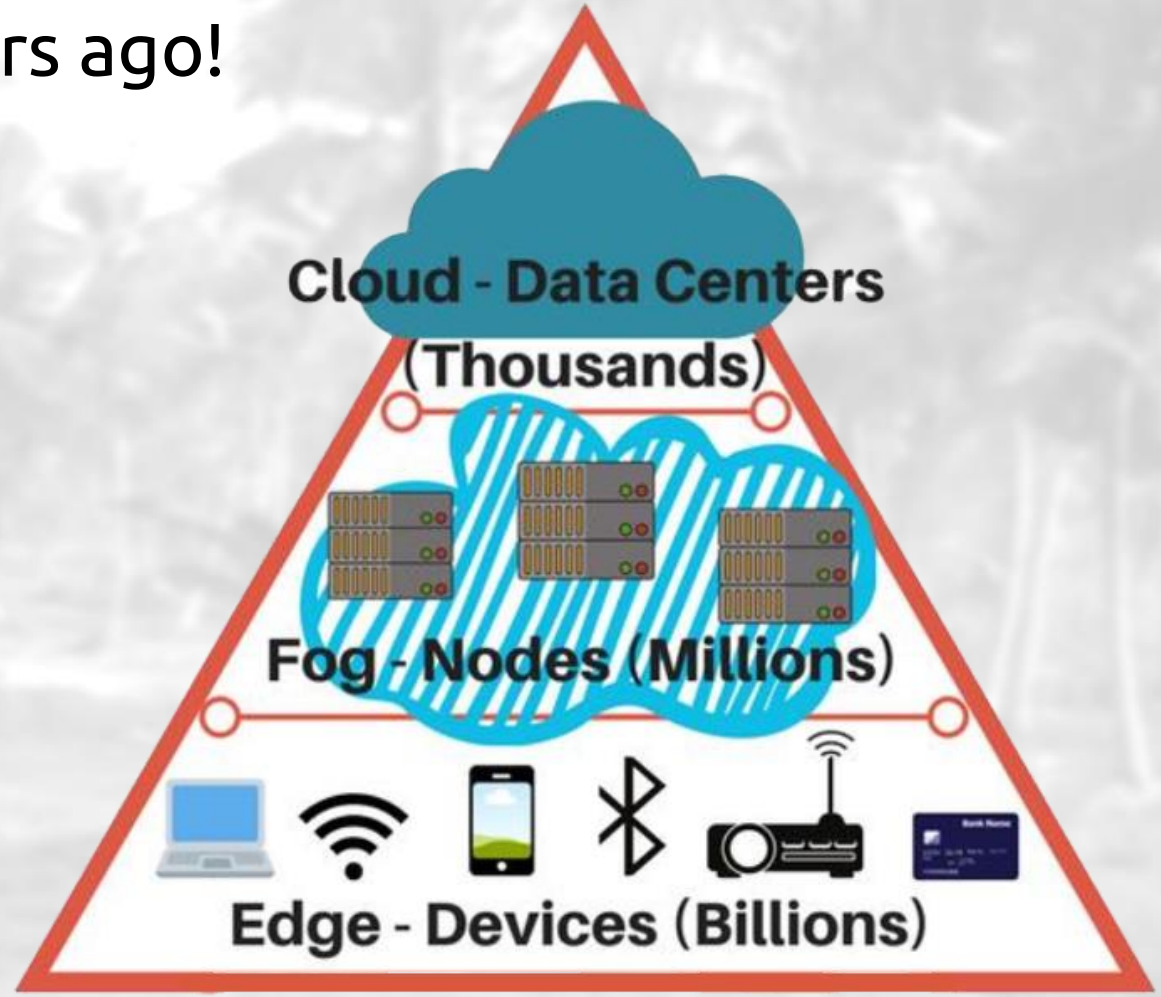
- Lots of nodes from everywhere:
  - Heterogeneous
  - Dispersed across several organizations
  - Can easily span a WAN
- Virtual organizations concept
- Google servers everywhere!

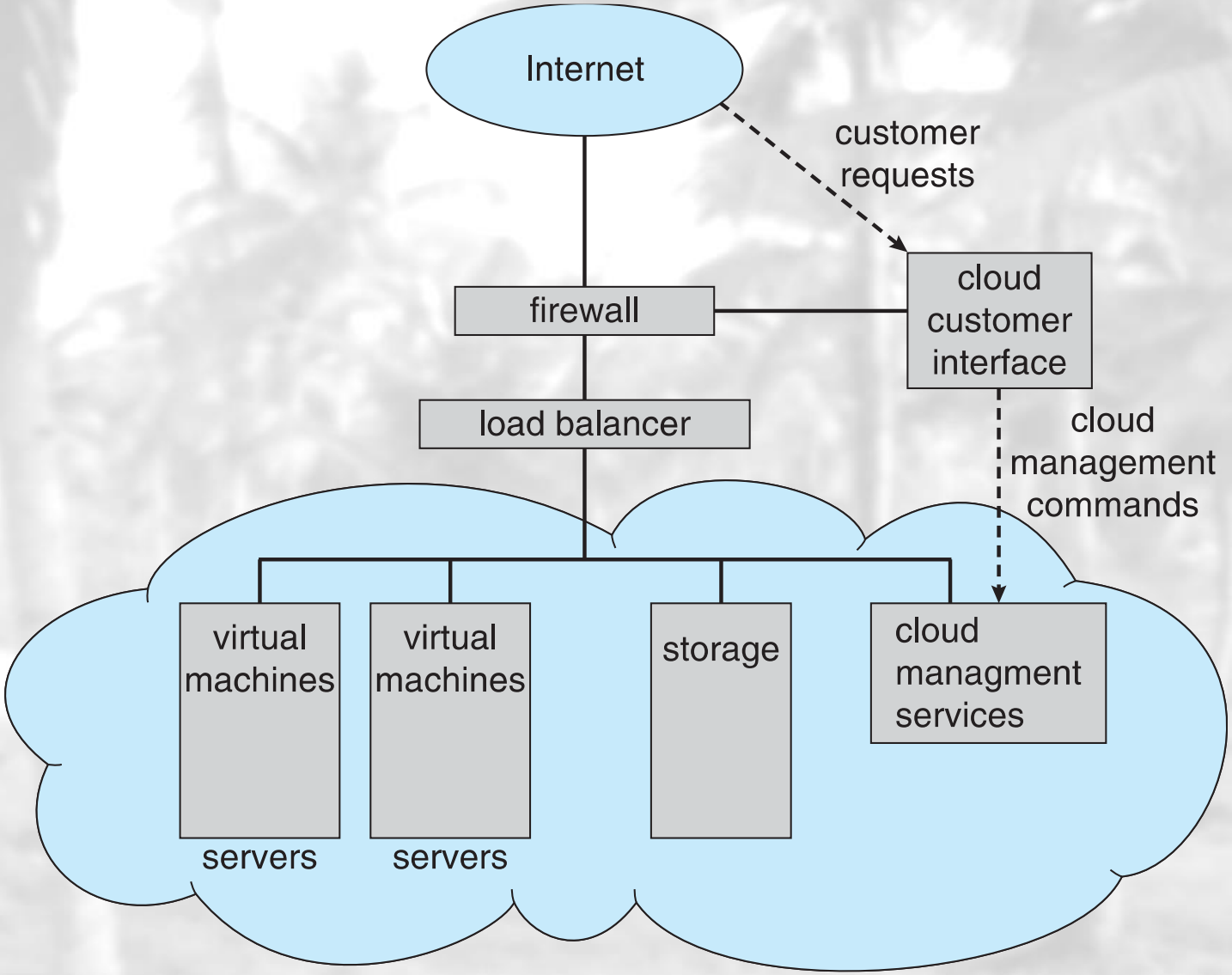


Cloud Systems

Cloud Systems

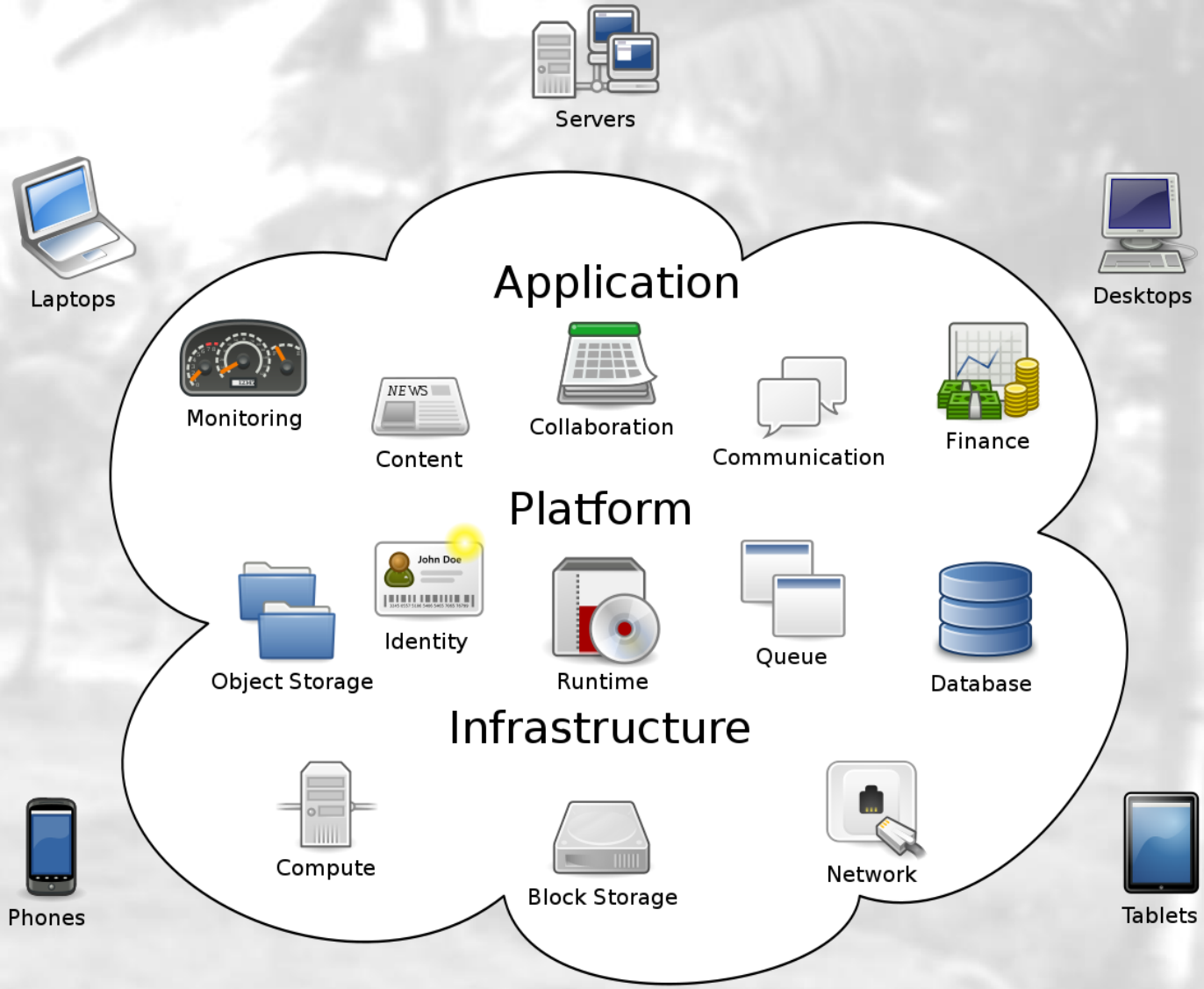
- Grid was a hit about 10 years ago!
- Grid is divided into
  - Cloud Computing
  - Edge/Fog Computing




Internet
customer requests
firewall
cloud customer interface
load balancer
cloud management commands
virtual machines
virtual machines
storage
cloud managment services
servers
servers

Cloud Systems (Cont.)

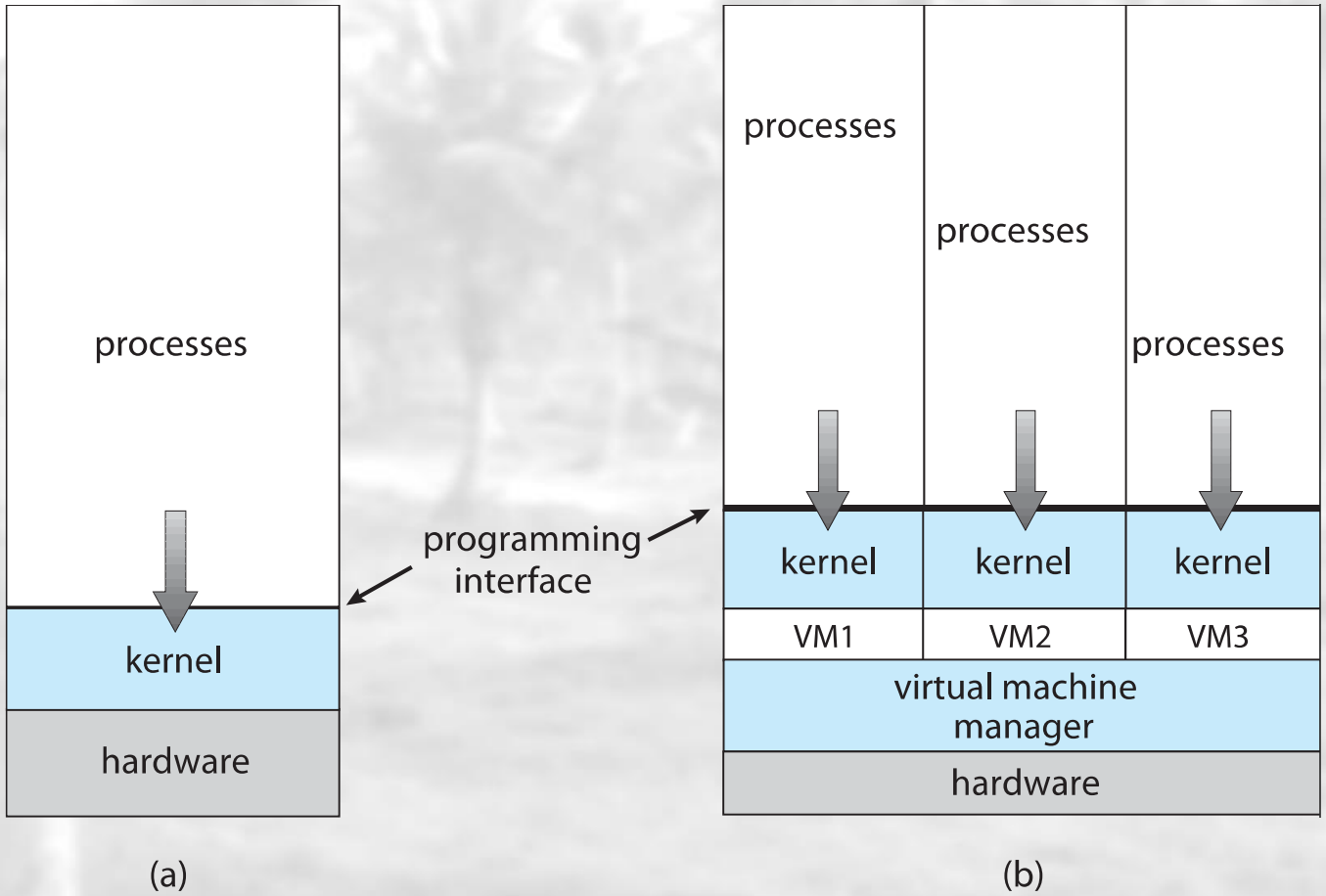
- Application (SaaS)
  - MS Office Suite
- Platform (PaaS)
  - MS Azure
- Infrastructure (IaaS)
  - VMs, CPU, Memory, …
  - e.g., Amazon S3



Virtualization

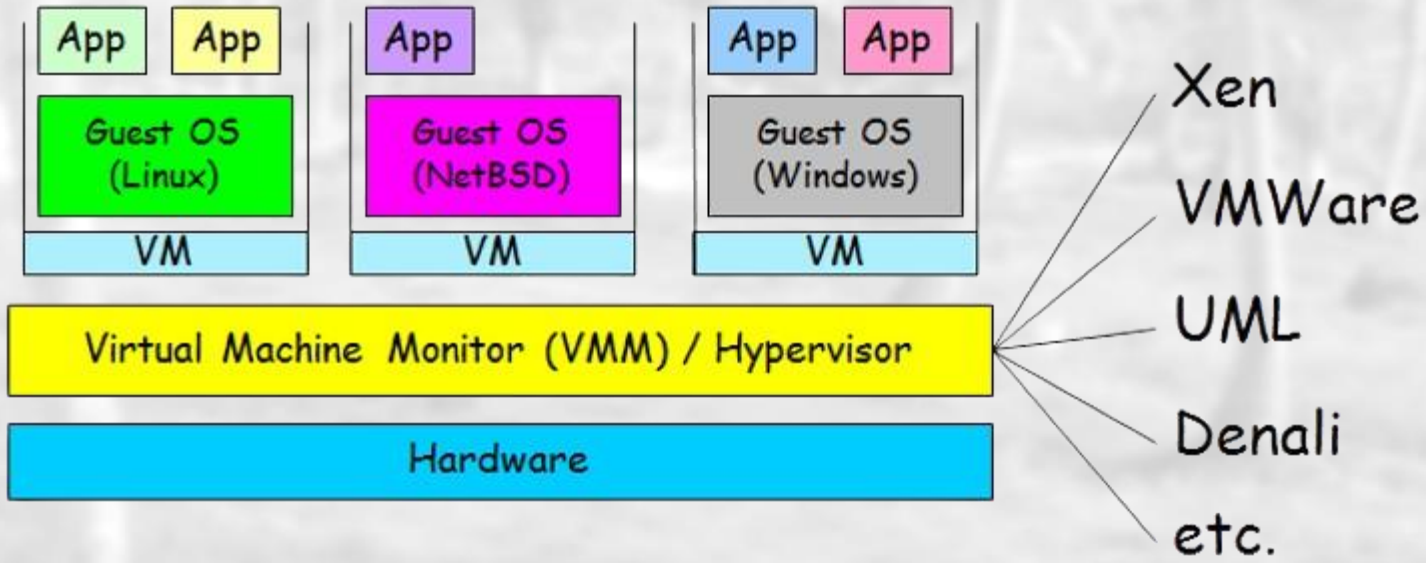
Virtualization

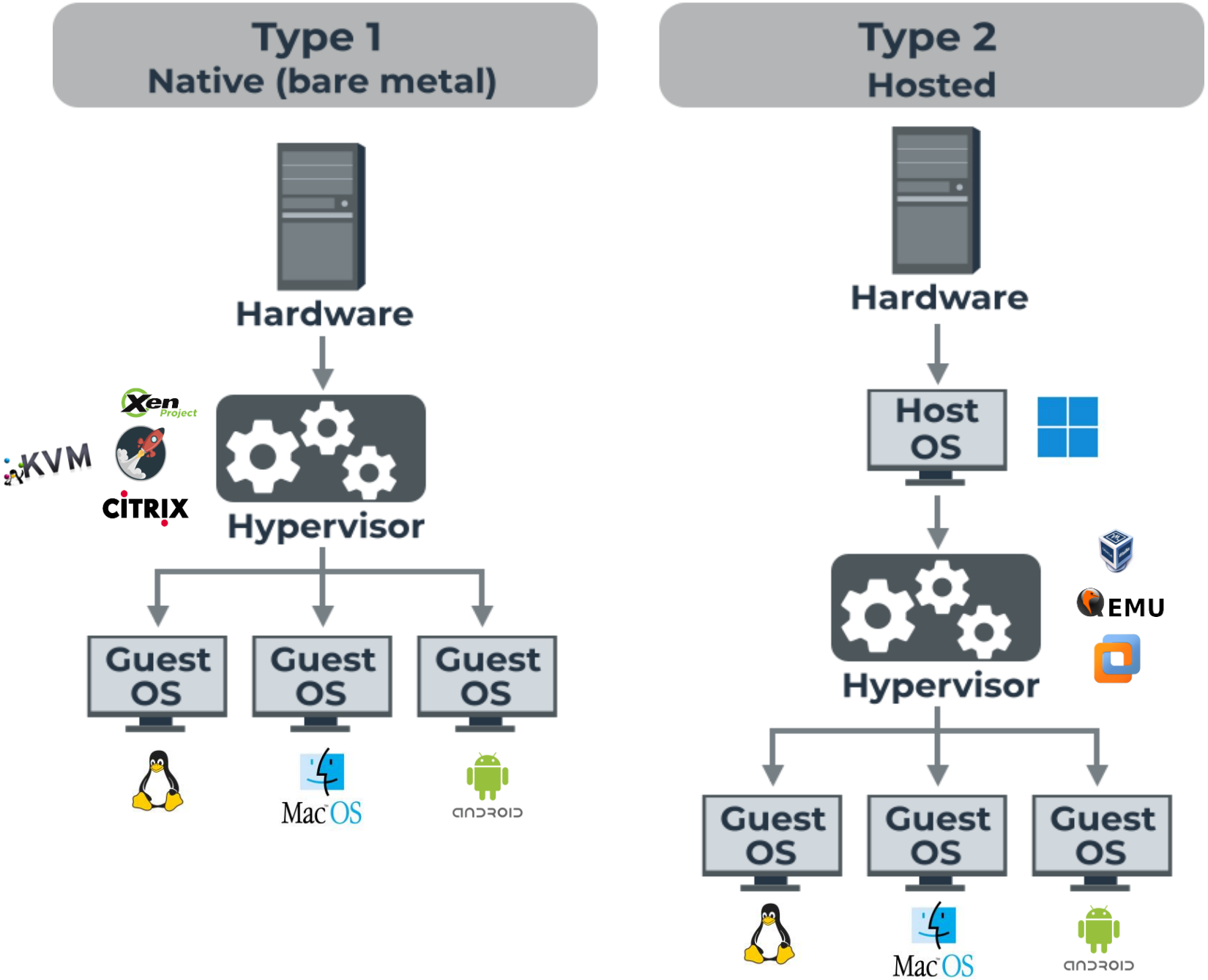
– Allows operating systems to run applications within other OSes.



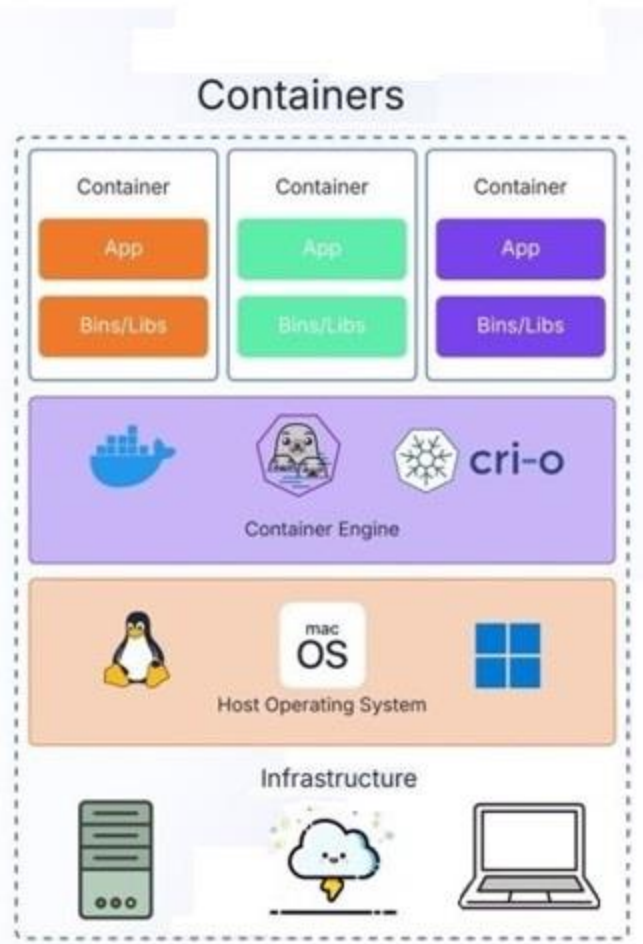
Virtualization (Cont.)

- VM technology allows multiple virtual machines to run on a single physical machine.
- Virtualization is delivered via so-called virtual machines (VMs).
- The host OS provides a **hypervisor**, which manages the access to the hardware.




Type 1
Native (bare metal)
Hardware
Hypervisor
KVM
Xen Project
CITRIX
Guest OS
Guest OS
Guest OS
Mac OS
ANDROID
Type 2
Hosted
Hardware
Host OS
Hypervisor
QEMU
Guest OS
Guest OS
Guest OS
Mac OS
ANDROID

VMs vs CONTAINERS



https://www.docker.com/

Real-Time Systems

Real-Time Operating Systems (*reactive*)

- Intended to serve real-time applications that *process data as it comes* in, typically without buffer delays.
- Real time operating systems are generally special-purpose ones designed to run embedded or specialized systems.
- Requirements
  - Less processing time (i.e., speed)
  - Significant constraints on hardware design
  - Great accuracy
- Disadvantages
  - Very Costly
  - Large memory required

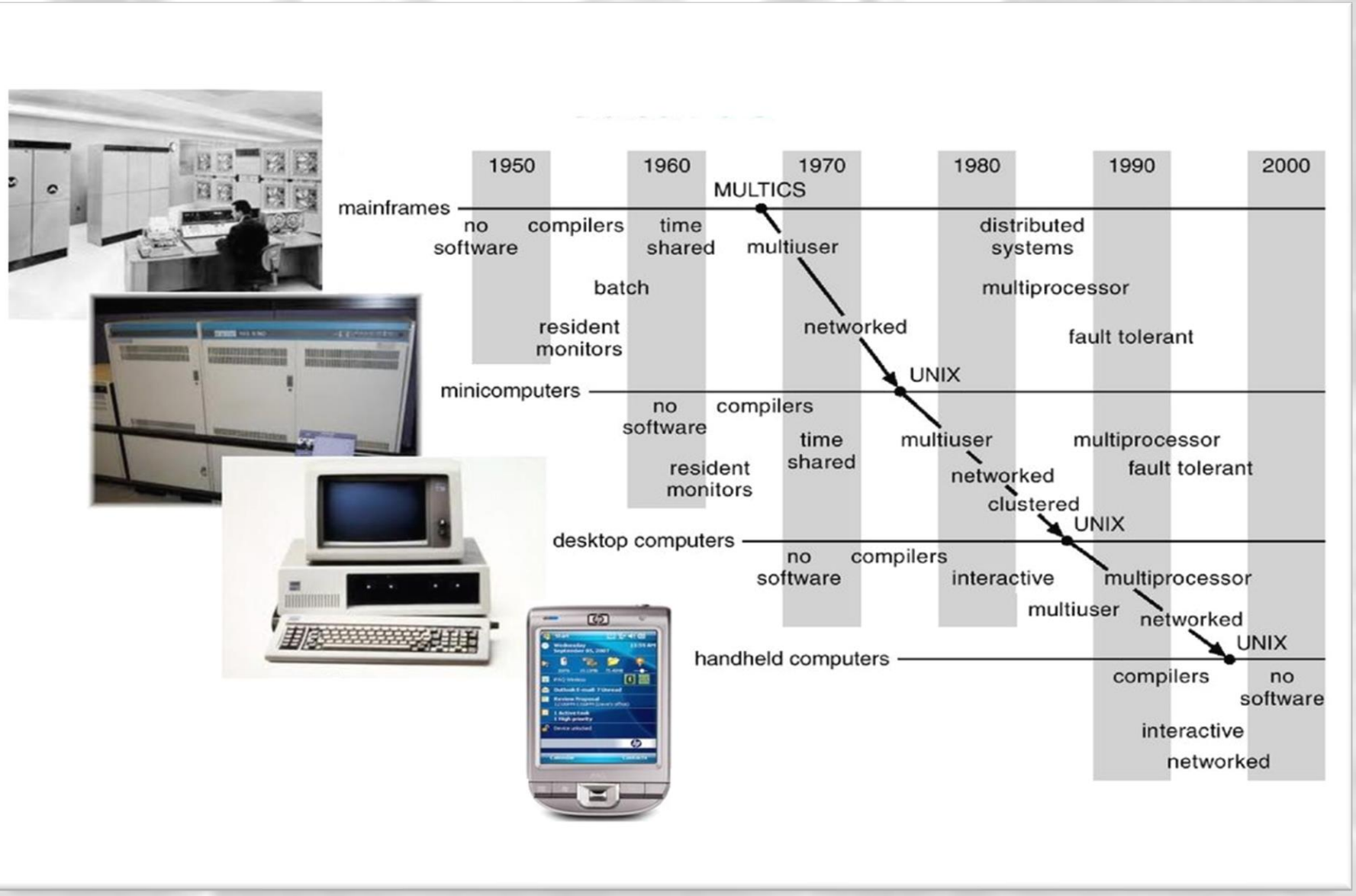
ATM (Embedded OS)

Handheld Systems

Handheld Systems

- Personal Digital Assistants (PDAs)
- Cellular telephones
- More OS features (e.g., GPS, gyroscope, …)
- Issues:
  - Limited memory
  - Slow processors
  - Small display screens
  - Battery bottleneck
- Popular Examples:
  - Apple iOS
  - Google Android

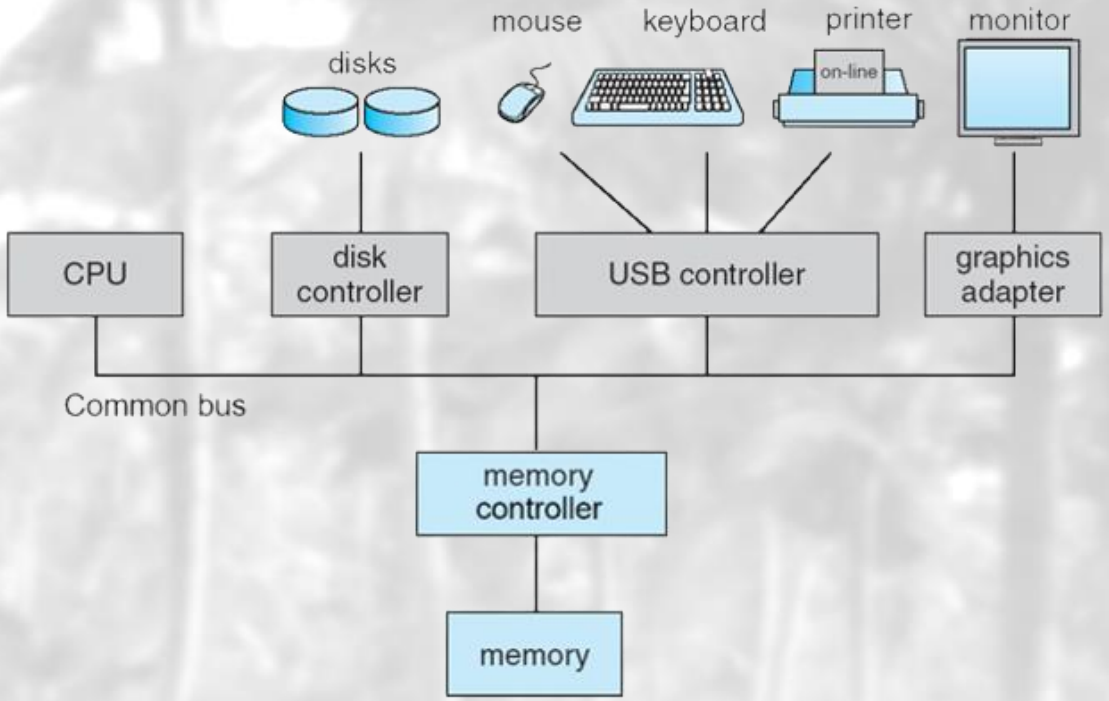
Migration of OS Concepts and Features



Computer System Operation

Computer System Operation

- Computer system consists of
  - CPU
  - Device controllers
  - Common bus
  - Shared memory



- CPU and device controllers execute concurrently.
- Memory controller synchronizes access to memory.

Computer System Operation

- Each device controller is in charge of a particular device type
- Each device controller has a local buffer
- CPU moves data from/to *main memory* to/from local buffers
- I/O is from the device to *local buffer* of controller
- Device controller informs CPU that it has finished its operation by causing an interrupt

Interrupts

- An interrupt is a signal sent to the CPU
- Interrupt transfers control to the interrupt service routine generally, through the interrupt vector, which contains the addresses of all the service routines.
- Interrupt types:
  - Hardware Interrupts (polling/vectored)
  - Software Interrupts: system calls (trap/exception)

Interrupts (Cont.)

- A trap is a software generated interrupt caused by
  - **Error:** division by zero or invalid memory access
  - **Request:** from a user program to O/S

- An operating system is interrupt driven.

Interrupt Handling

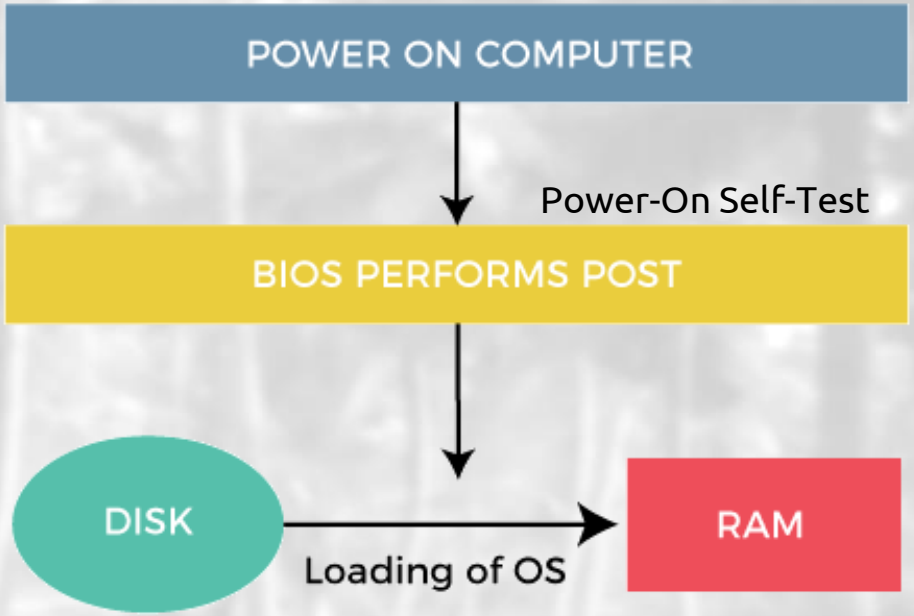
1) CPU is interrupted
2) CPU stops current process
3) CPU transfers execution to a fixed location
4) CPU executes interrupt service routine
5) CPU resumes process

'registers + Program Counter'

- Notes:
  - Interrupts must be handled quickly
  - Interrupted process information must be stored

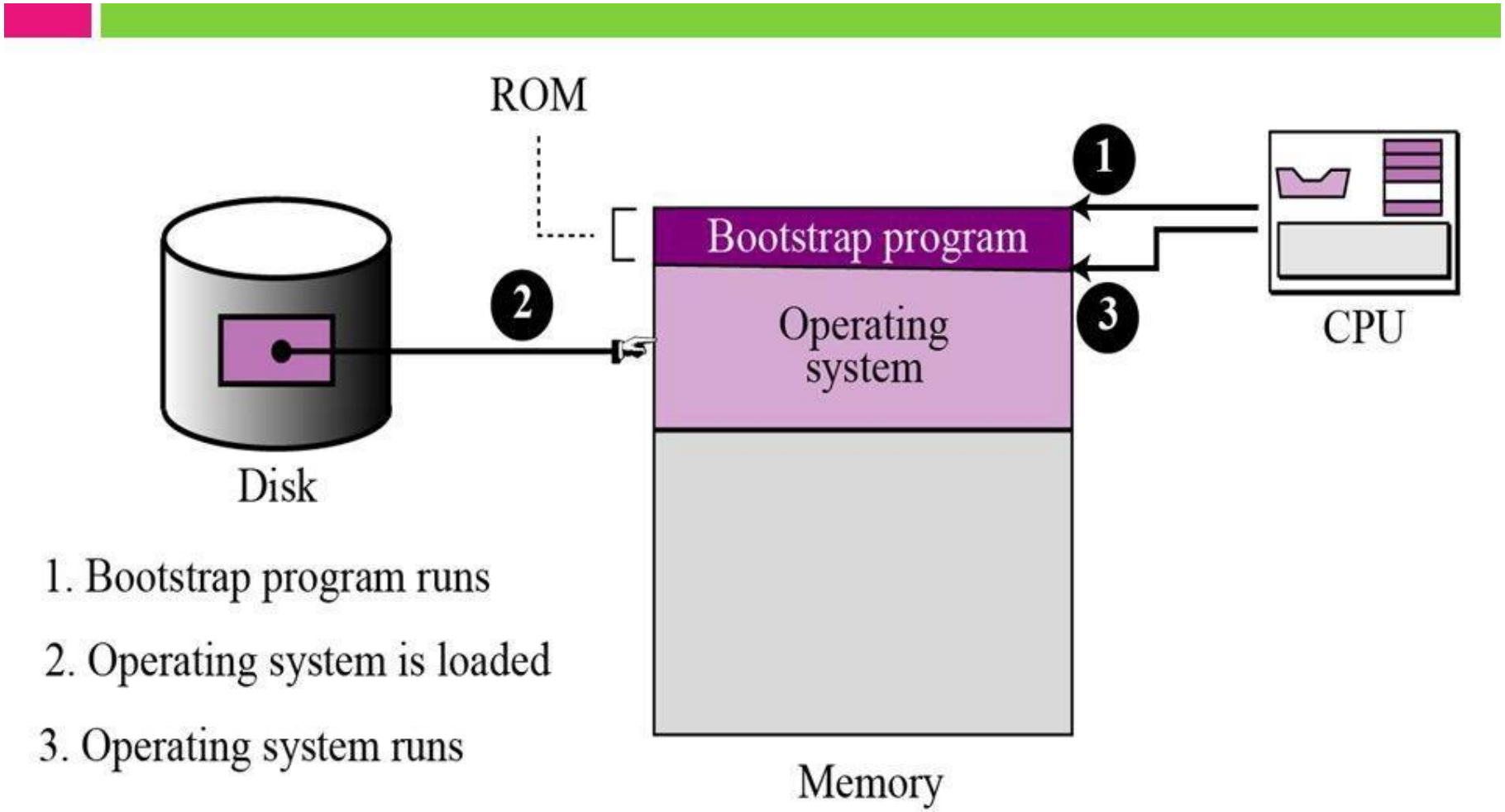
Computer System Startup

1) Power up
2) Initial program: bootstrap (*firmware*)
   - Stored in ROM or EPROM
   - Initialize:
     1) CPU registers
     2) Device controllers
     3) Memory contents
     4) Load the operating system (kernel)
3) Kernel starts the first process – init
4) Init waits for an event (interrupt) to occur



BIOS: basic input/output system

Bootstrap Process

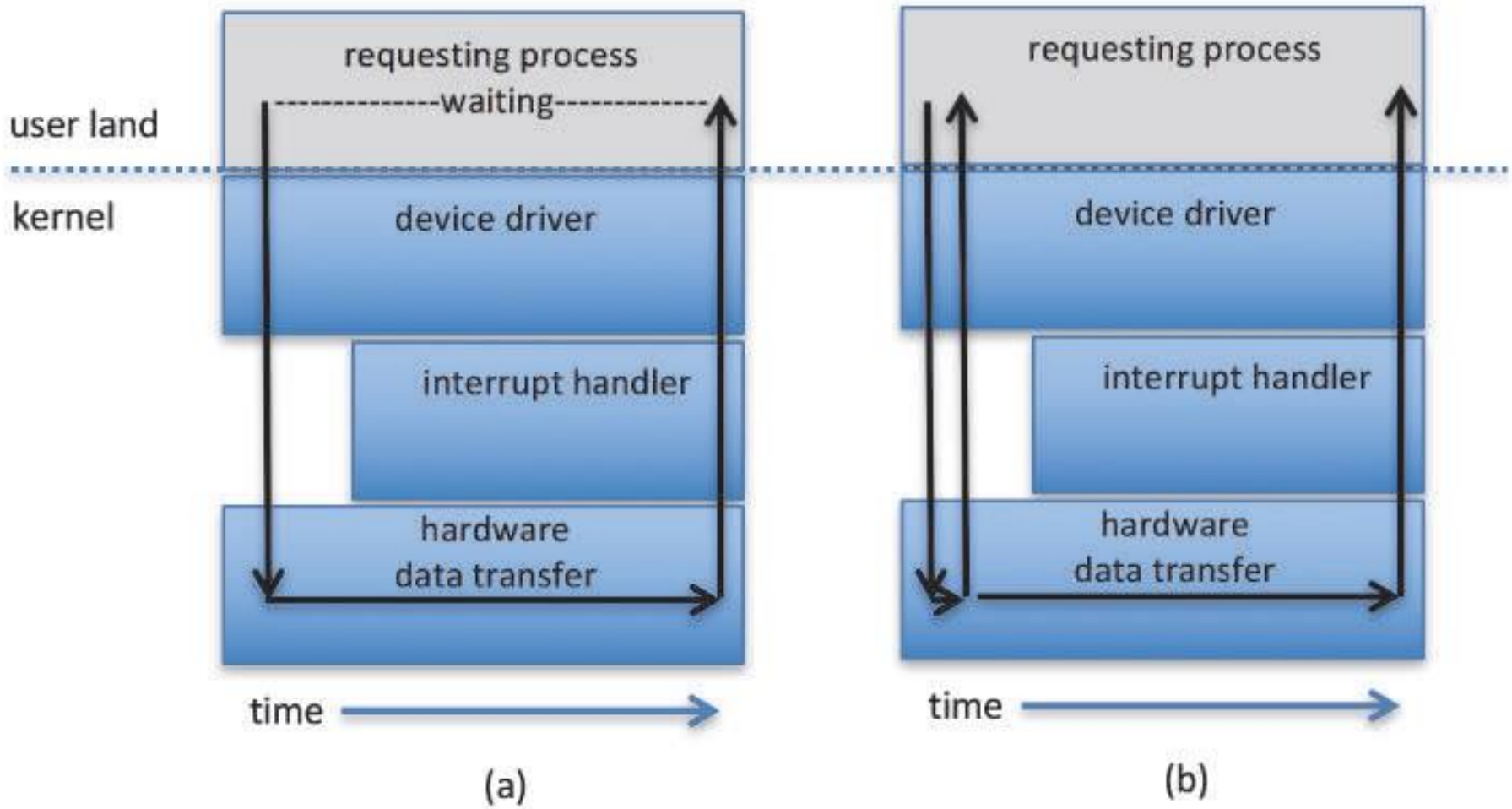


I/O Structure

I/O Structure

- Controllers:
  - Are in charge of a specific type of device
  - Moves data between device and local buffer
  - A controller may have more than one device
  - Buffer size varies

- Two types (Synchronous + Asynchronous):
  - **Type 1:** After I/O starts, control returns to user program *only upon I/O completion*.
  - **Type 2:** After I/O starts, control returns to user program *without waiting for I/O completion*.



Two I/O methods: (a) synchronous and (b) asynchronous.

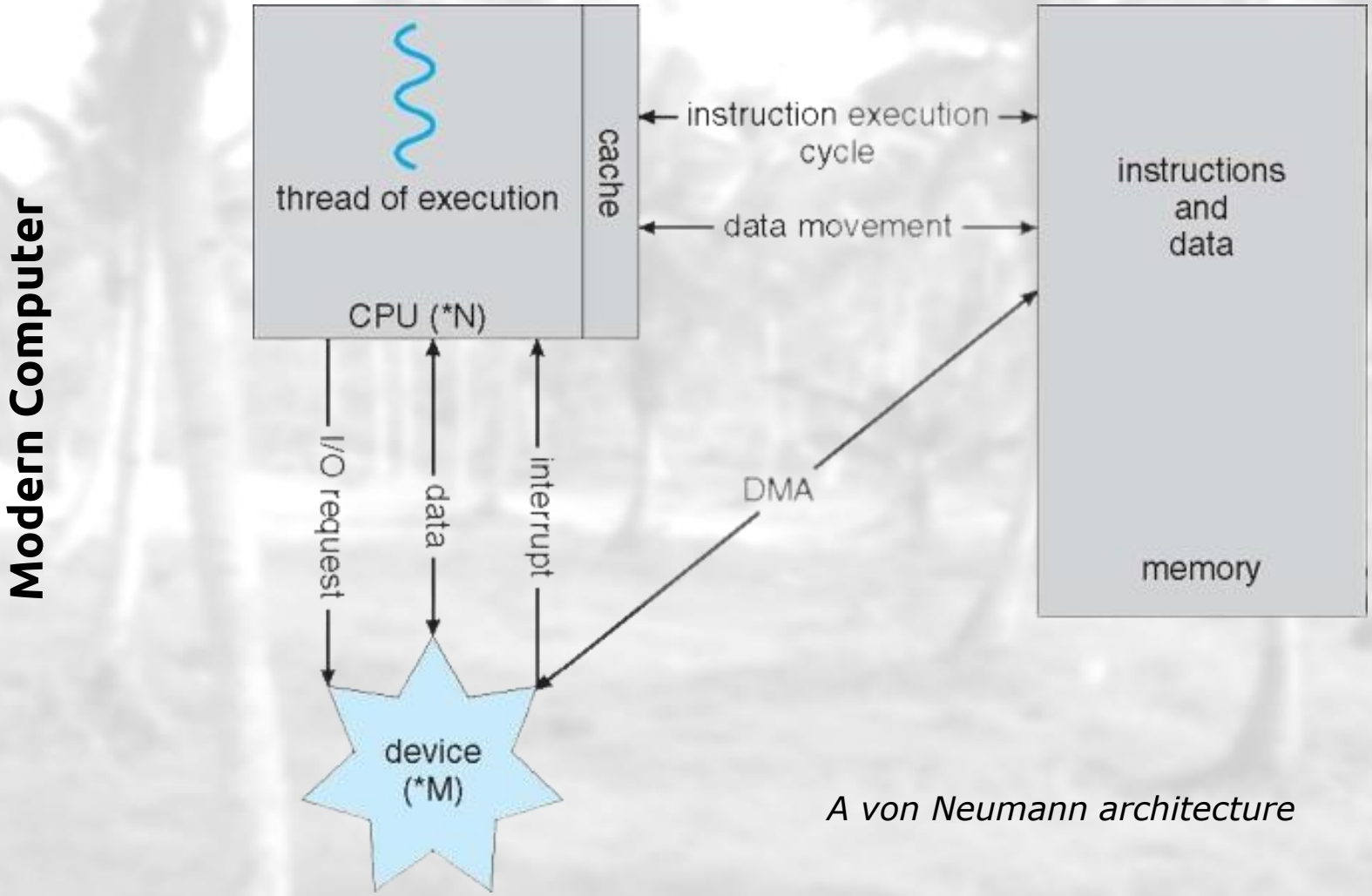
Direct Memory Access (DMA) Structure

- Used for high-speed I/O devices able to transmit information at close to memory speeds.
- Device controller transfers blocks of data from buffer storage *directly to main memory without CPU intervention*.
- Only *one interrupt is generated per block*, rather than the one interrupt per byte.

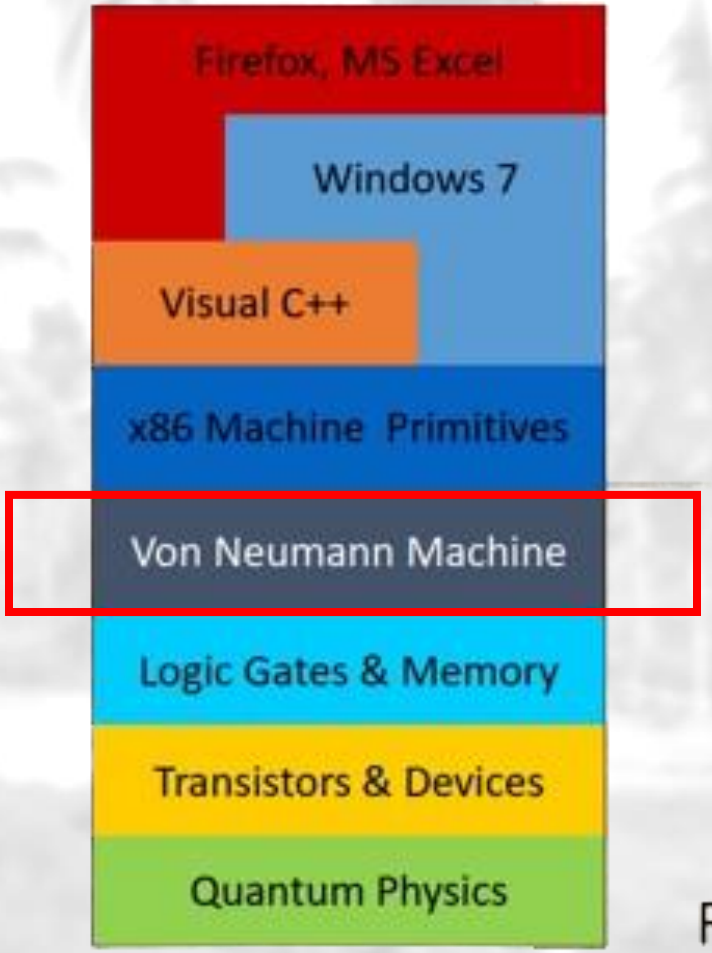
Old Structure: *I/O* → *CPU* → *Memory*

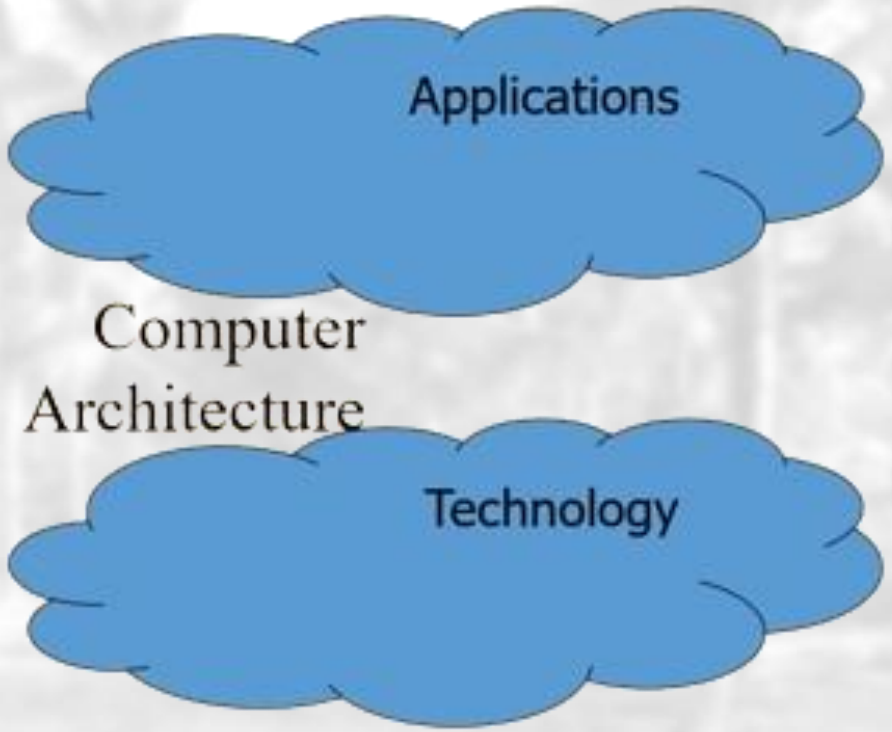
Modern Structure: *I/O* → *Memory*

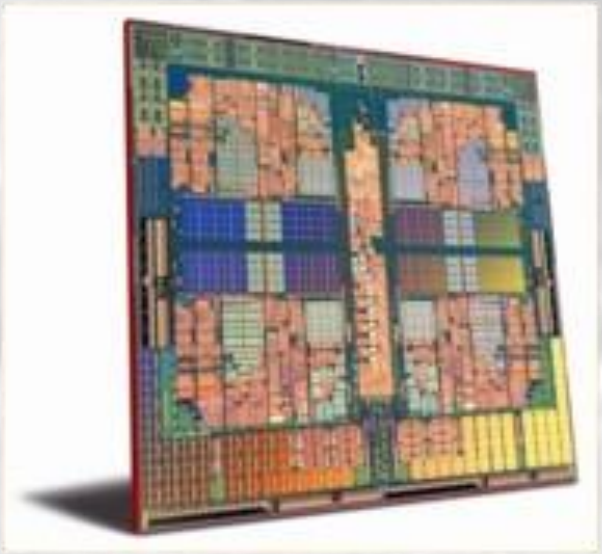
Direct Memory Access (DMA) Structure



A von Neumann architecture







Rely on *abstraction layers* to manage complexity •

Von Neumann Machine •

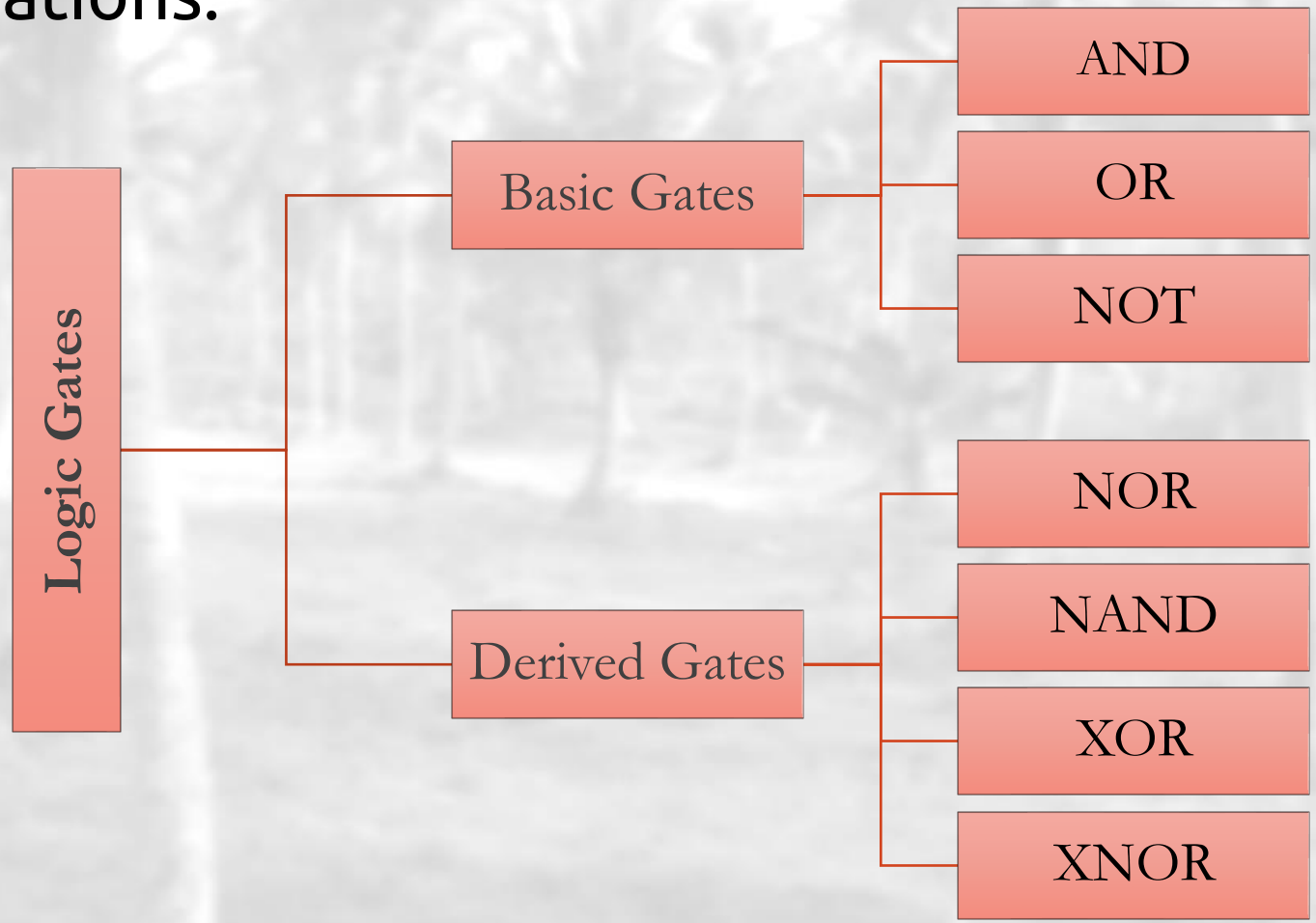
Binary Logic

Binary Logic – Definition

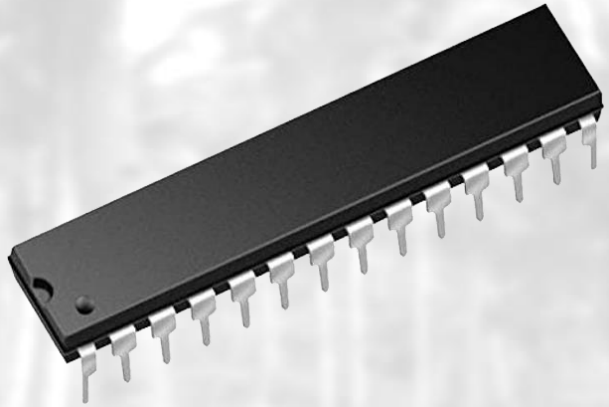
- Binary logic consists of binary variables and a set of logical operations.
- Each variable having two and only two distinct possible values: 1 and 0.
- Three Basic logical operations: AND, OR, and NOT.

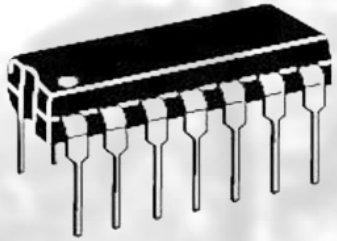
Binary Logic – Definition

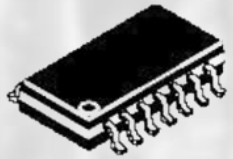
- Binary logic consists of binary variables and a set of logical operations.

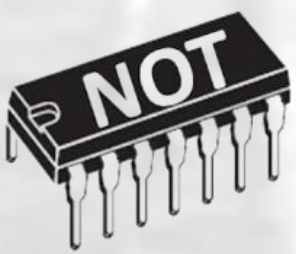


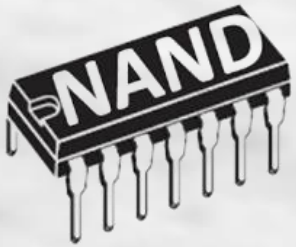
How do they look ?

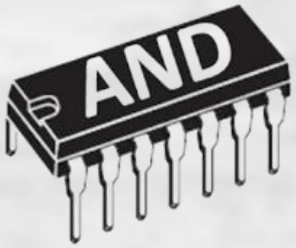


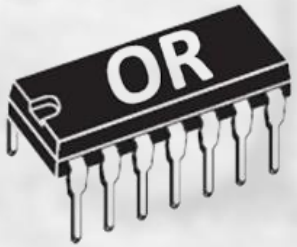


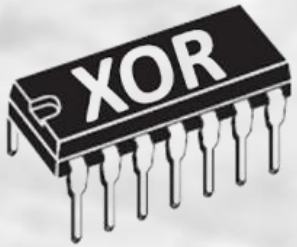













crucial
by Micron
8BB12
D9HXT
8BB12
D9HXT
47J6
4G85
4G85
4G85

Storage Structure

Storage Definitions and Notation Review

- The basic unit of computer storage is the **bit** (**bi**nary digi**t**).
- Computer Storage:
  - A byte is 8 bits
  - A kilobyte (KB) is $1{,}024^1$ (2^{10}) bytes
  - A megabyte (MB) is $1{,}024^2$ (2^{20}) bytes
  - A gigabyte (GB) is $1{,}024^3$ (2^{30}) bytes
  - A terabyte (TB) is $1{,}024^4$ (2^{40}) bytes
  - A petabyte (PB) is $1{,}024^5$ (2^{50}) bytes
- Given enough bits a computer can represent:
  - numbers, letters, images, sounds, documents … etc.

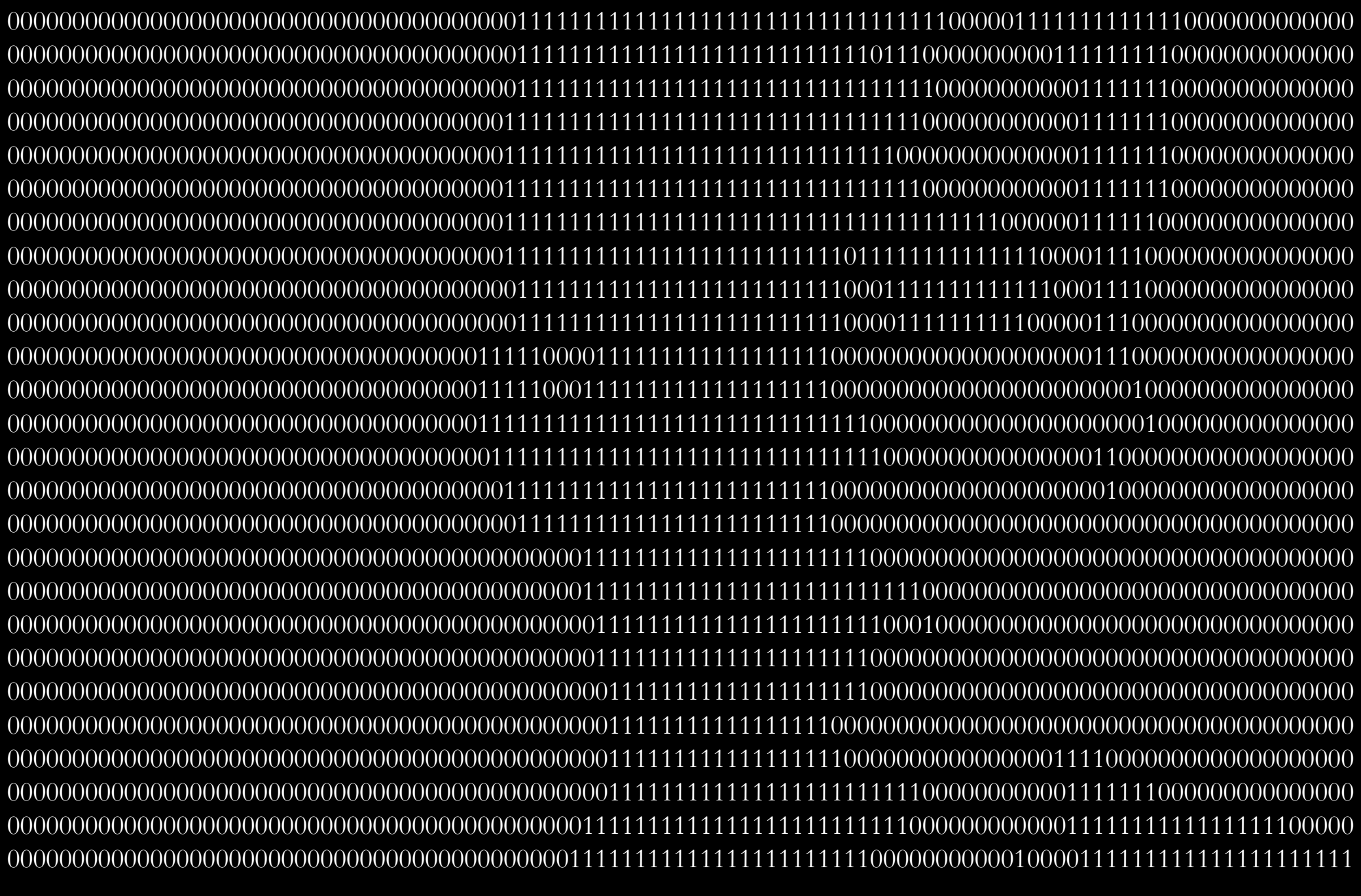
Character '**A**'

01000001

ASCII Table

Storage Definitions and Notation Review (Cont.)

Memory Unit	Description
Bit	0 or 1
Byte	8 bits
KiloByte (KB)	1024 Bytes
MegaByte (MB)	1024 KB
GigaByte (GB)	1024 MB
TeraByte (TB)	1024 GB
PetaByte (PB)	1024 TB
HexaByte or exaByte (EB)	1024 PB
ZettaByte (ZB)	1024 EB
YottaByte (YB)	1024 ZB
BrontoByte	1024 YB
GeopByte	1024 Bronto Bytes



ASCII TABLE

Decimal	Hexadecimal	Binary	Octal	Char
0	0	0	0	*[NULL]*
1	1	1	1	*[START OF HEADING]*
2	2	10	2	*[START OF TEXT]*
3	3	11	3	*[END OF TEXT]*
4	4	100	4	*[END OF TRANSMISSION]*
5	5	101	5	*[ENQUIRY]*
6	6	110	6	*[ACKNOWLEDGE]*
7	7	111	7	*[BELL]*
8	8	1000	10	*[BACKSPACE]*
9	9	1001	11	*[HORIZONTAL TAB]*
10	A	1010	12	*[LINE FEED]*
11	B	1011	13	*[VERTICAL TAB]*
12	C	1100	14	*[FORM FEED]*
13	D	1101	15	*[CARRIAGE RETURN]*
14	E	1110	16	*[SHIFT OUT]*
15	F	1111	17	*[SHIFT IN]*
16	10	10000	20	*[DATA LINK ESCAPE]*
17	11	10001	21	*[DEVICE CONTROL 1]*
18	12	10010	22	*[DEVICE CONTROL 2]*
19	13	10011	23	*[DEVICE CONTROL 3]*
20	14	10100	24	*[DEVICE CONTROL 4]*
21	15	10101	25	*[NEGATIVE ACKNOWLEDGE]*
22	16	10110	26	*[SYNCHRONOUS IDLE]*
23	17	10111	27	*[END OF TRANS. BLOCK]*
24	18	11000	30	*[CANCEL]*
25	19	11001	31	*[END OF MEDIUM]*
26	1A	11010	32	*[SUBSTITUTE]*
27	1B	11011	33	*[ESCAPE]*
28	1C	11100	34	*[FILE SEPARATOR]*
29	1D	11101	35	*[GROUP SEPARATOR]*
30	1E	11110	36	*[RECORD SEPARATOR]*
31	1F	11111	37	*[UNIT SEPARATOR]*
32	20	100000	40	*[SPACE]*
33	21	100001	41	!
34	22	100010	42	"
35	23	100011	43	#
36	24	100100	44	$
37	25	100101	45	%
38	26	100110	46	&
39	27	100111	47	'
40	28	101000	50	(
41	29	101001	51	)
42	2A	101010	52	*
43	2B	101011	53	+
44	2C	101100	54	,
45	2D	101101	55	-
46	2E	101110	56	.
47	2F	101111	57	/
48	30	110000	60	0
49	31	110001	61	1
50	32	110010	62	2
51	33	110011	63	3
52	34	110100	64	4
53	35	110101	65	5
54	36	110110	66	6
55	37	110111	67	7
56	38	111000	70	8
57	39	111001	71	9
58	3A	111010	72	:
59	3B	111011	73	;
60	3C	111100	74	<
61	3D	111101	75	=
62	3E	111110	76	>
63	3F	111111	77	?
64	40	1000000	100	@
65	41	1000001	101	A
66	42	1000010	102	B
67	43	1000011	103	C
68	44	1000100	104	D
69	45	1000101	105	E
70	46	1000110	106	F
71	47	1000111	107	G
72	48	1001000	110	H
73	49	1001001	111	I
74	4A	1001010	112	J
75	4B	1001011	113	K
76	4C	1001100	114	L
77	4D	1001101	115	M
78	4E	1001110	116	N
79	4F	1001111	117	O
80	50	1010000	120	P
81	51	1010001	121	Q
82	52	1010010	122	R
83	53	1010011	123	S
84	54	1010100	124	T
85	55	1010101	125	U
86	56	1010110	126	V
87	57	1010111	127	W
88	58	1011000	130	X
89	59	1011001	131	Y
90	5A	1011010	132	Z
91	5B	1011011	133	[
92	5C	1011100	134	\
93	5D	1011101	135	]
94	5E	1011110	136	^
95	5F	1011111	137	_
96	60	1100000	140	`
97	61	1100001	141	a
98	62	1100010	142	b
99	63	1100011	143	c
100	64	1100100	144	d
101	65	1100101	145	e
102	66	1100110	146	f
103	67	1100111	147	g
104	68	1101000	150	h
105	69	1101001	151	i
106	6A	1101010	152	j
107	6B	1101011	153	k
108	6C	1101100	154	l
109	6D	1101101	155	m
110	6E	1101110	156	n
111	6F	1101111	157	o
112	70	1110000	160	p
113	71	1110001	161	q
114	72	1110010	162	r
115	73	1110011	163	s
116	74	1110100	164	t
117	75	1110101	165	u
118	76	1110110	166	v
119	77	1110111	167	w
120	78	1111000	170	x
121	79	1111001	171	y
122	7A	1111010	172	z
123	7B	1111011	173	{
124	7C	1111100	174	|
125	7D	1111101	175	}
126	7E	1111110	176	~
127	7F	1111111	177	*[DEL]*

Binary Coding Schemes

Binary Coding Schemes

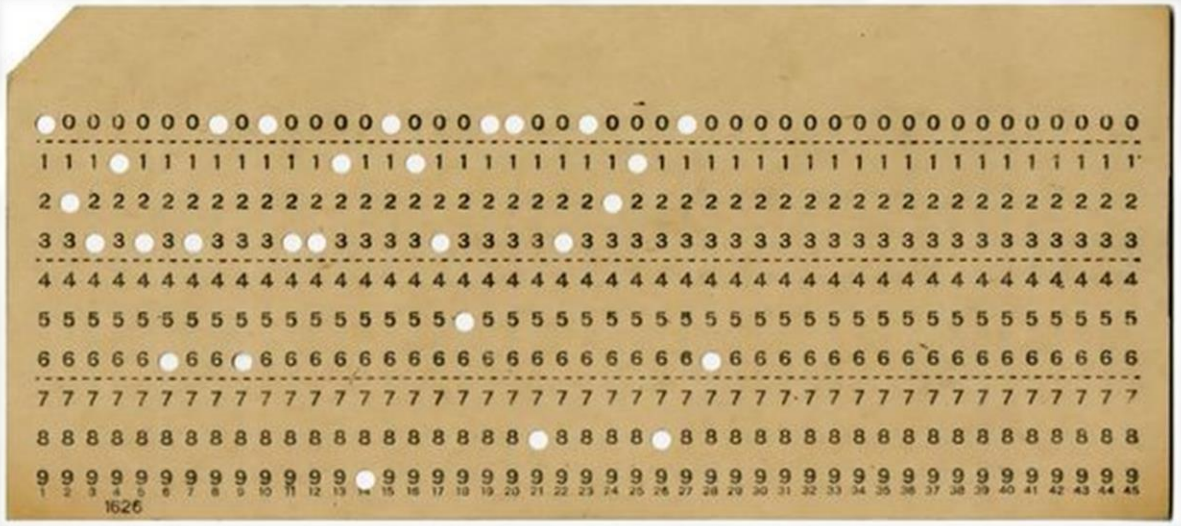
- Combination of bits are used to represent:
  - alphabetic data
  - numeric data
  - Symbols
  - sound and video data
- Binary Coding schemes are used for this purpose
  - EBCDIC (Extended Binary Coded Decimal Interchange Code)
  - ASCII (American Standard Code for Information Interchange)
  - Unicode (UTF-8)

ASCII Code: Character to Binary

0	0011 0000	O	0100 1111	m	0110 1101
1	0011 0001	P	0101 0000	n	0110 1110
2	0011 0010	Q	0101 0001	o	0110 1111
3	0011 0011	R	0101 0010	p	0111 0000
4	0011 0100	S	0101 0011	q	0111 0001
5	0011 0101	T	0101 0100	r	0111 0010
6	0011 0110	U	0101 0101	s	0111 0011
7	0011 0111	V	0101 0110	t	0111 0100
8	0011 1000	W	0101 0111	u	0111 0101
9	0011 1001	X	0101 1000	v	0111 0110
A	0100 0001	Y	0101 1001	w	0111 0111
B	0100 0010	Z	0101 1010	x	0111 1000
C	0100 0011	a	0110 0001	y	0111 1001
D	0100 0100	b	0110 0010	z	0111 1010
E	0100 0101	c	0110 0011	.	0010 1110
F	0100 0110	d	0110 0100	,	0010 0111
G	0100 0111	e	0110 0101	:	0011 1010
H	0100 1000	f	0110 0110	;	0011 1011
I	0100 1001	g	0110 0111	?	0011 1111
J	0100 1010	h	0110 1000	!	0010 0001
K	0100 1011	I	0110 1001	'	0010 1100
L	0100 1100	j	0110 1010	"	0010 0010
M	0100 1101	k	0110 1011	(	0010 1000
N	0100 1110	l	0110 1100	)	0010 1001
				space	0010 0000

Early Data Storage

- **Punched Card**
  - Early method of data storage used with early computers
  - Containing several punched holes that represents data



Storage Structure (Cont.)

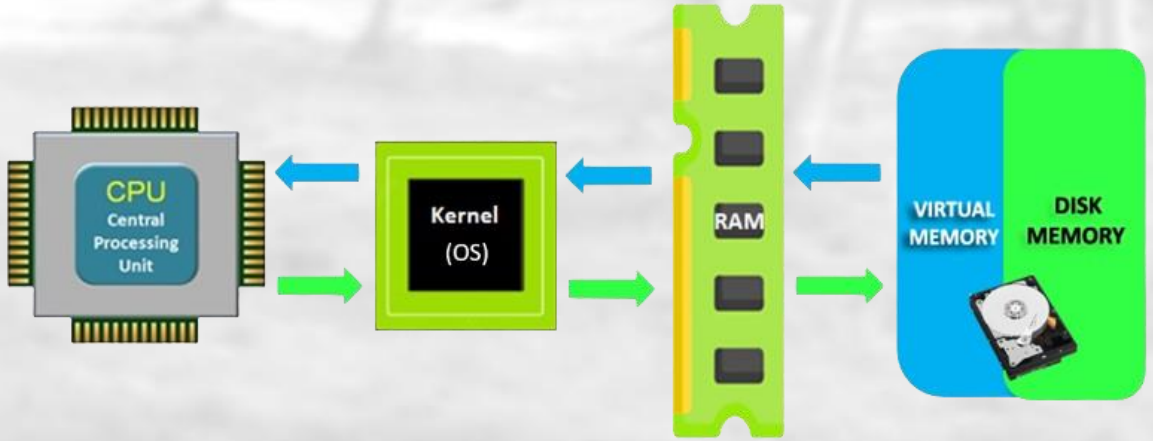
- Main memory - RAM (volatile)
  - Only large storage media that the CPU can access directly.
- Secondary storage (non-volatile)
  - Extension of main memory that provides large nonvolatile storage capacity.
- Tertiary Storage

Random Access Memory (RAM)

- Array of memory words (1 memory word = 8 bytes)
- Each byte has an address
- Memory Address:
  - Physical
  - Logical
- CPU Instructions:
  - **Load:** moves a word from main memory to CPU register
  - **Store:** move a word from CPU register to main memory

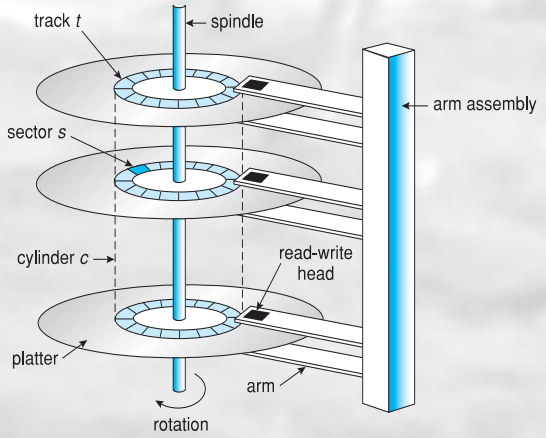
What is a Virtual Memory

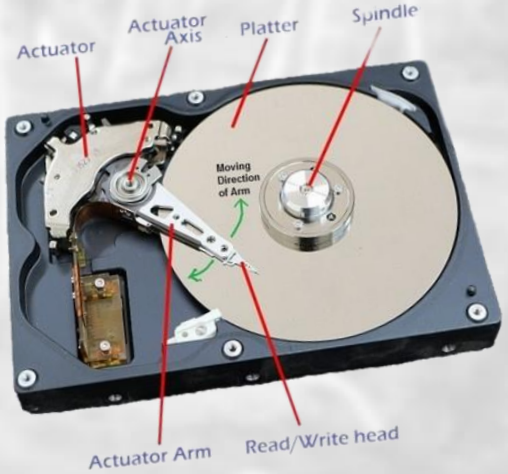
- Virtual Memory is a storage mechanism which offers user an illusion of having a very big main memory.
- This is done by treating a part of the secondary storage (i.e., HDD or SSD) as the main memory (RAM).
- It is a crucial part of modern operating systems by creating the illusion of having more memory than physically available, enabling multitasking and handling larger applications.



Magnetic Disk (Hard Disk Drive – HDD)

- Rigid metal or glass platters covered with magnetic recording material.
  - Disk surface is *logically* divided into tracks, which are subdivided into sectors.
  - The disk controller determines the logical interaction between the device and the computer.





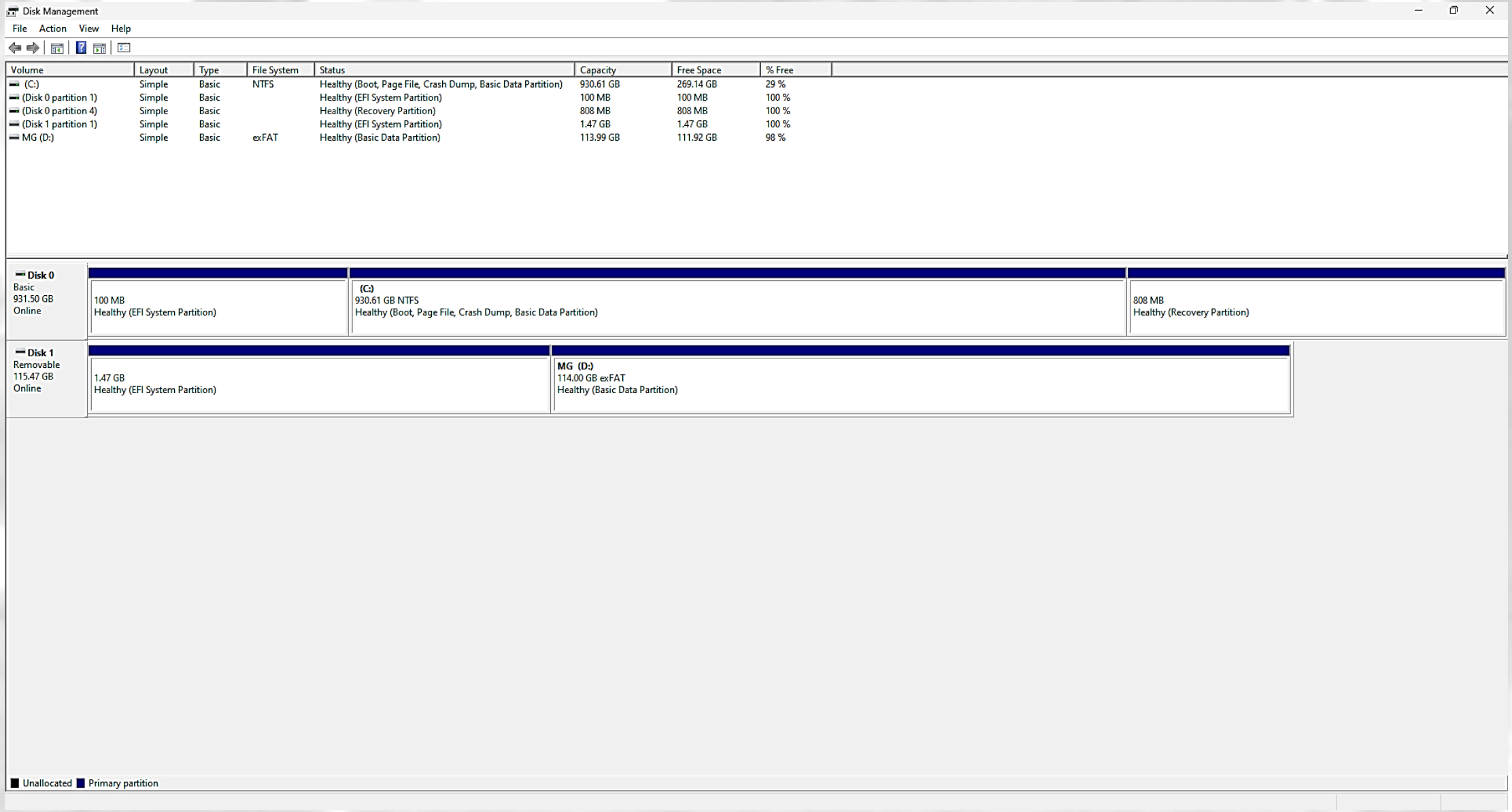
First Hard Disk Drive (HDD)

- IBM RAMDAC computer – 1956
- Solid-state drives (SSD) now

Tertiary Storage

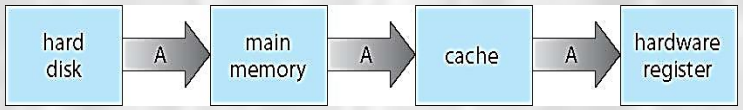
- Early secondary storage
- Slow access time
- Usage
  - Backup
  - Storage of infrequently used information
- Examples:
  - Magnetic tape
  - Optical disc (CDs)
  - Optical tapes (DVDs)
  - External Hard Drives (HDDs & SSDs)

An LTO-6 Tape drive with tape cartridge inserted

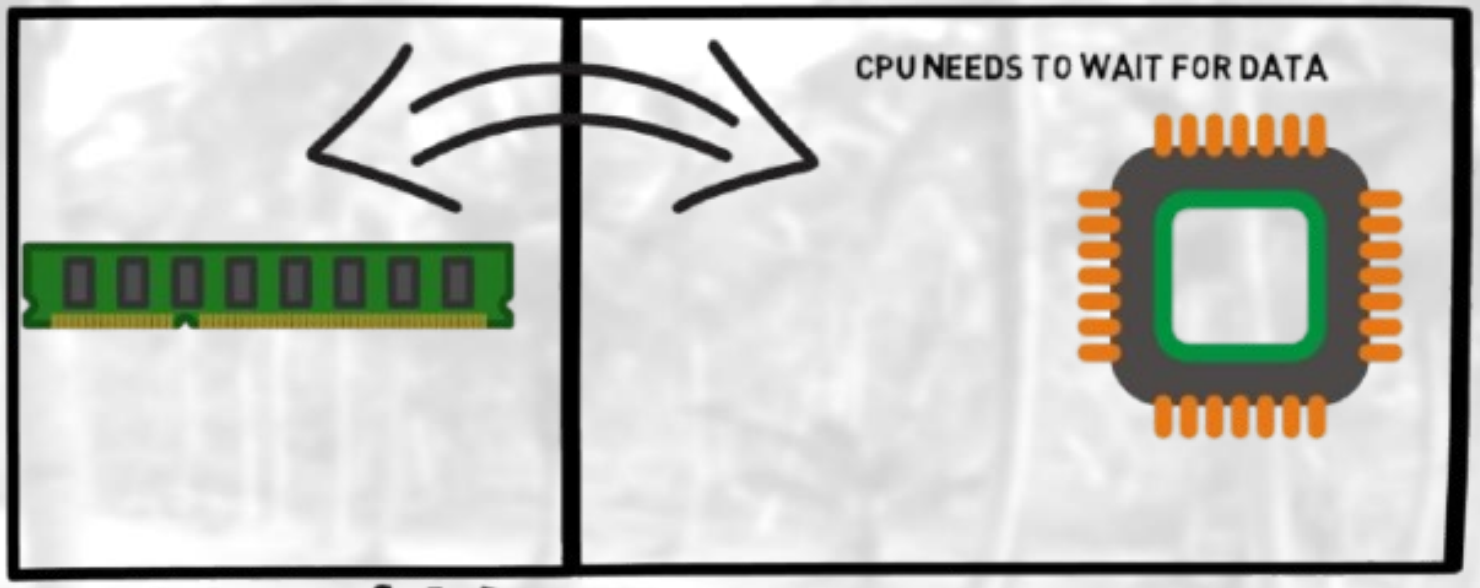

Disk Management
File Action View Help
Volume Layout Type File System Status Capacity Free Space % Free
(C:) Simple Basic NTFS Healthy (Boot, Page File, Crash Dump, Basic Data Partition) 930.61 GB 269.14 GB 29 %
(Disk 0 partition 1) Simple Basic Healthy (EFI System Partition) 100 MB 100 MB 100 %
(Disk 0 partition 4) Simple Basic Healthy (Recovery Partition) 808 MB 808 MB 100 %
(Disk 1 partition 1) Simple Basic Healthy (EFI System Partition) 1.47 GB 1.47 GB 100 %
MG (D:) Simple Basic exFAT Healthy (Basic Data Partition) 113.99 GB 111.92 GB 98 %
Disk 0
Basic
931.50 GB
Online
100 MB
Healthy (EFI System Partition)
(C:)
930.61 GB NTFS
Healthy (Boot, Page File, Crash Dump, Basic Data Partition)
808 MB
Healthy (Recovery Partition)
Disk 1
Removable
115.47 GB
Online
1.47 GB
Healthy (EFI System Partition)
MG (D:)
114.00 GB exFAT
Healthy (Basic Data Partition)
Unallocated
Primary partition

Caching Concept

- Information in use copied *from slower to faster storage temporarily*
- Faster storage (cache) checked first to determine if information is there
  - If it is, information used directly from the cache (fast)
  - If not, data copied to cache and used there
- Cache is smaller than storage being cached
- Cache depends on locality (nearest storage to access).
- Multitasking environments depends on caching!

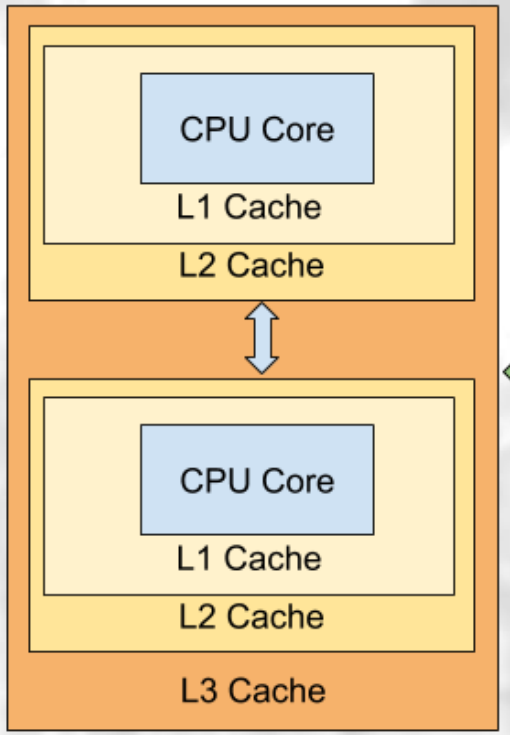


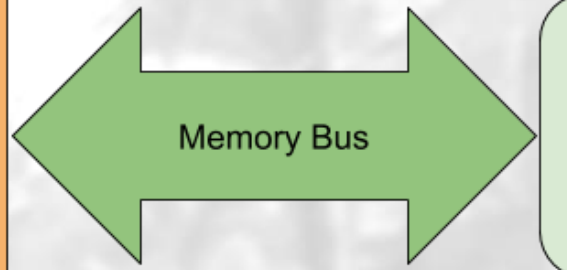
THE VON NEUMANN BOTTLENECK

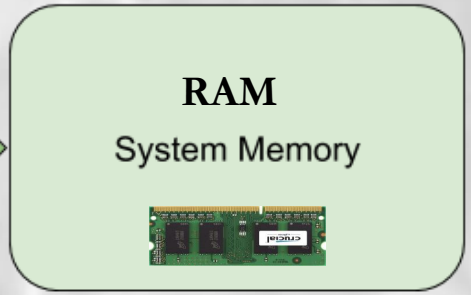


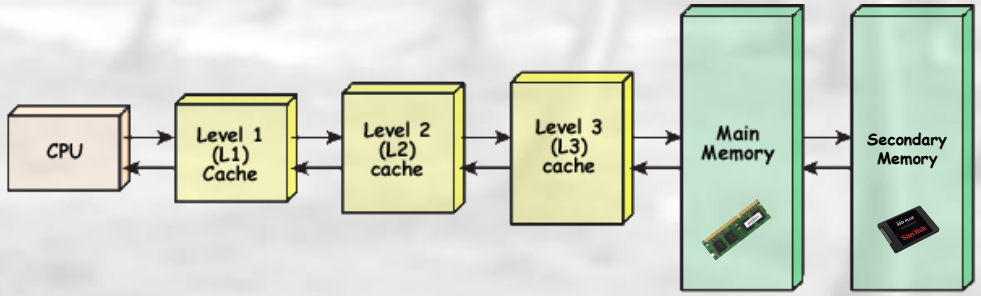
Solution


intel
Core™ i7

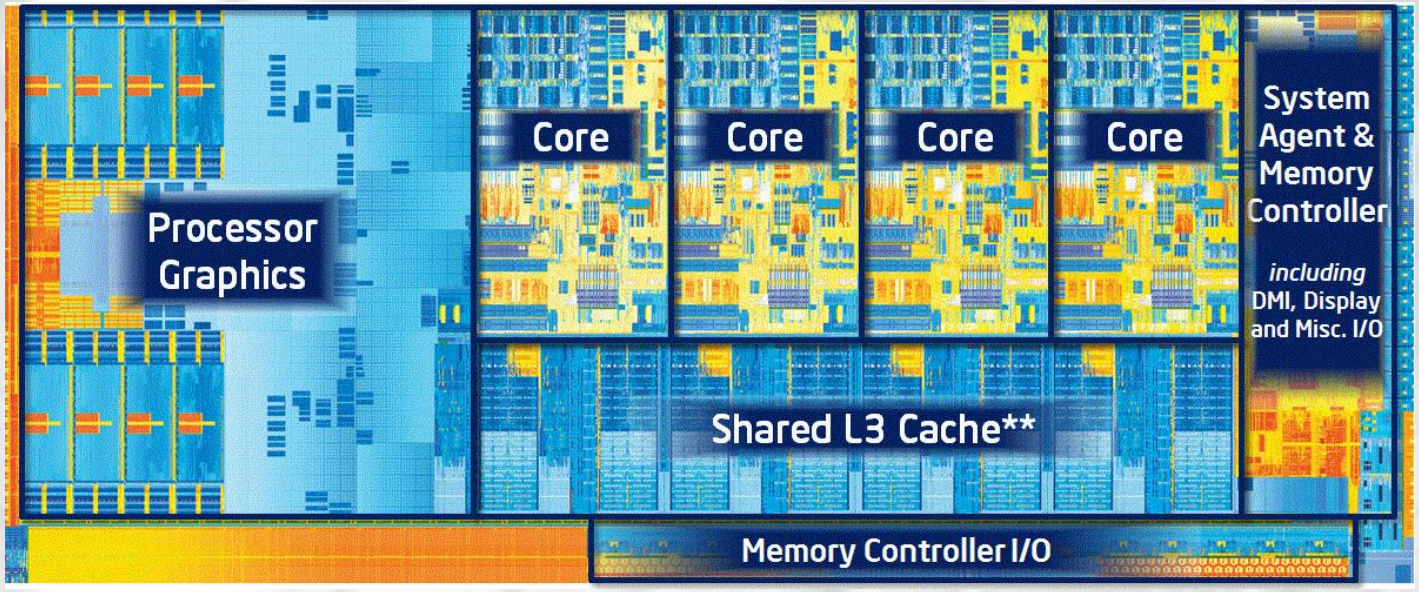

CPU Core
L1 Cache
L2 Cache
CPU Core
L1 Cache
L2 Cache
L3 Cache


Memory Bus

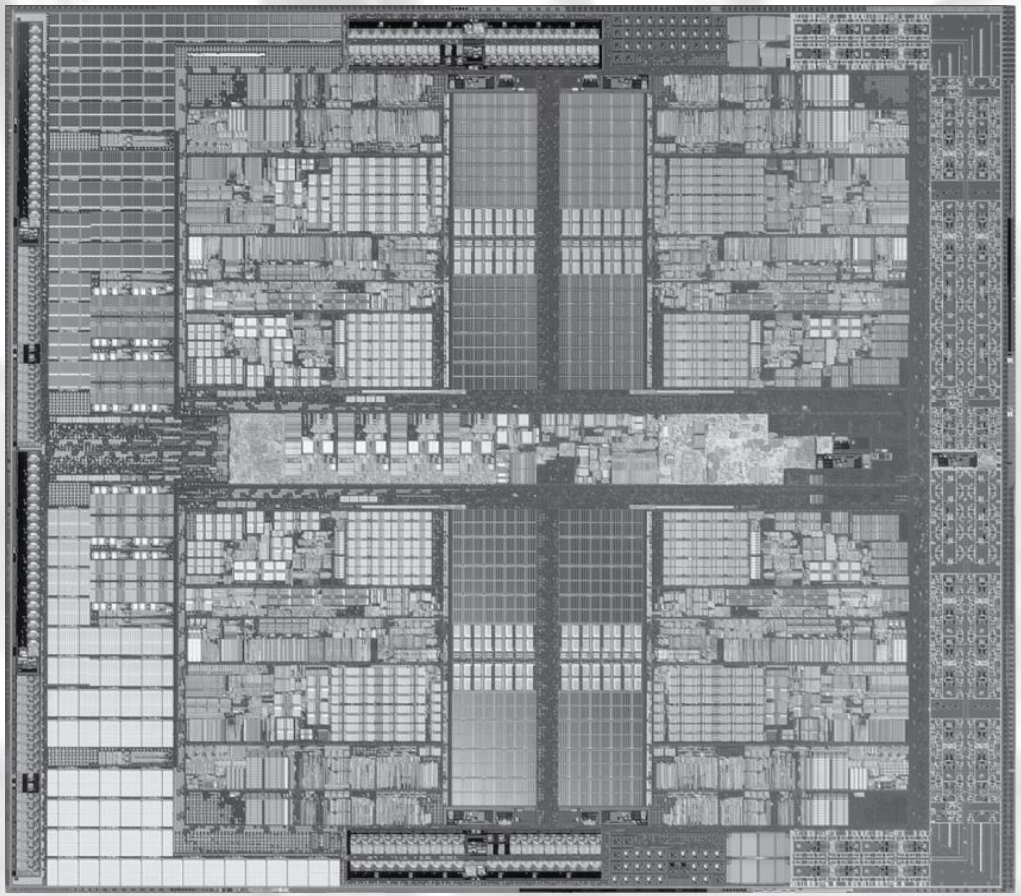

RAM
System Memory

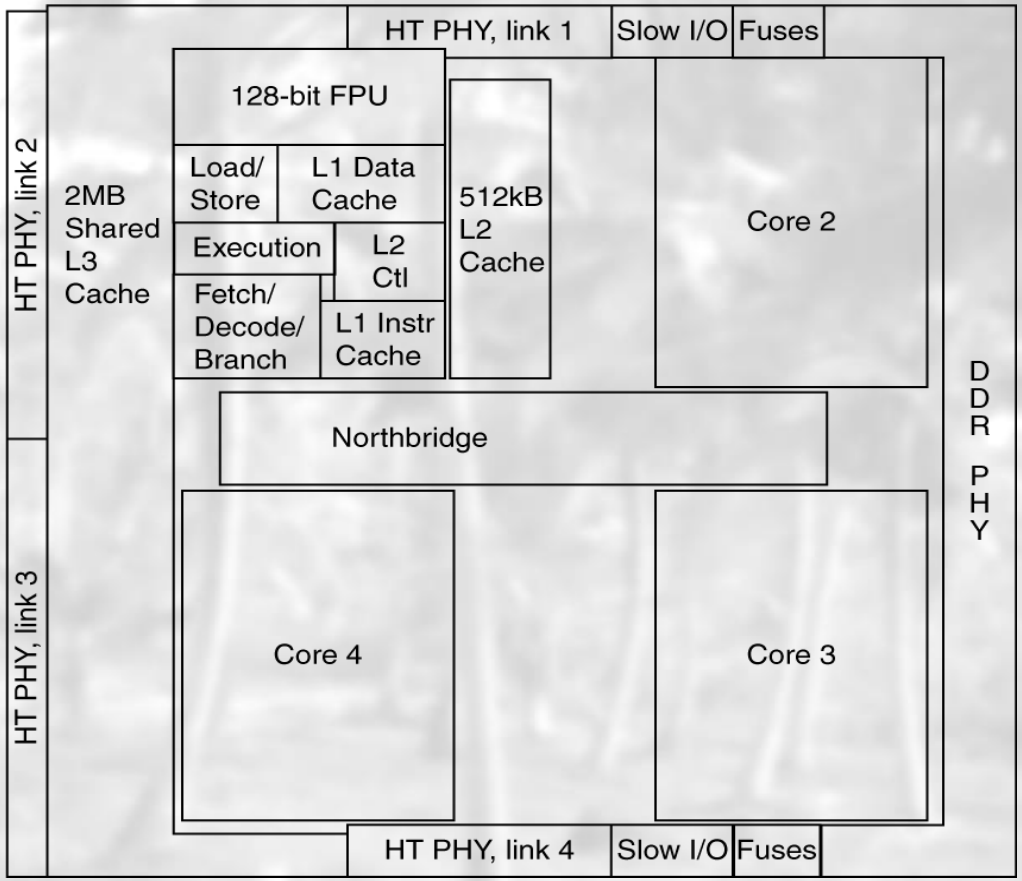

CPU
Level 1 (L1) Cache
Level 2 (L2) cache
Level 3 (L3) cache
Main Memory
Secondary Memory
FAST
Transfer speed
SLOW

A Peak inside the processor



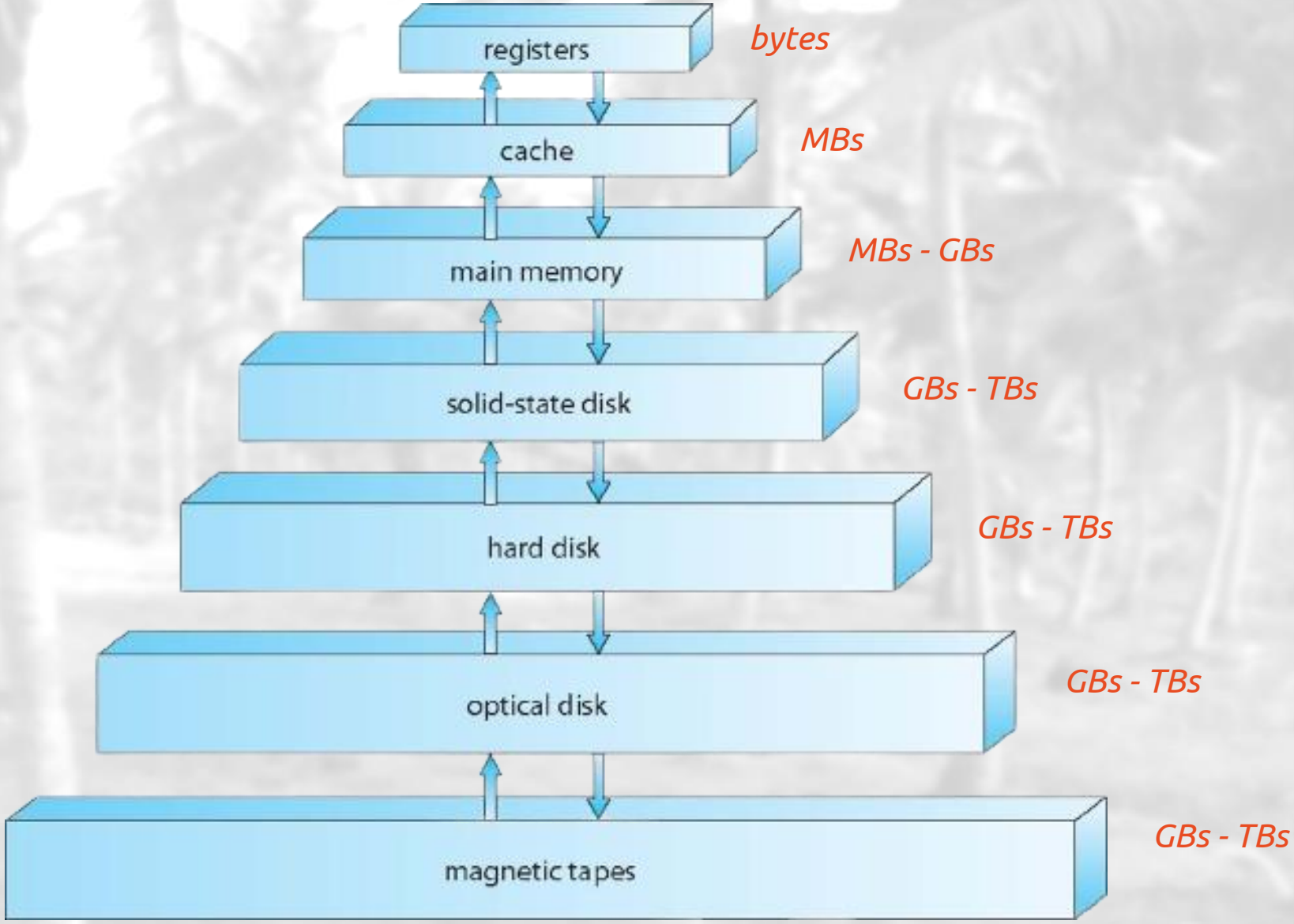
https://www.amd.com/en/products/processors/desktops/ryzen/5000-series/amd-ryzen-7-5800x.html





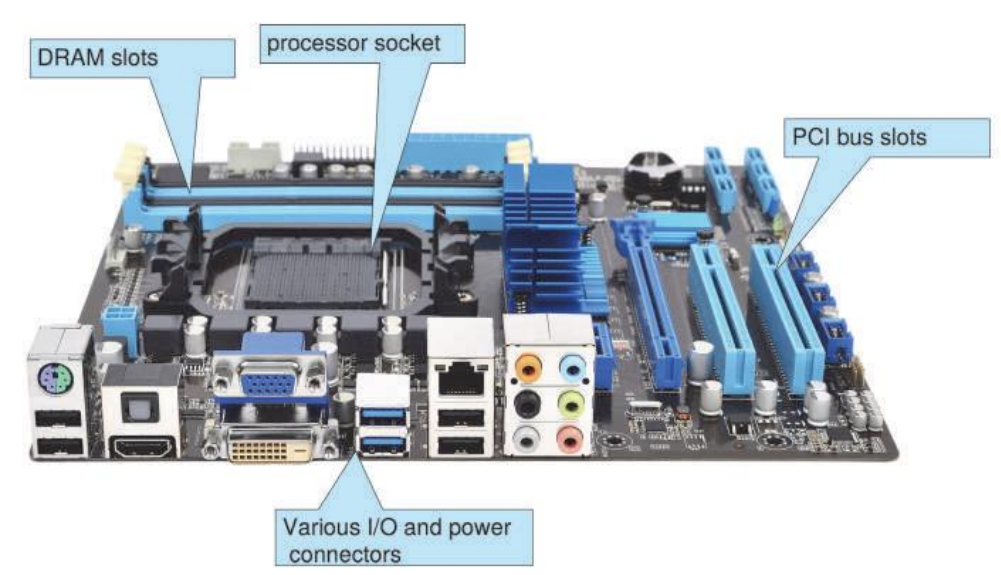
AMD Barcelona (4 processor cores)

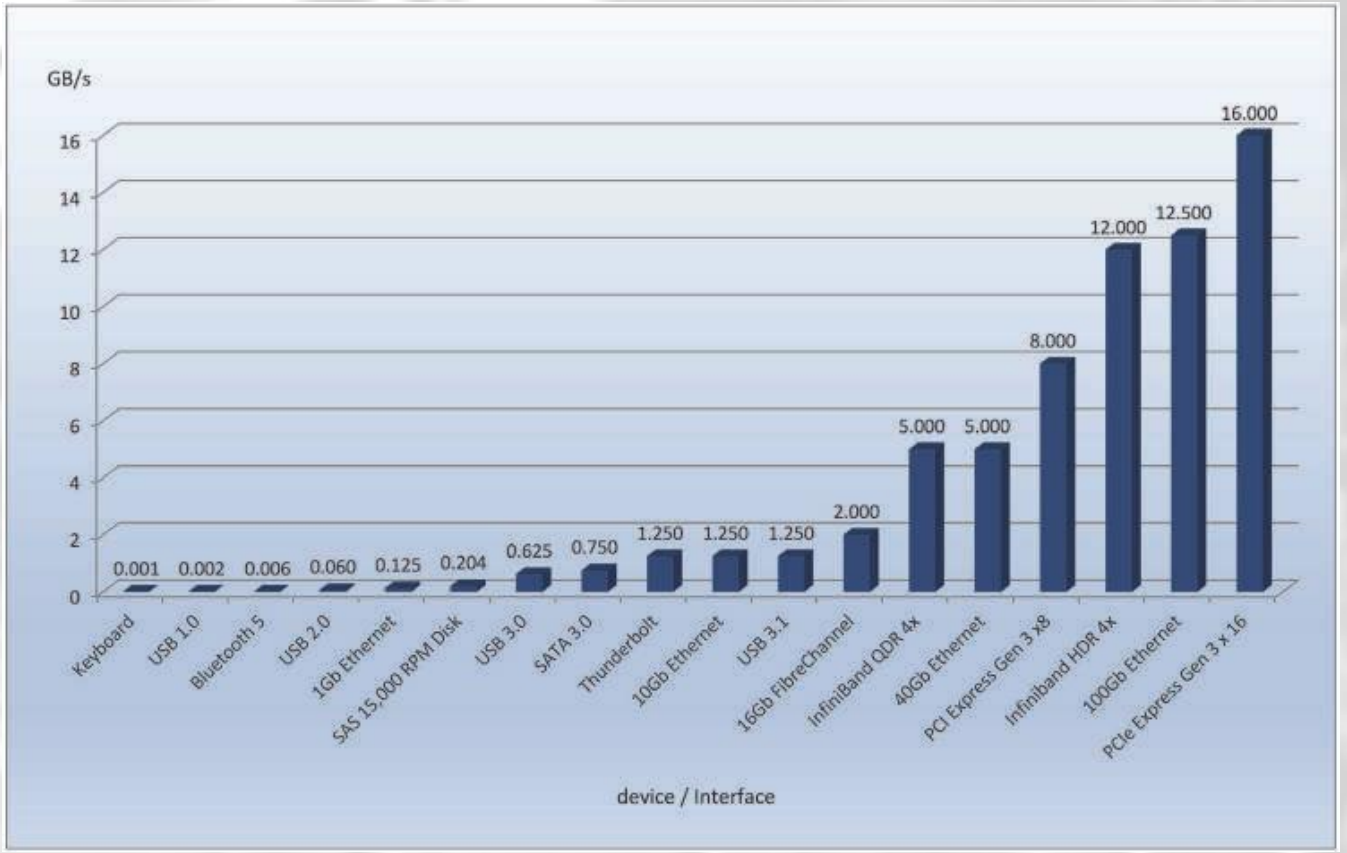
Storage-Device Hierarchy



Performance of Various Levels of Storage

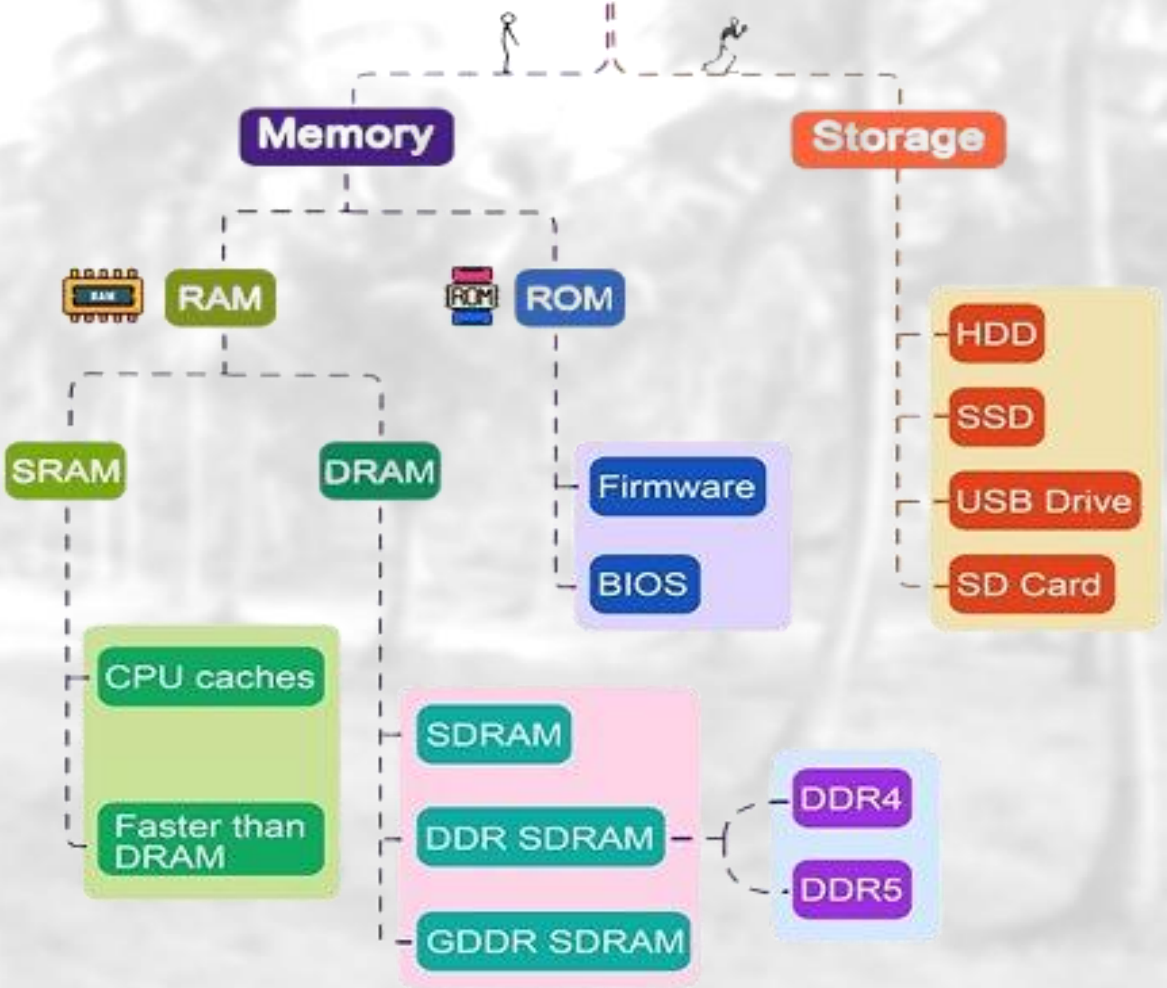
Level	1	2	3	4	5
Name	registers	cache	main memory	solid state disk	magnetic disk
Typical size	< 1 KB	< 16MB	< 64GB	< 1 TB	< 10 TB
Implementation technology	custom memory with multiple ports CMOS	on-chip or off-chip CMOS SRAM	CMOS SRAM	flash memory	magnetic disk
Access time (ns)	0.25 - 0.5	0.5 - 25	80 - 250	25,000 - 50,000	5,000,000
Bandwidth (MB/sec)	20,000 - 100,000	5,000 - 10,000	1,000 - 5,000	500	20 - 150
Managed by	compiler	hardware	operating system	operating system	operating system
Backed by	cache	main memory	disk	disk	disk or tape





Common PC and data-center I/O device and interface speeds.

Types of Memory and Storage



System Protection

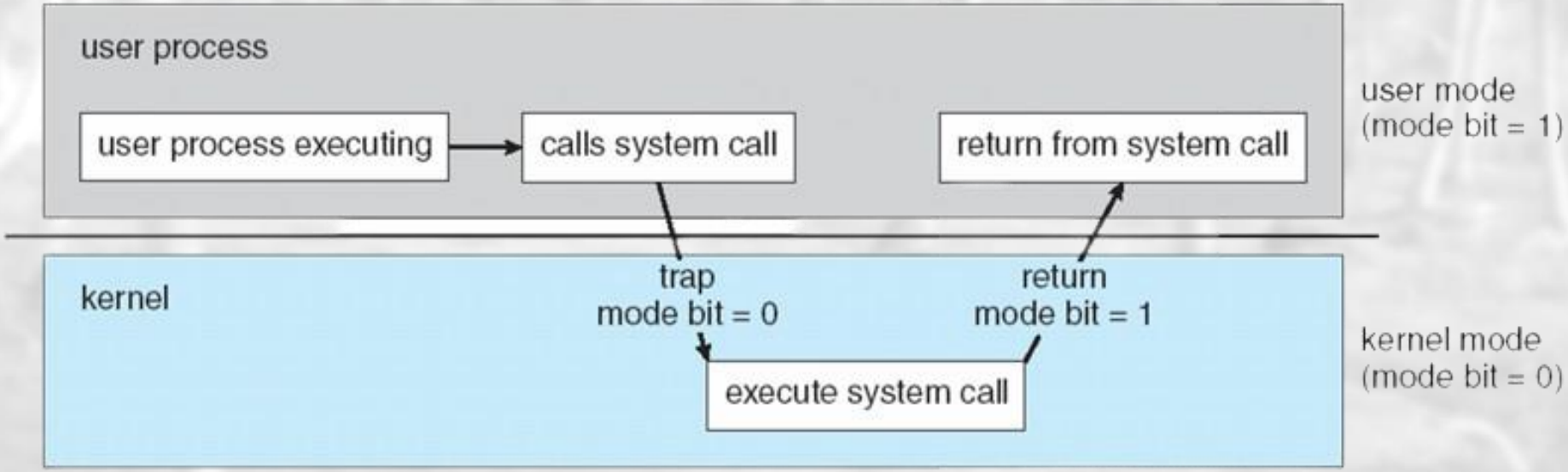
Hardware Dual-Mode

- Mode bit *provided by hardware* to indicate the current mode
  - User mode (bit 0)
  - kernel mode (bit 1)

- Provides ability to distinguish when system is running:
  - user code (Application Program - 0)
  - kernel code (System Program - 1)

Hardware Dual-Mode (Cont.)

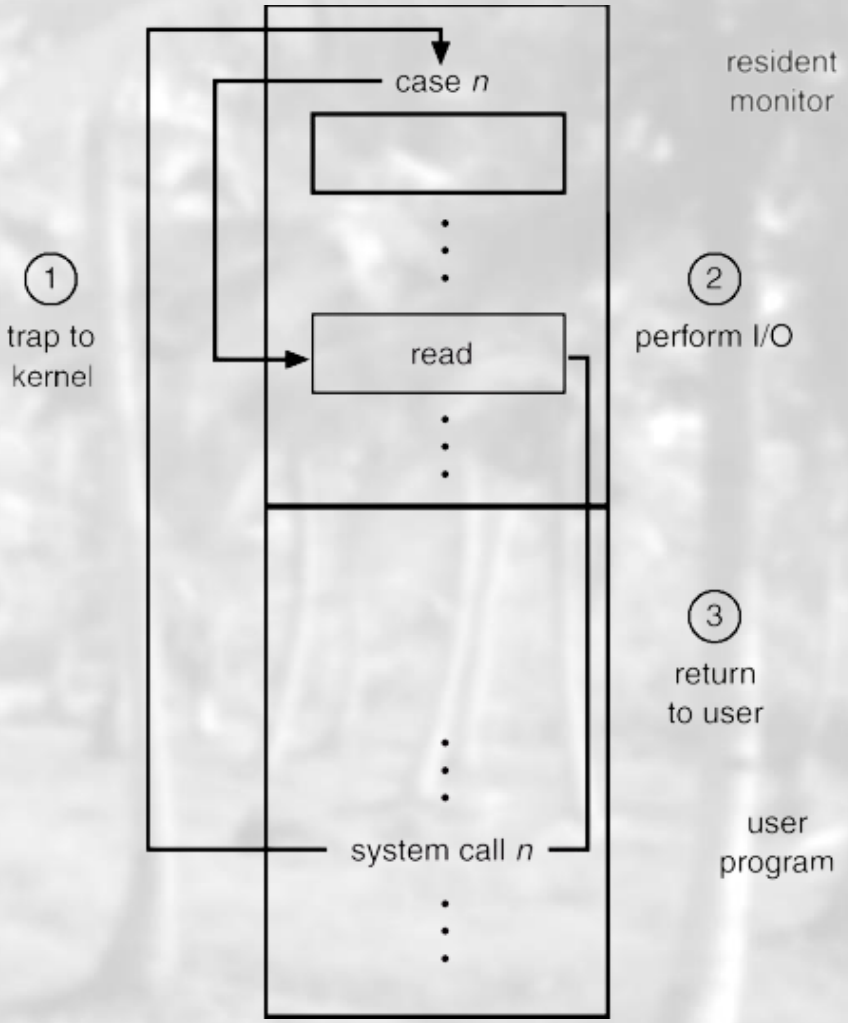
- Some instructions designated as privileged, and are only executable in kernel mode.
- System call changes mode to kernel, and the return from call resets it to user.



OS Operations

I) I/O Protection

- All I/O instructions are privileged
- I/O device registers are inaccessible by user
- Users cannot issue I/O instructions directly, they must do it through the OS

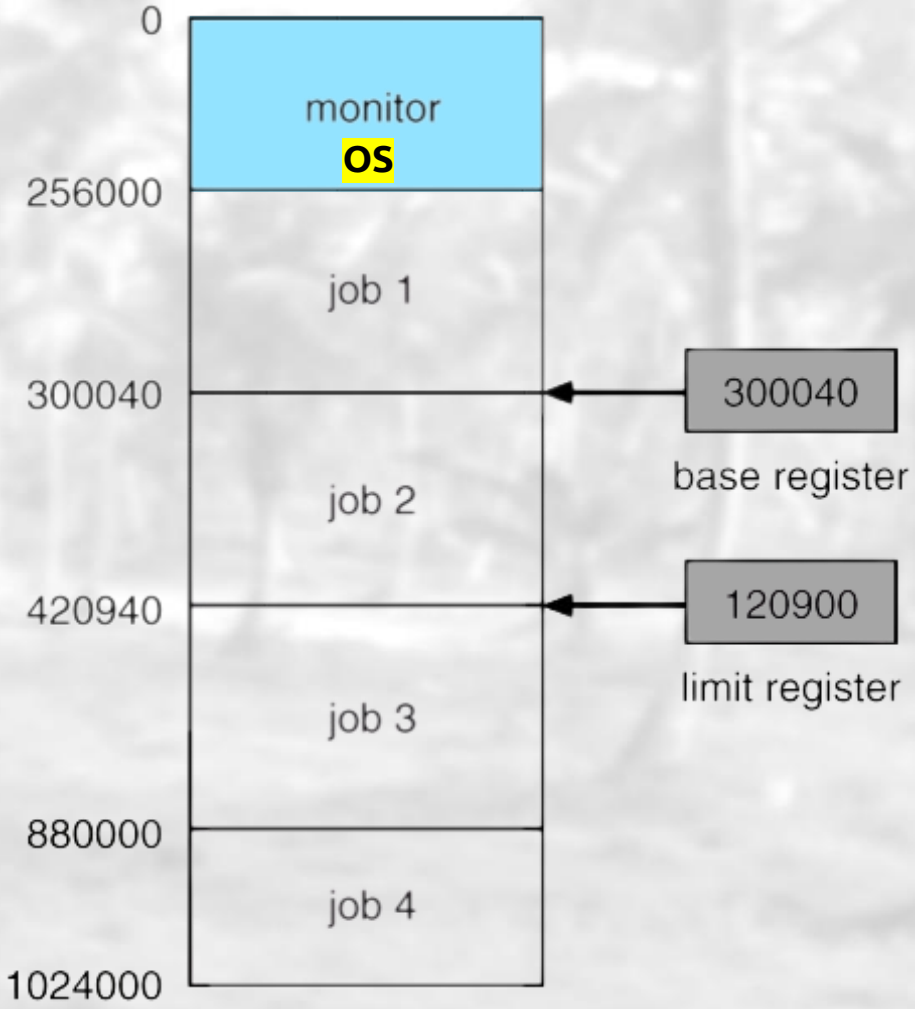


2) Memory Protection

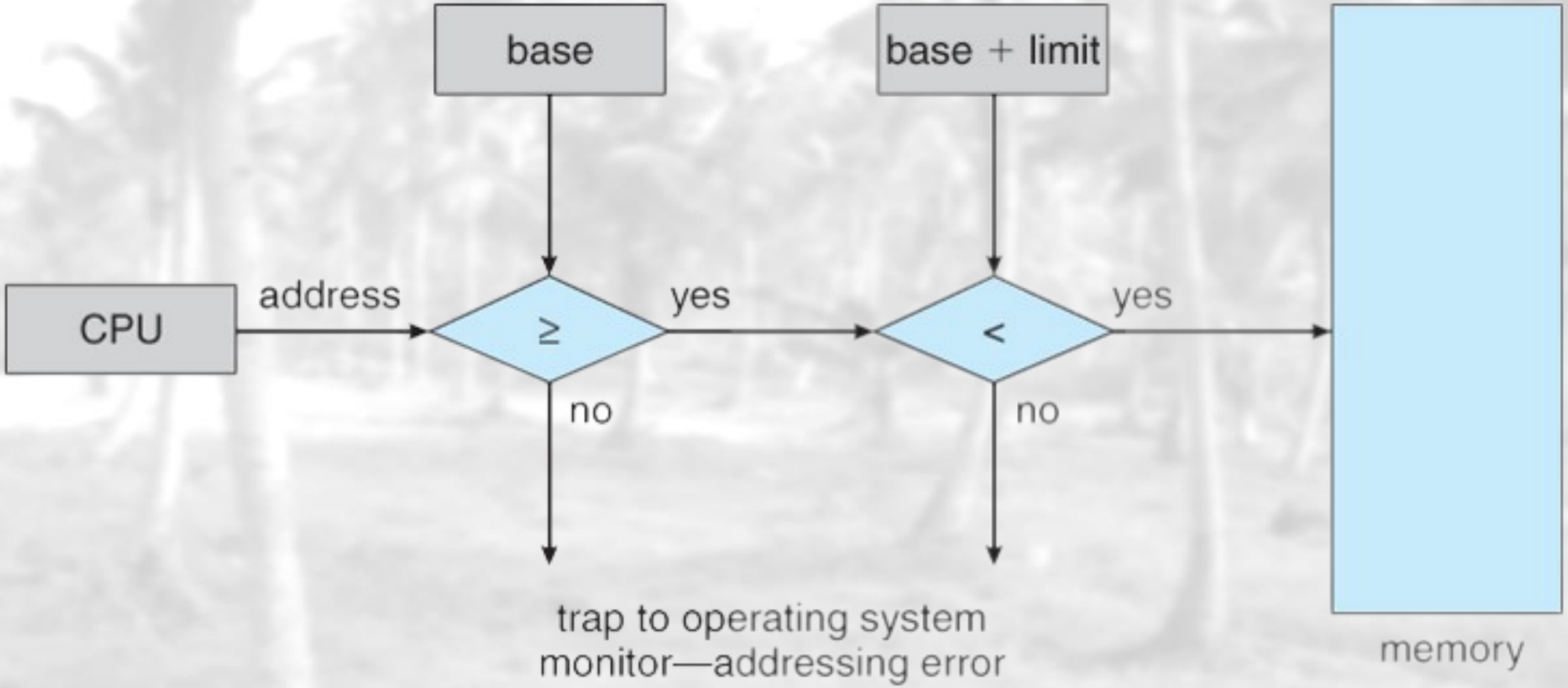
- Must provide memory protection at least for the interrupt vector and the interrupt service routines.

- In order to have memory protection:
  - add two registers that determine the range of legal addresses a program may access
    - ✓ **Base Register:** holds the smallest legal physical memory address.
    - ✓ **Limit Register:** contains the size of the range.
  - Memory outside the defined range is protected.

2) Memory Protection (Cont.)

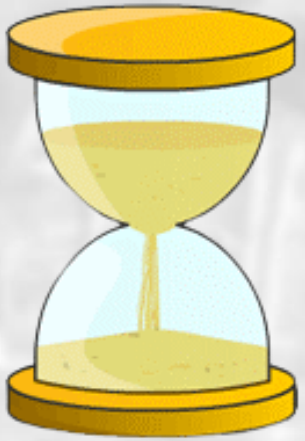


2) Memory Protection (Cont.)



3) CPU Protection (Cont.)

- Timers
  - interrupts CPU after specified period
- Timer is decremented every clock tick
- Use of timers
  - Time sharing systems
    - ✓ Interrupt every N milliseconds
    - ✓ Switch control to other processes
  - Computing current time

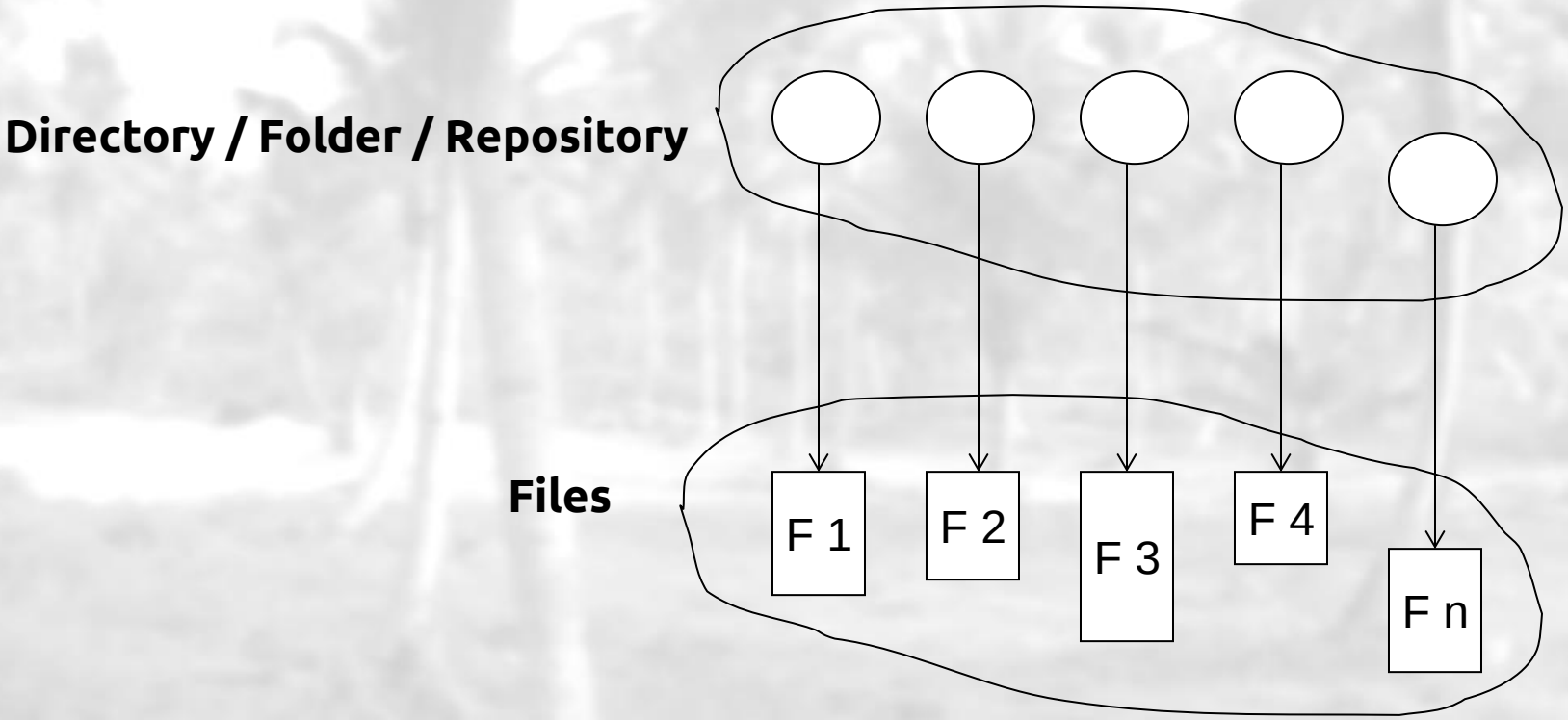


File Types – Name, Extension

File type	Extension	Function
executable	.exe	ready to run program
source code	.c .cpp .py .java … etc.	source code in various languages
Batch	.bat .sh	Commands to the CMD
Text	.txt .doc .docx .pdf .ppt .pptx	textual data and documents
Images	.png .jpg .jpeg .svg .webp .gif .heic	Viewing format
Audio	.mp3 .mp4 .wav .ogg .webm	songs and other audio related
Archive	.zip .rar .7z	compressed files

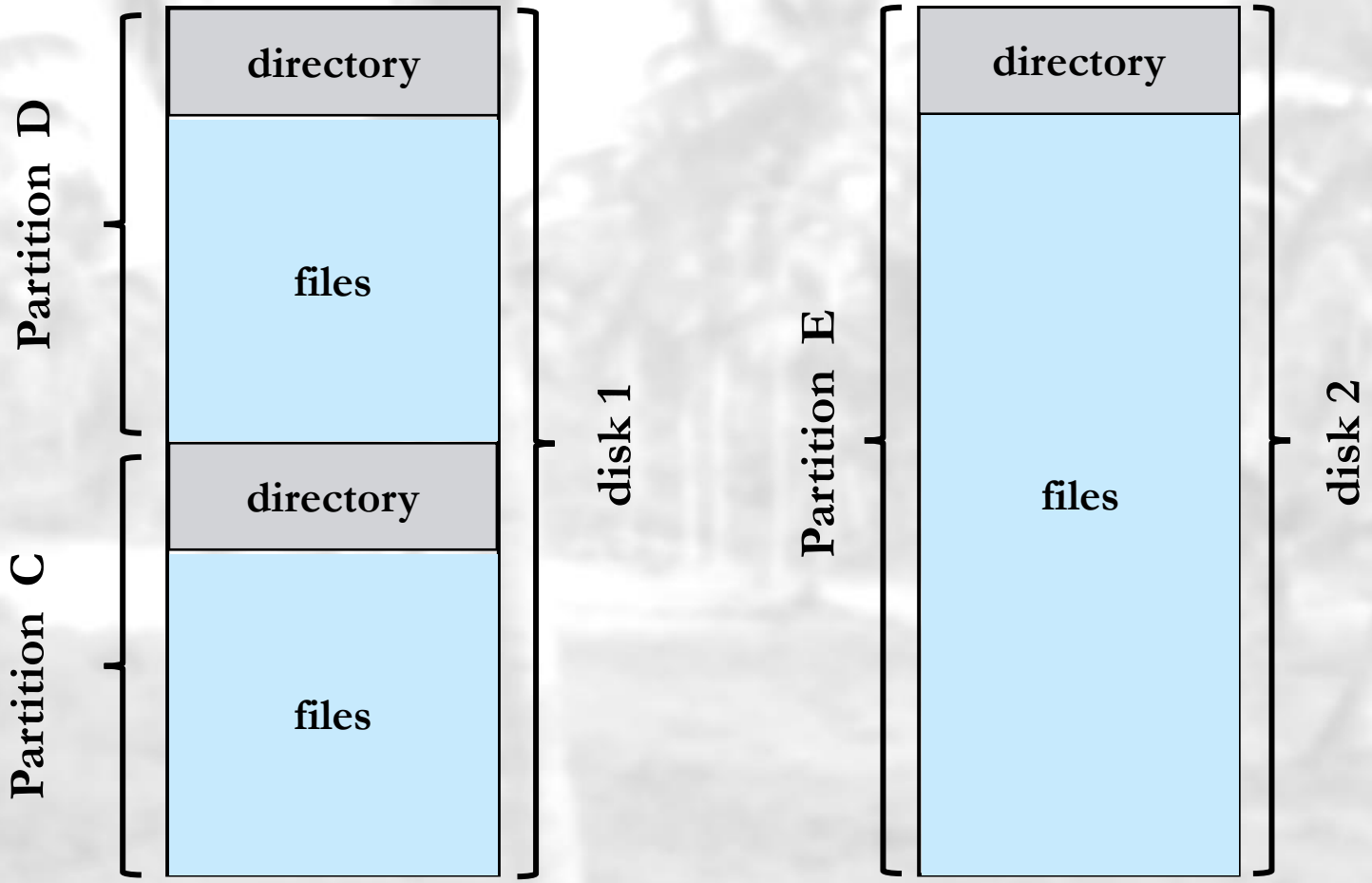
Directory Structure

- A collection of nodes containing information about all files



Both the directory structure and the files reside on disk

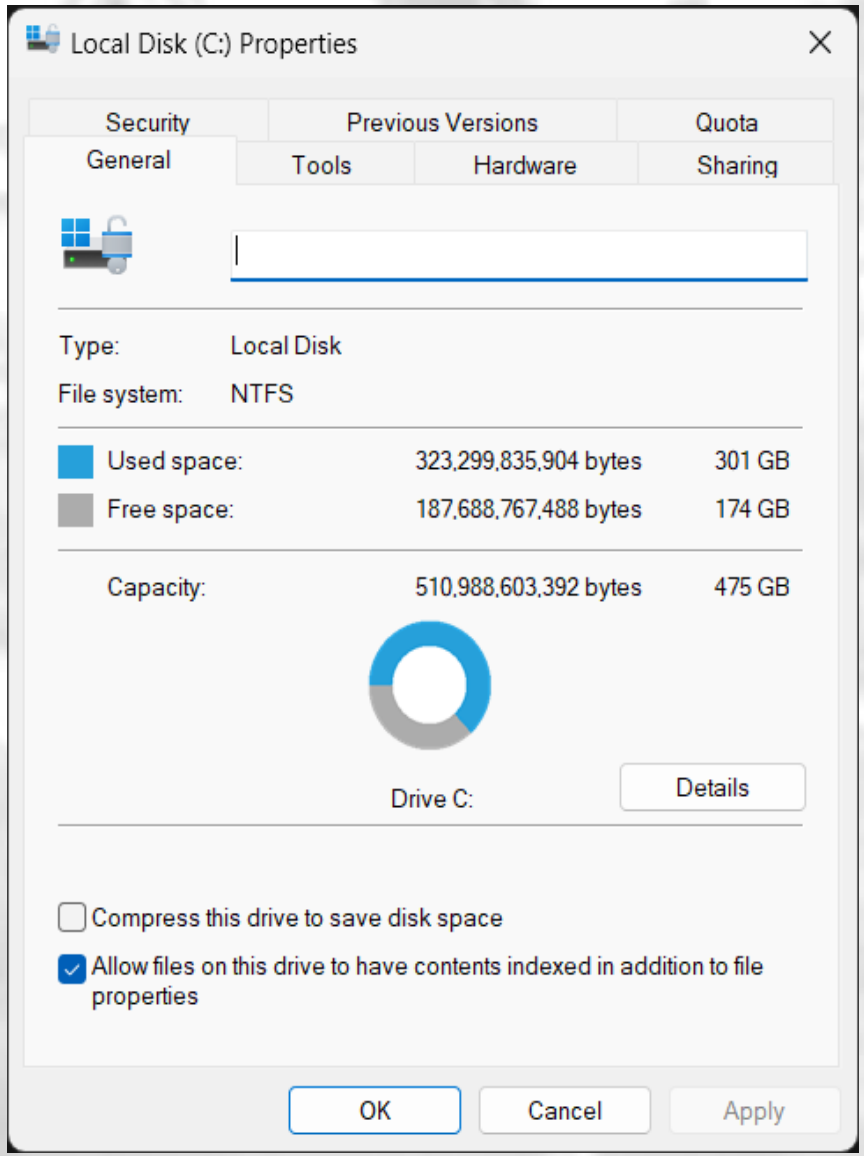
A Typical File-system Organization

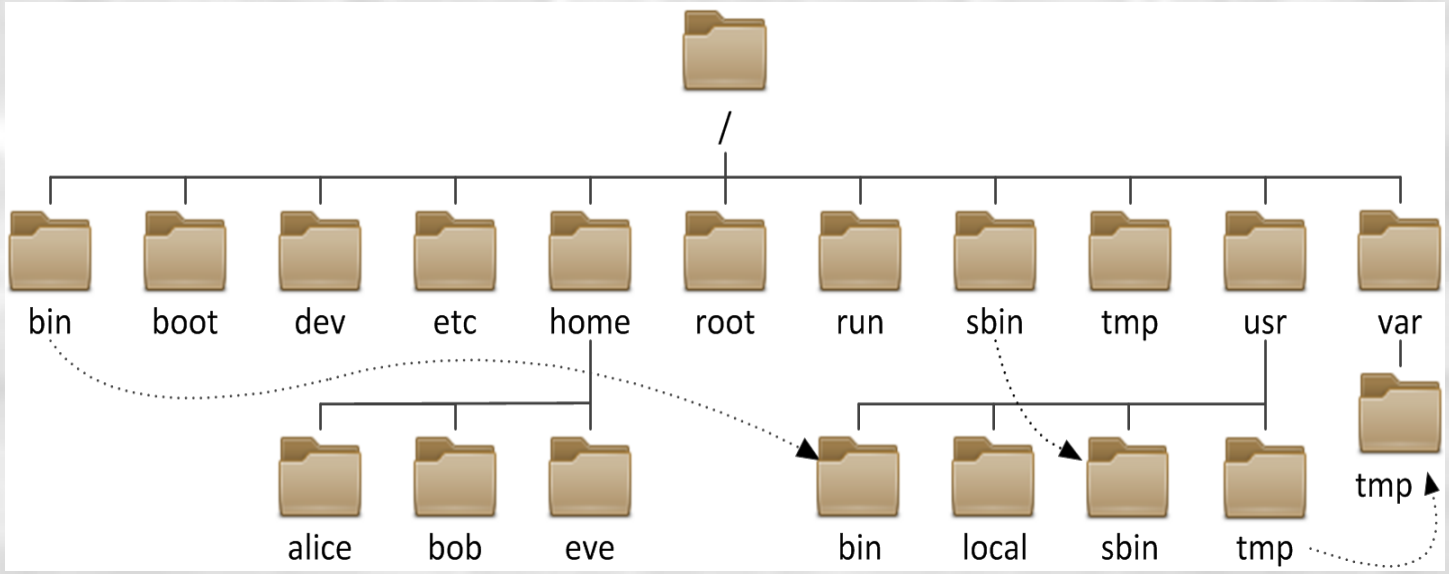


- Windows File System:
  - ✓ NTFS (New Technology File System)
- Linux File System:
  - ✓ ext4 (Fourth Extended File System)
- macOS File System:
  - ✓ APFS (Apple File System)
- Others:
  - ✓ FAT32 (File Allocation Table)
  - ✓ exFAT (Extended File Allocation Table)

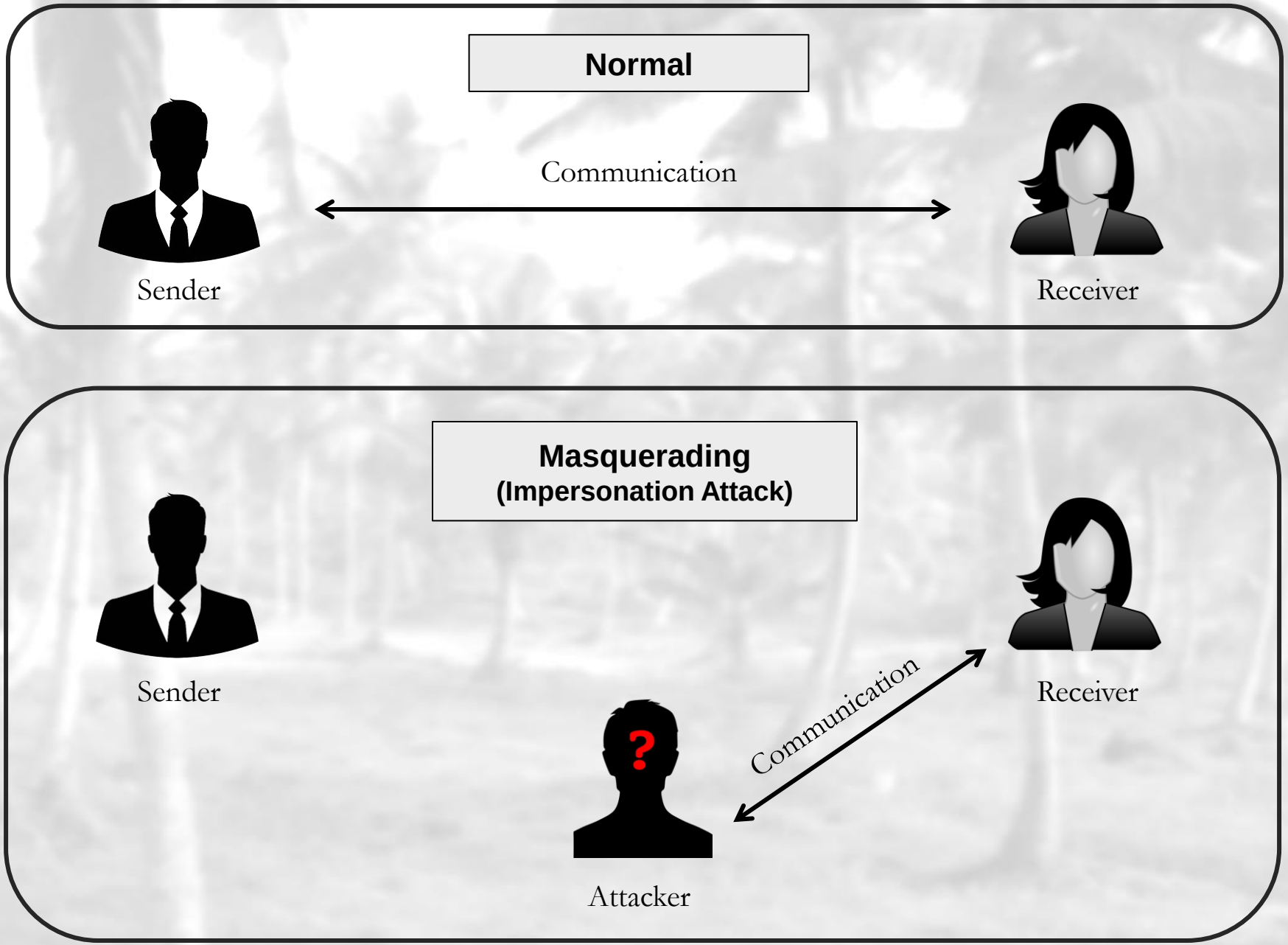
Protection Issue: read, write, execute, append, delete … etc.

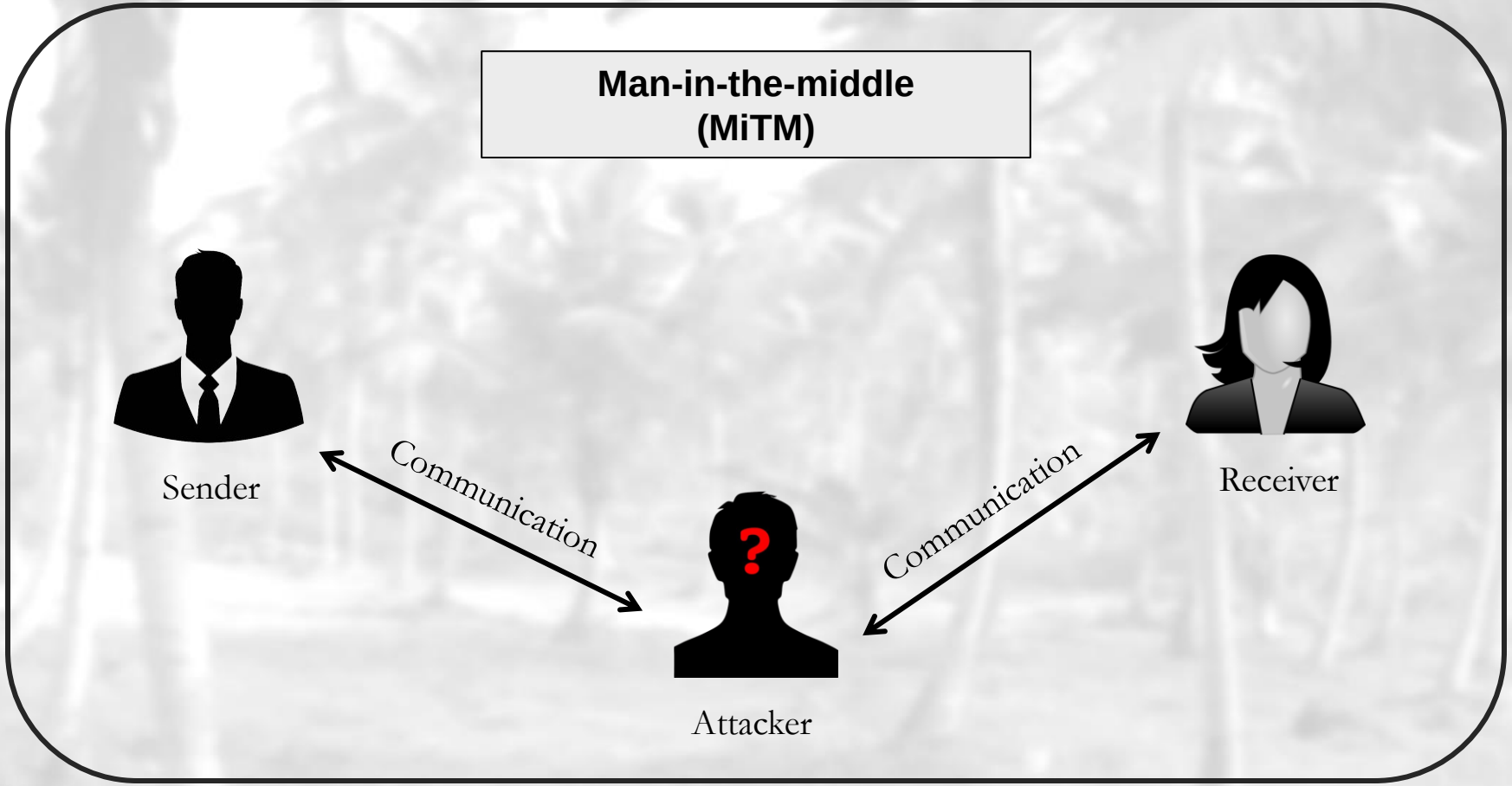
A Typical File-system Organization



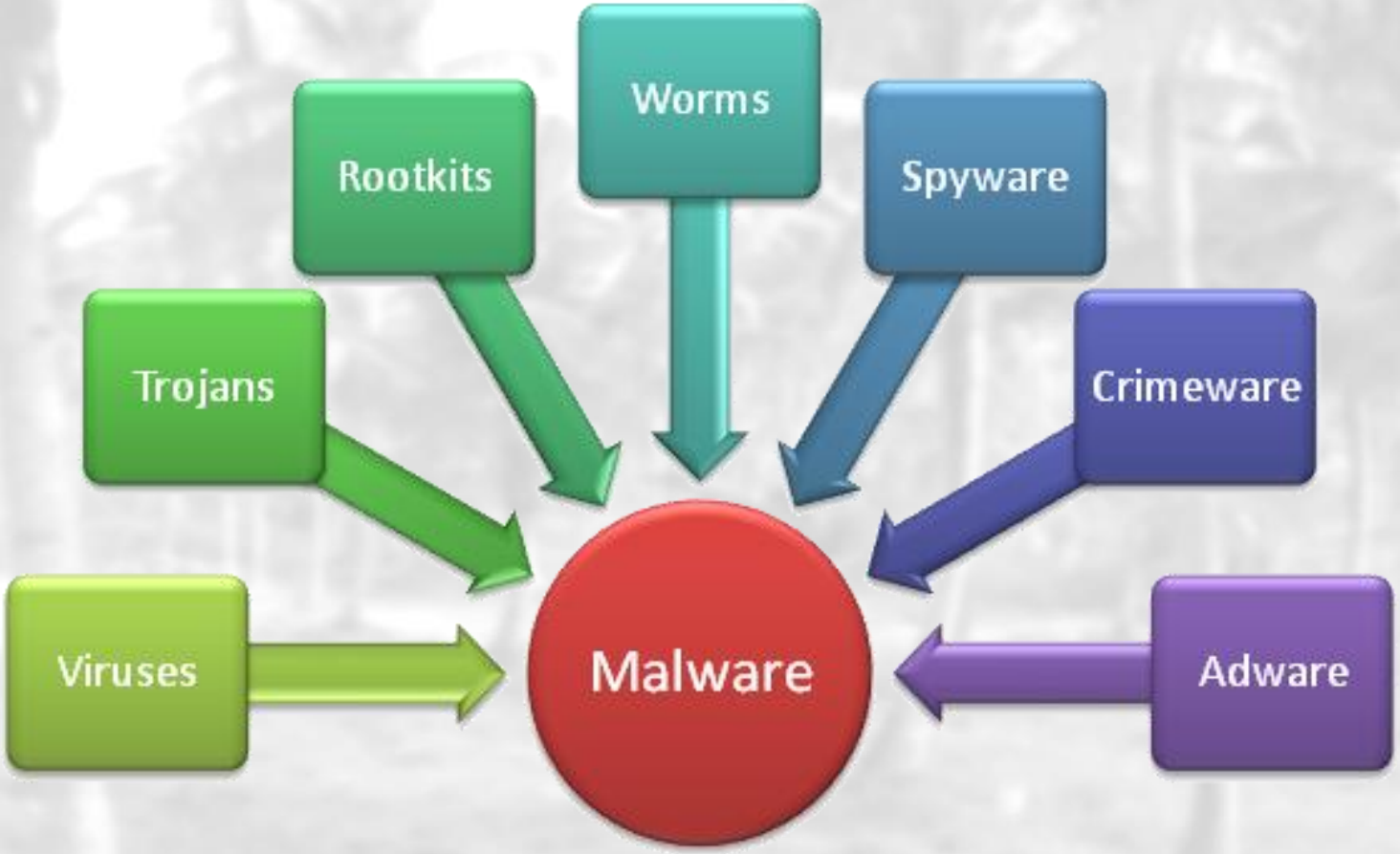


Protection and Security


Normal
Communication
Sender
Receiver
Masquerading
(Impersonation Attack)
Sender
Receiver
Communication
?
Attacker


Man-in-the-middle
(MiTM)
Sender
Communication
Attacker
Communication
Receiver

Types of viruses



Types of viruses (Cont.)

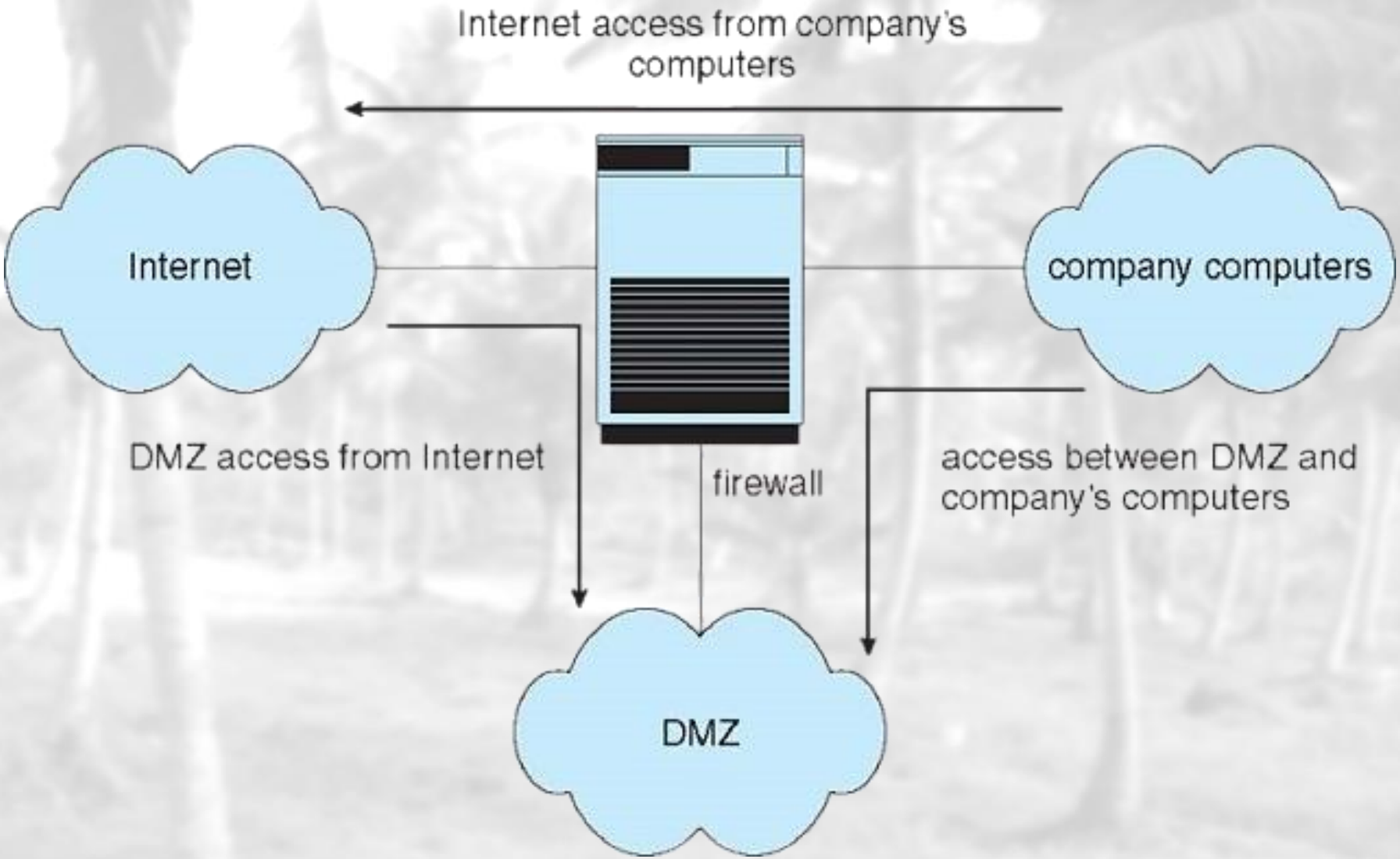
Type	Function
Worms	a computer program that replicates itself.
Trojan Horse	a sort of destructive program that remains disguised in a normal software program.
Ransomware	designed to encrypt your files and block access to them until a ransom is paid.
Adware	an unwanted software that displays advertisements on your screen.
Spyware	invades your computer and attempts to steal your personal information such as credit card or banking information, web accounts.

How to protect yourself ?

- **Antivirus**
  - is a program that secures your system from any potential viruses.

- **Firewall**
  - is a protective shield that keeps bad things away from your computer.
    - ✓ IDS / IPS

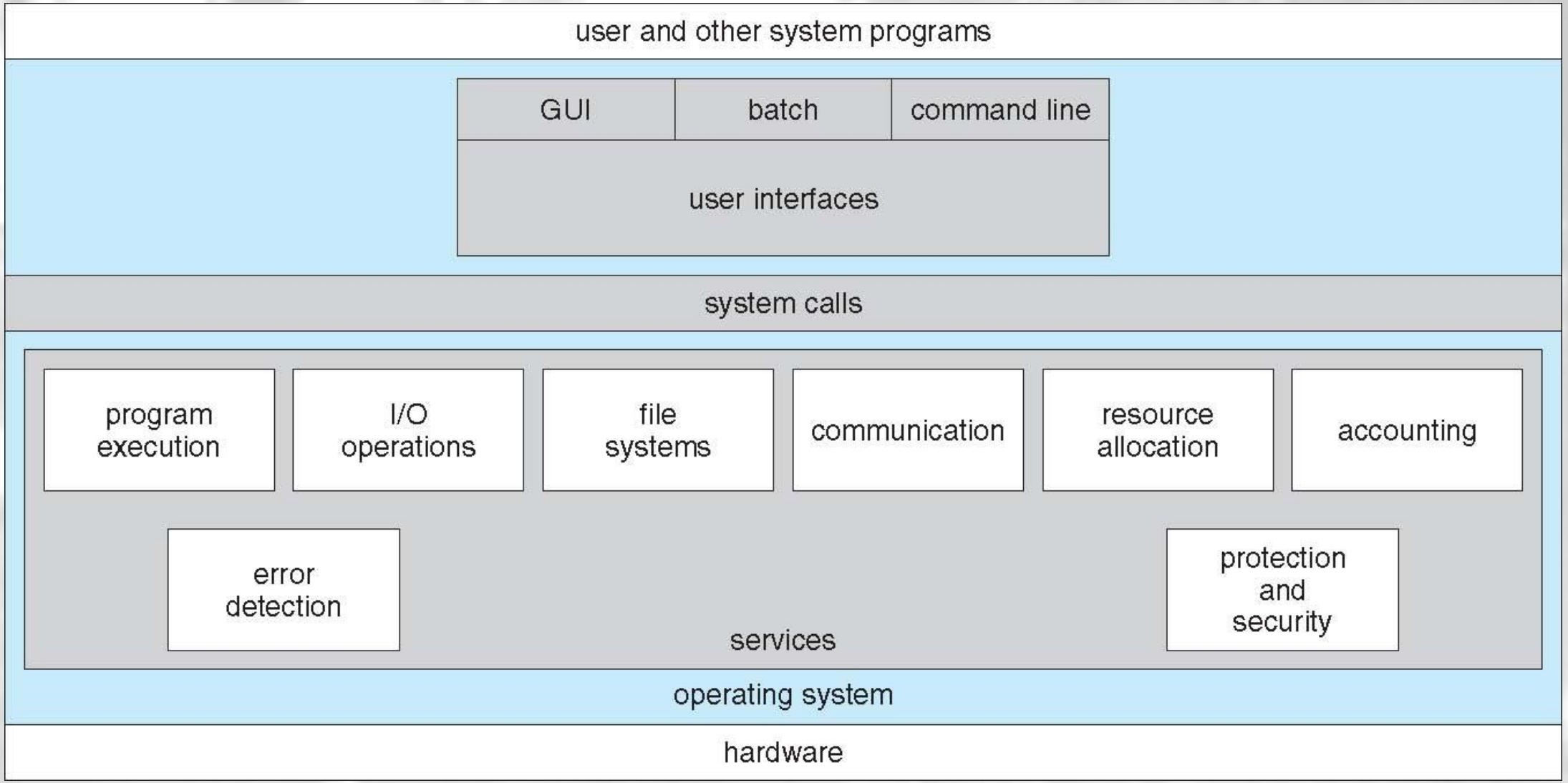
avast 2020 AVG McAfee


Internet access from company's computers
Internet
company computers
DMZ access from Internet
firewall
access between DMZ and company's computers
DMZ


https://ddosecrets.com/

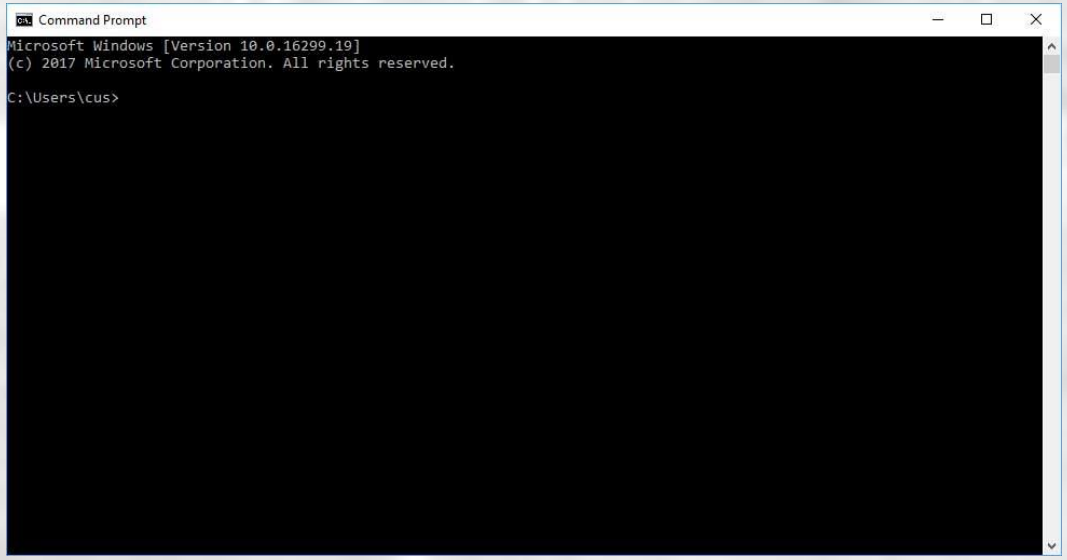
Operating System Services

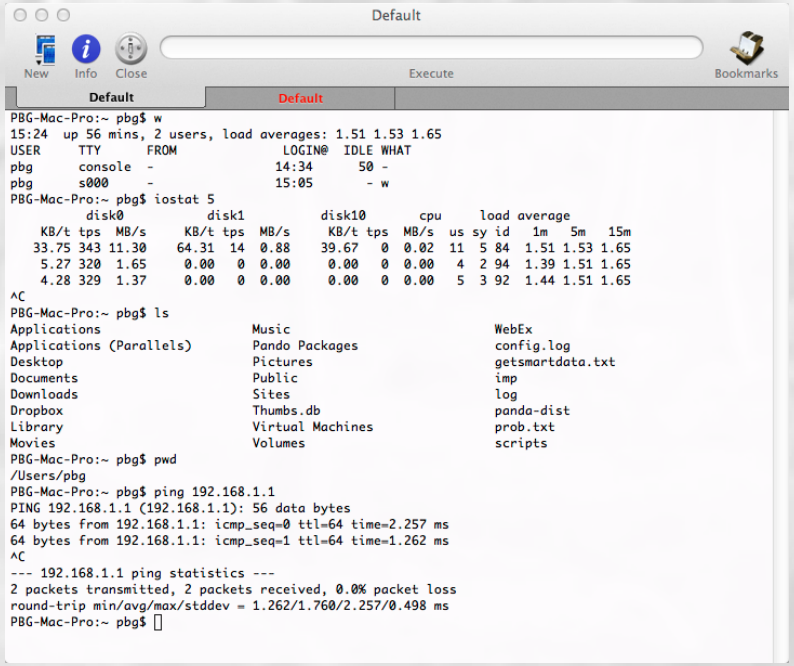
Operating System Services



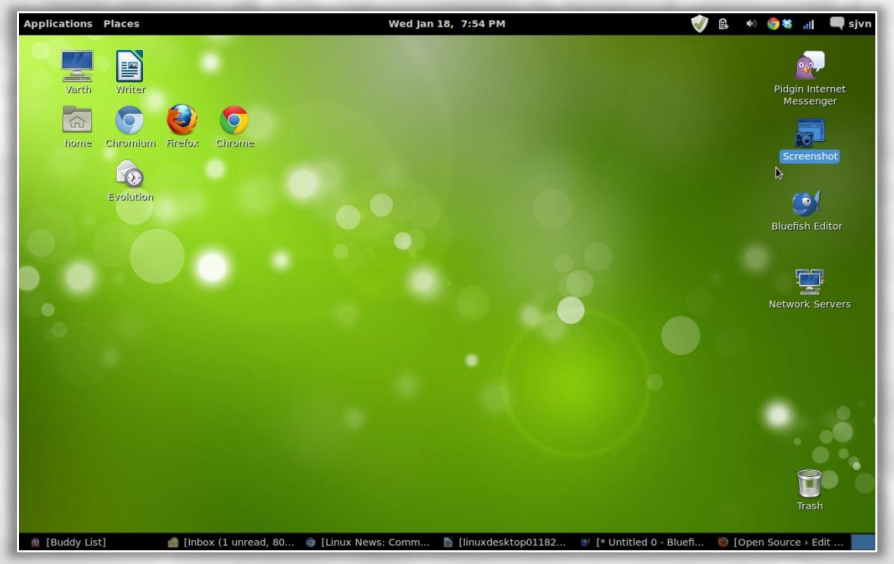
I) User Interface

- Almost all operating systems have a user interface (UI)
  - Command-Line (CLI)
  - Graphics User Interface (GUI)
  - Batch





Types of User Interfaces (UI)



2) System Calls

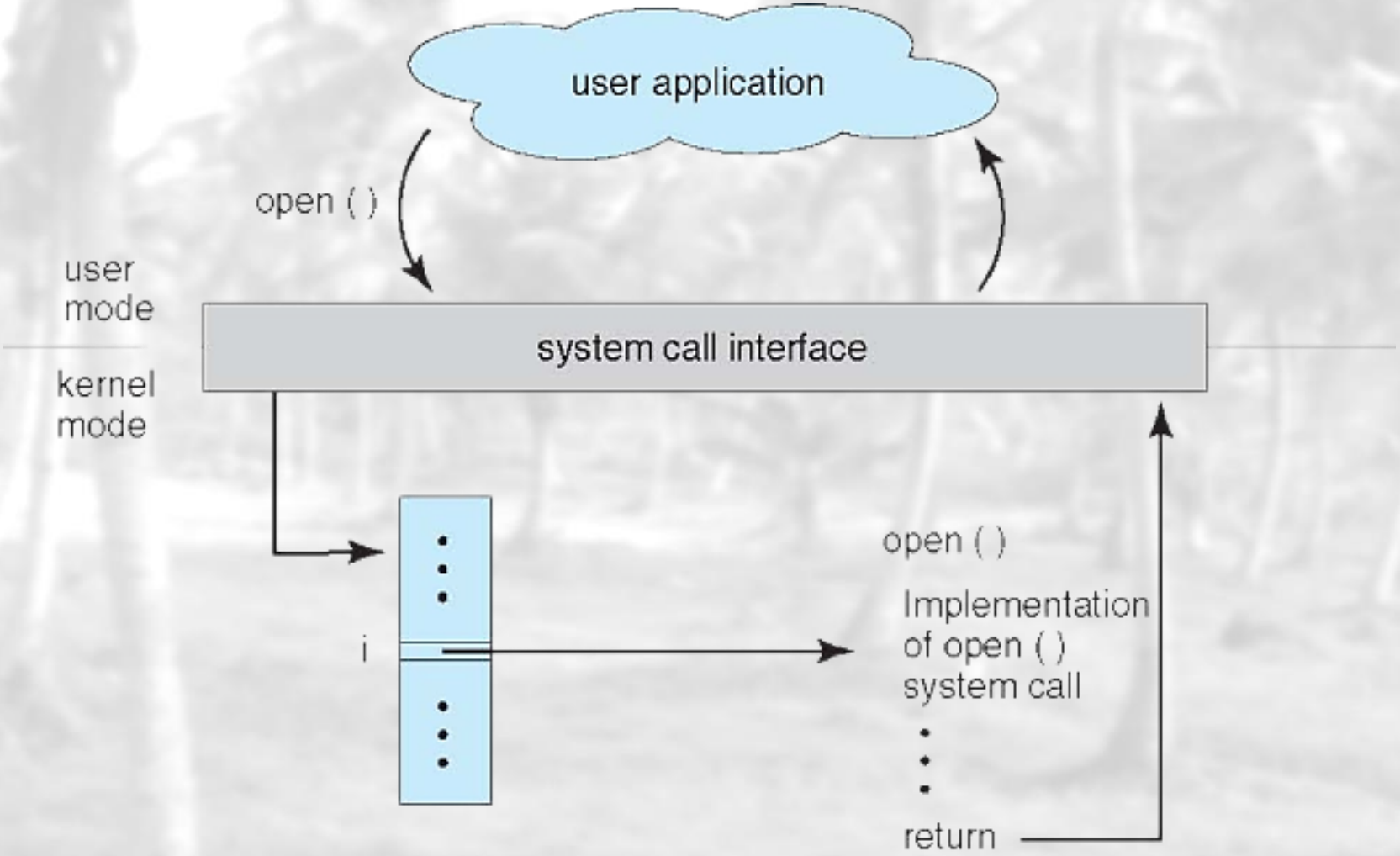
- Provide a programming interface to the services provided by the OS.
- Generally available as assembly-language instructions.
- Typically written in a *high-level language* (C or C++)
- Mostly accessed by programs via a high-level Application Programming Interface (API) rather than direct system call use

System Call Position

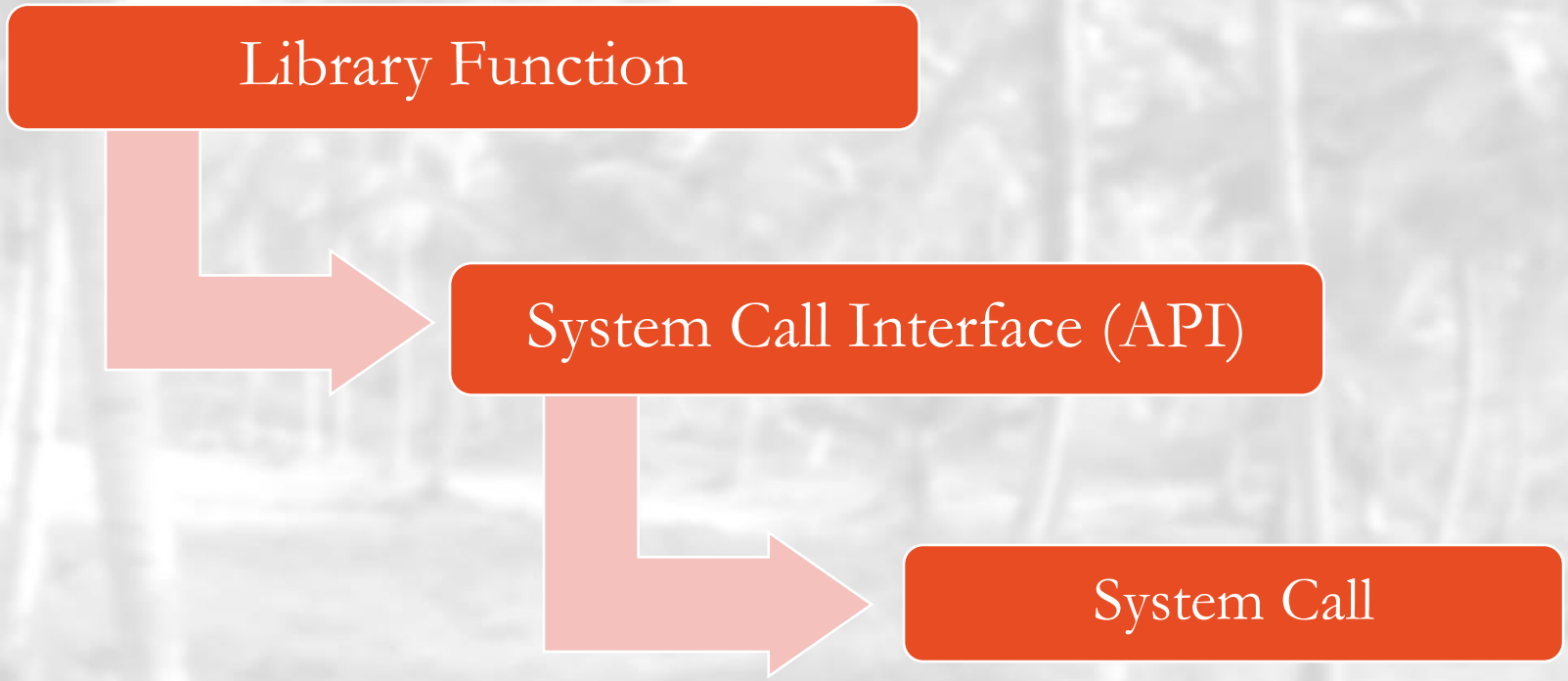
* **unisd.h** is a standard c library follow the POSIX specification to connect to the system calls.

* **windows.h** is another c library to access functions in the NTDLL.dll MS Windows

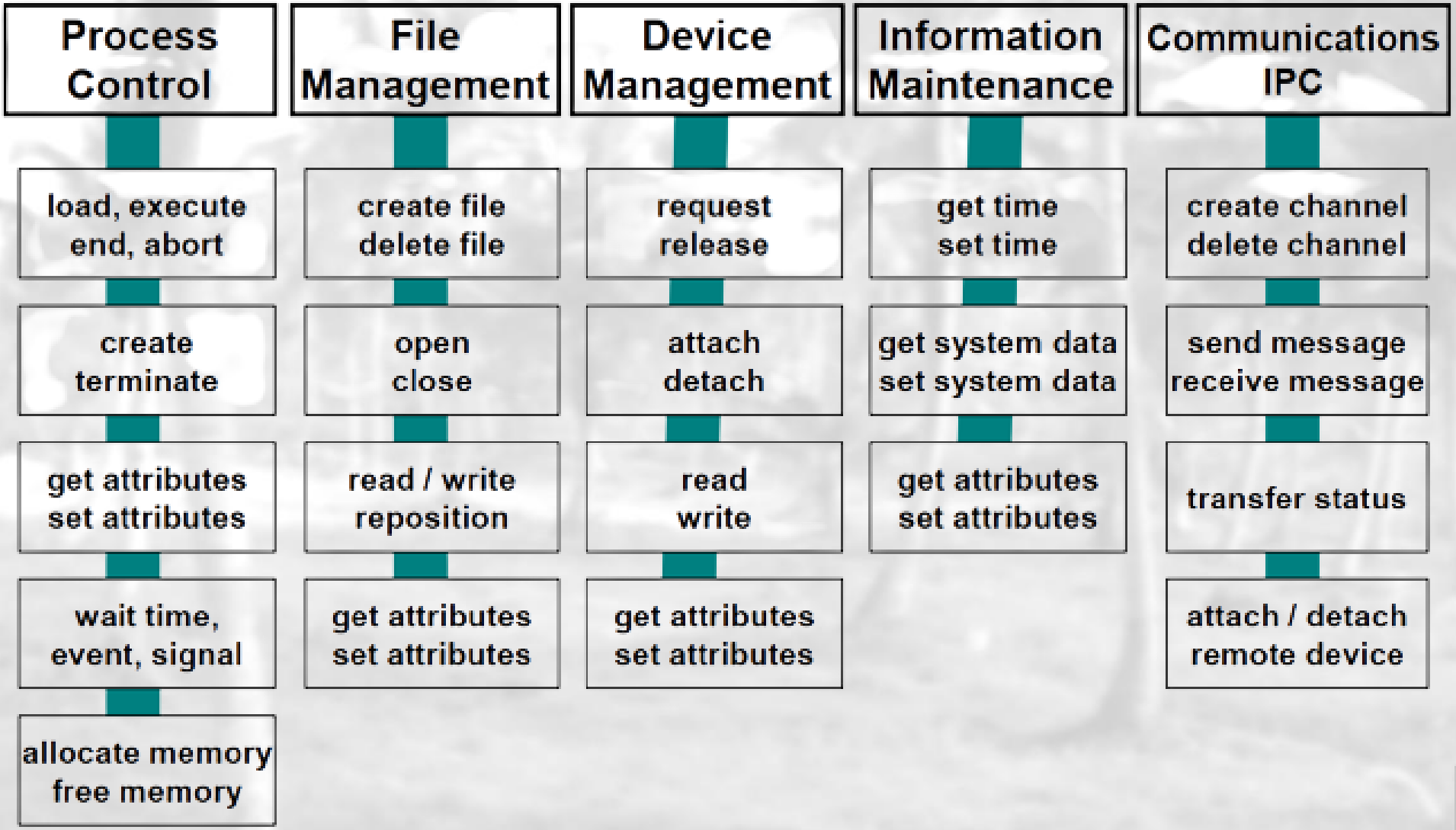
System Call Interface (API)



Hierarchy Summary



System Calls Types



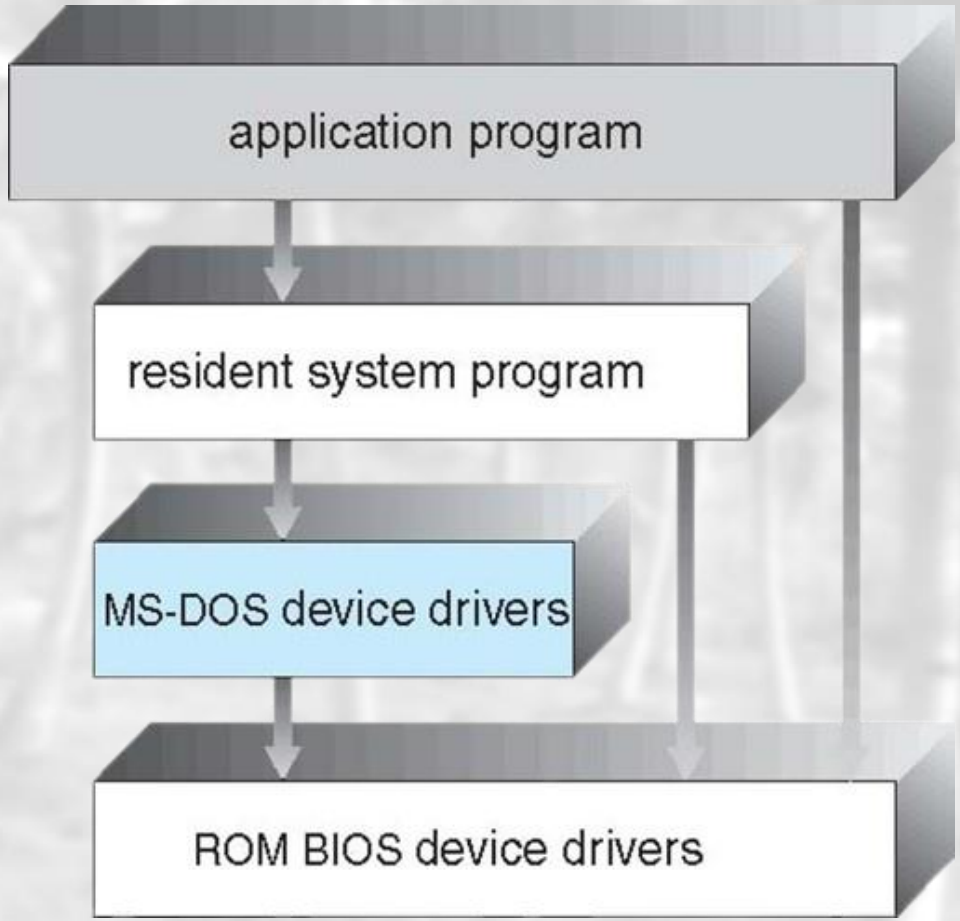
System Structure

OS Design Structure

- Simple Structure (*MS-DOS*)
- More Complex (*UNIX – monolithic*)
- Layered (*Conceptual*)
- Microkernel (*Mach*)
- Modular (*modules*)

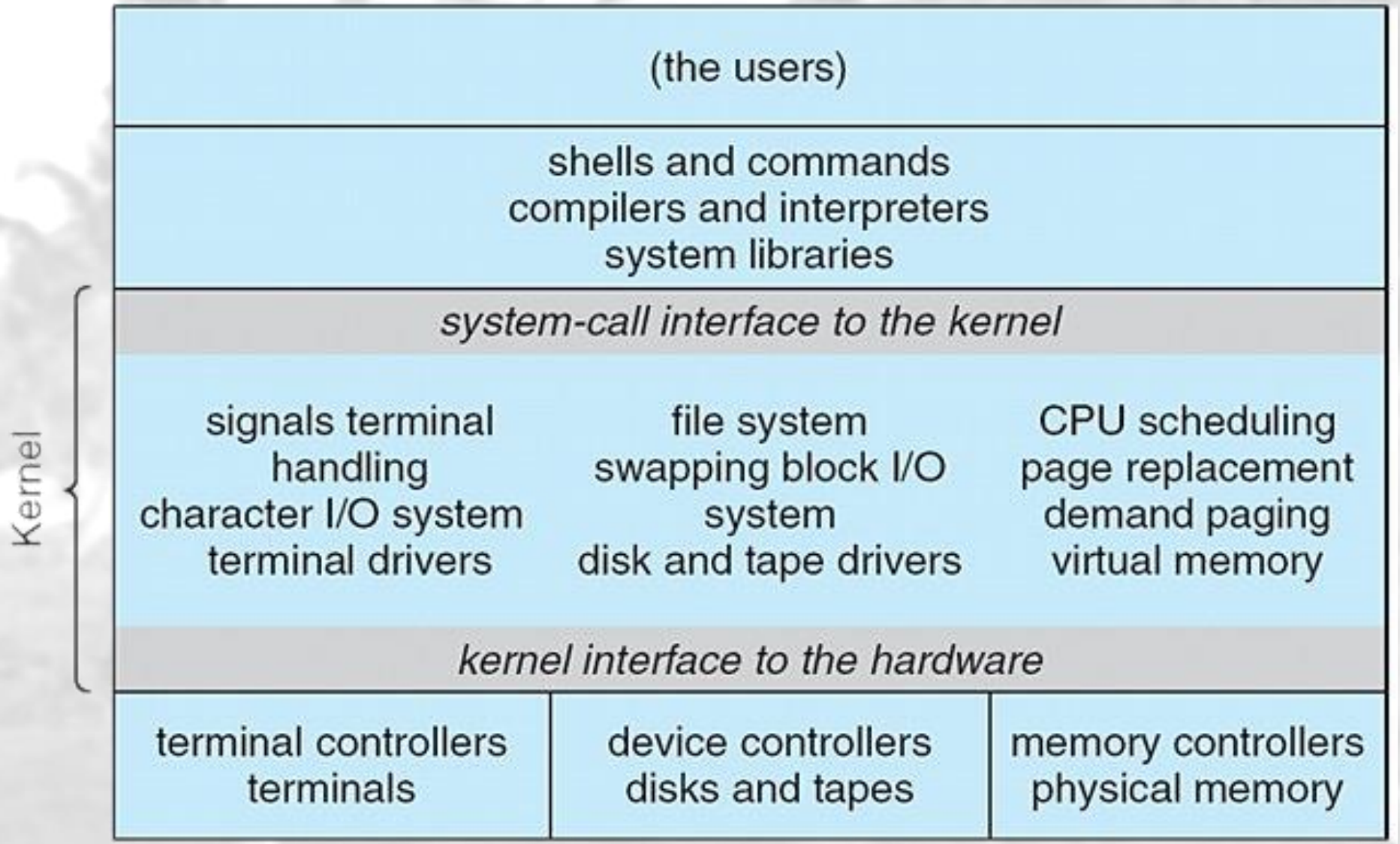
Simple Structure – MS-DOS

- MS-DOS is written to provide the most functionality in the *least space*
  - Higher performance
  - Smaller size
  - Not divided into modules
  - Interfaces and levels of functionality are *not well separated*
  - Very complex to maintain later
  - Hardware dependent



MS-DOS
1981 – 2000

More Complex Structure – UNIX



More Complex Structure – UNIX (Cont.)

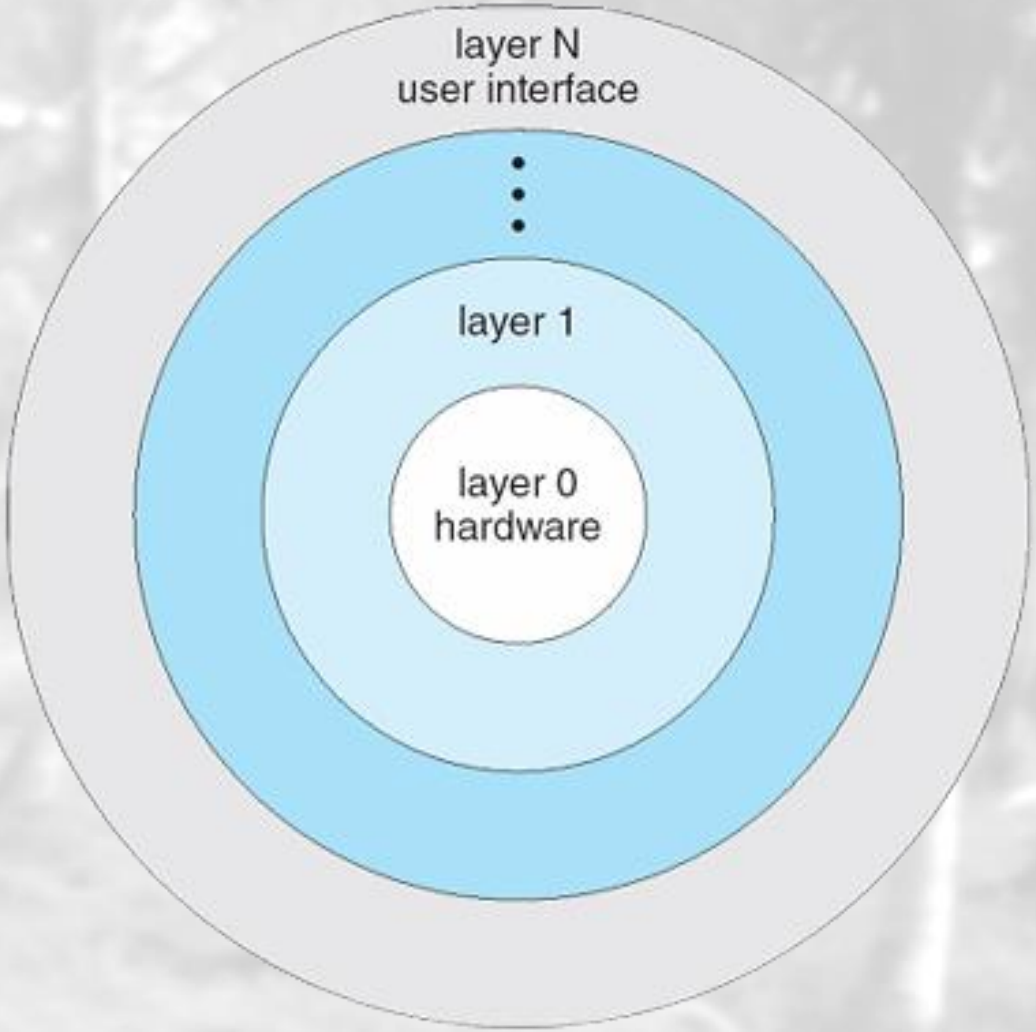
- **Advantages:**
  - Higher performance
  - Adds some protection

- **Disadvantages:**
  - Very complex to maintain later
  - Still Hardware dependent

Examples: Windows XP – Monolithic

Layered Approach (*Conceptual*)

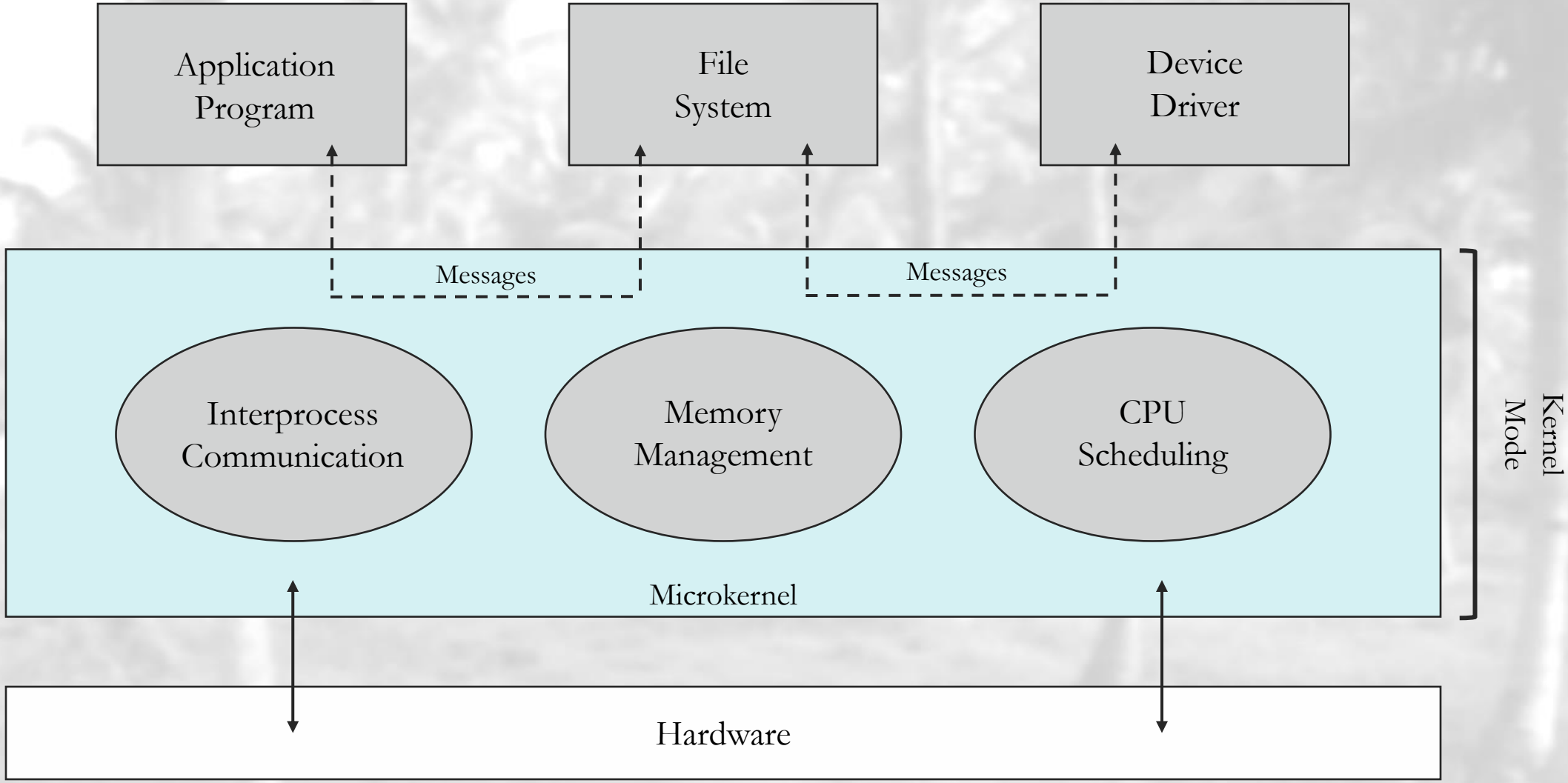
- The operating system is *divided into a number of layers (levels)*, each built on top of lower layers.
- The bottom layer (layer 0), is the hardware; the highest (layer N) is the user interface.
- With modularity, layers are selected such that each *uses functions (operations) and services of only lower-level layers*



Layered Approach (Cont.)

- **Advantages:**
  - Modularity
  - Debugging
  - Modification
- **Disadvantages:**
  - Layering overhead to the system call

Microkernel System Structure



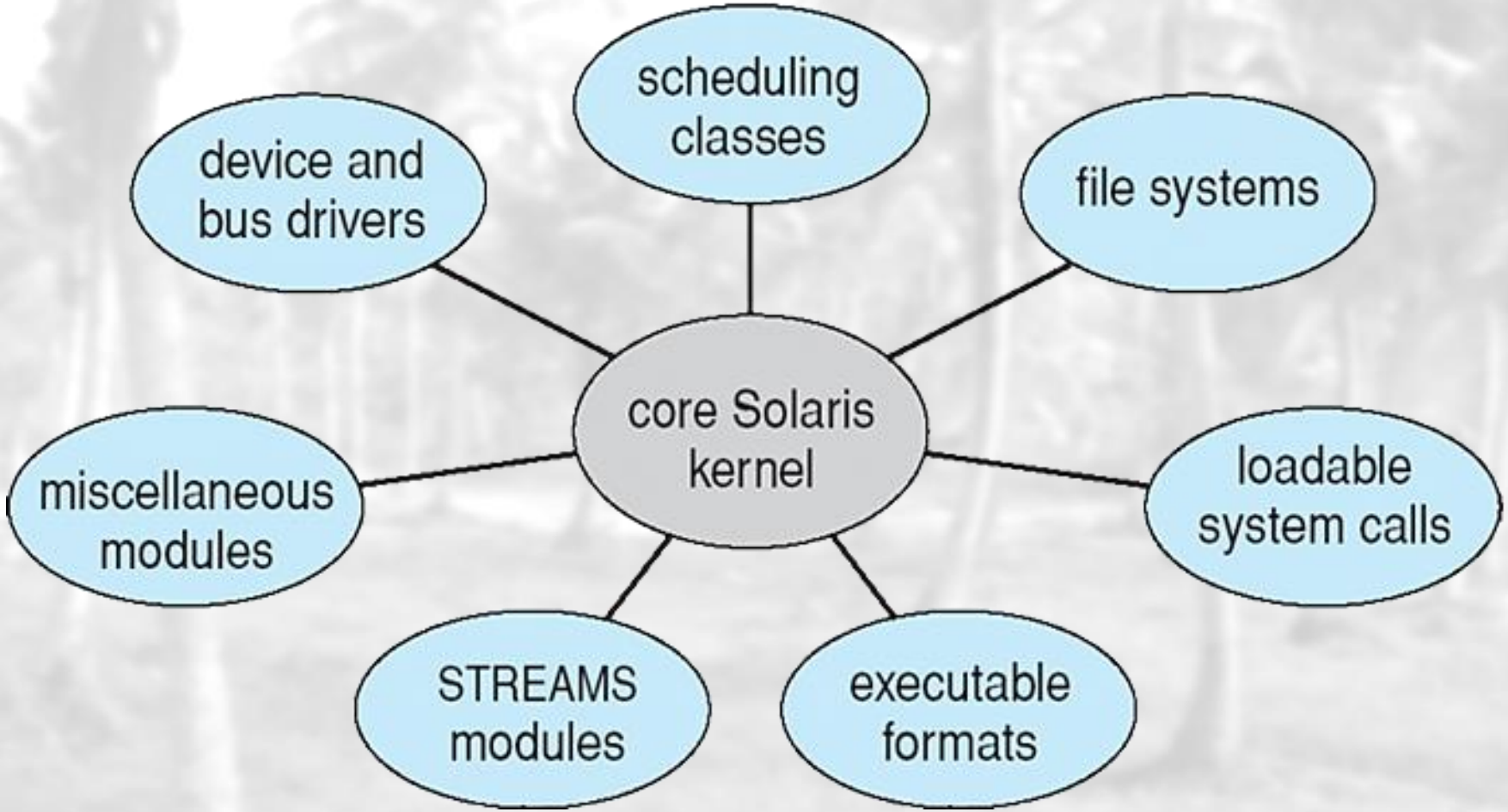
Microkernel System Structure (Cont.)

- **Advantages:**
  - Easier to extend
  - More reliable
  - More secure
- **Disadvantages:**
  - Performance overhead of user space to kernel space communication
    - ✓ That's why Windows XP designed almost as monolithic again!

Modular System Structure

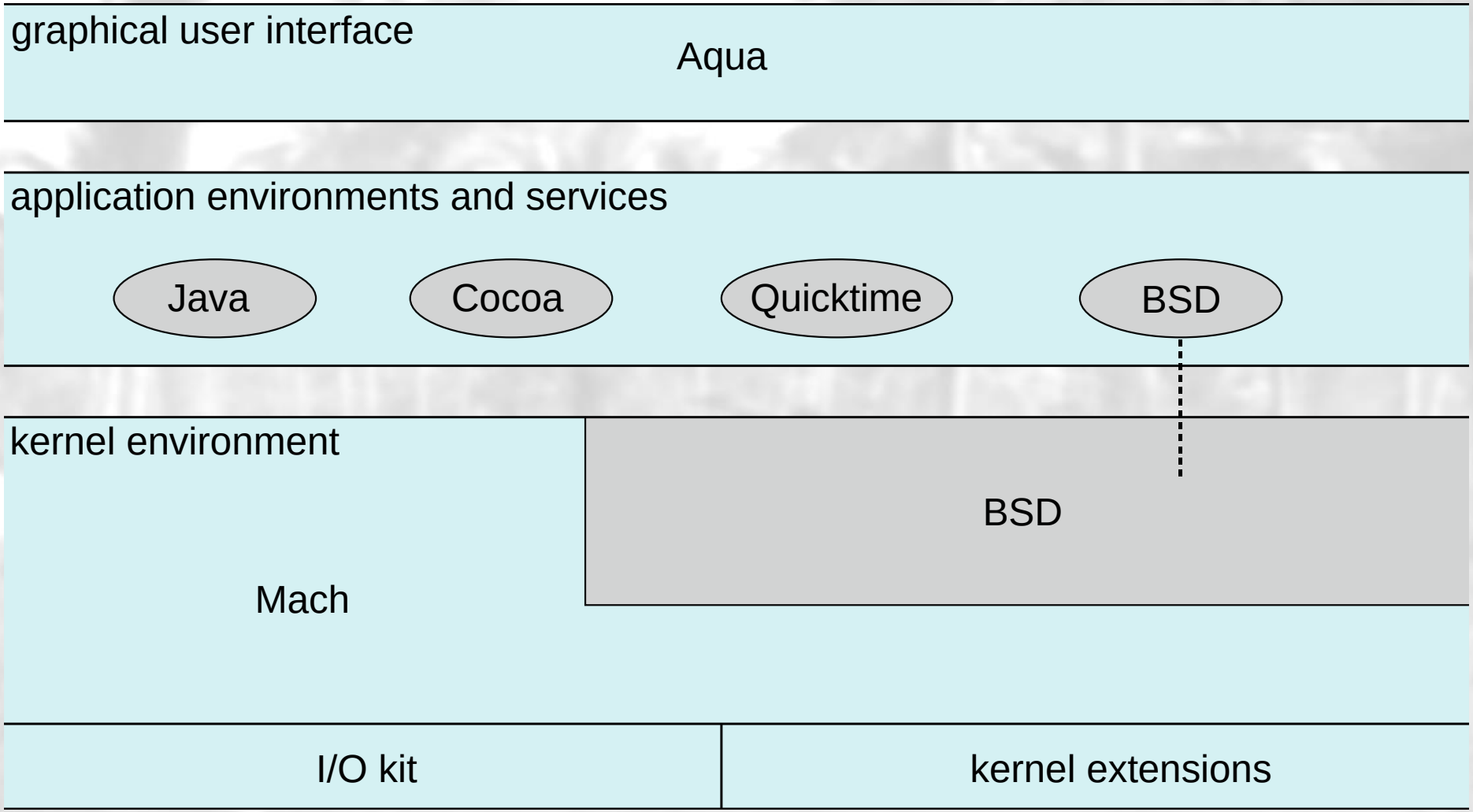
- Many *modern operating systems* implement loadable kernel modules
  - Uses object-oriented approach
  - Each core component is *separate*
  - Each talks to the others over known interfaces
  - Each is loadable as needed within the kernel
- Similar to layers but with more flexibility (any module can call another module)
  - Linux, Solaris … etc.

Modular System Structure (Cont.)

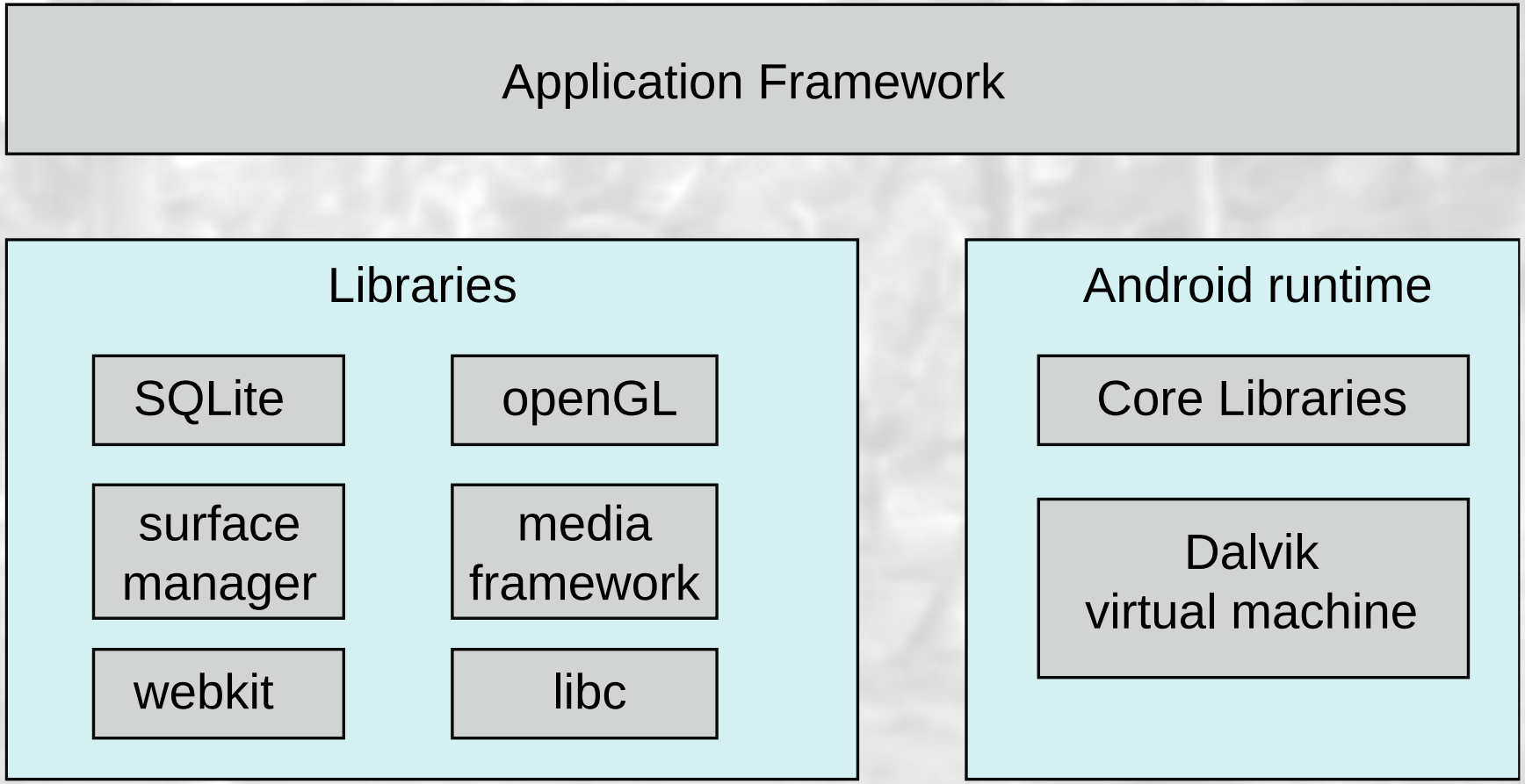


Solaris User Interface (GUI)

Mac OS X Structure



Android Architecture



Modular System Structure (Cont.)

- **Advantages:**
  - Resembles a layered system
  - More flexibility
    - ✓ any module can call any other module
  - Similar to microkernel approach but more efficient
    - ✓ modules do not need to invoke message passing to communicate

Hybrid Systems

- Most *modern operating systems* are actually not one pure model
  - Hybrid combines multiple approaches to address performance, security, and usability needs
  - Linux and Solaris kernels:
    - ✓ monolithic, plus modular
  - Windows:
    - ✓ monolithic, plus microkernel

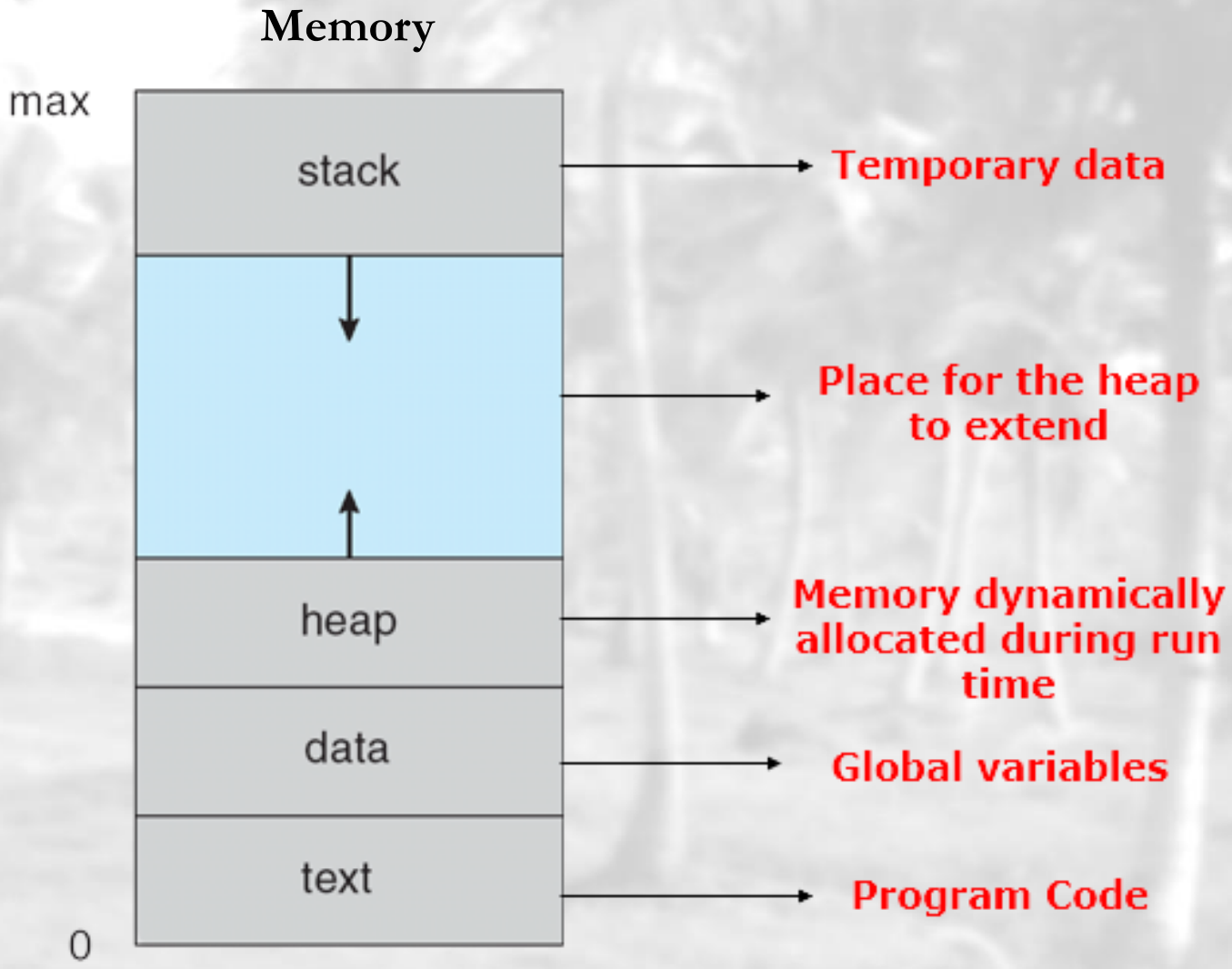
Process Concept

Process Concept

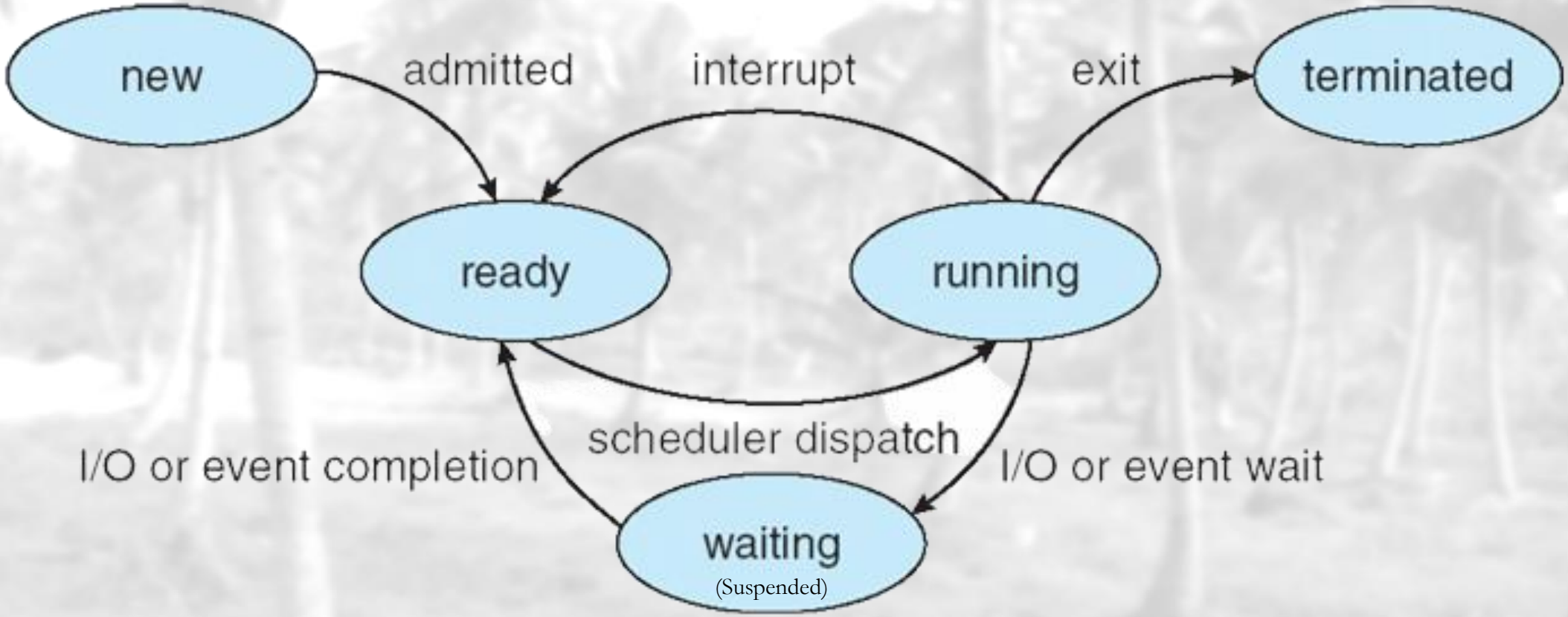
- Program: a passive entity stored on disk (executable file – .exe).
- Process: a program in execution; process execution must progress in sequential fashion.
- Program becomes a process when *executable file loaded into memory*.
- One program can be several processes.

Process in Memory

- **Text section**
  - Program instructions
- **Program counter**
  - Next instruction
- **Stack**
  - Local variables
  - Return addresses
  - Method parameters
- **Data section**
  - Global variables



Process State

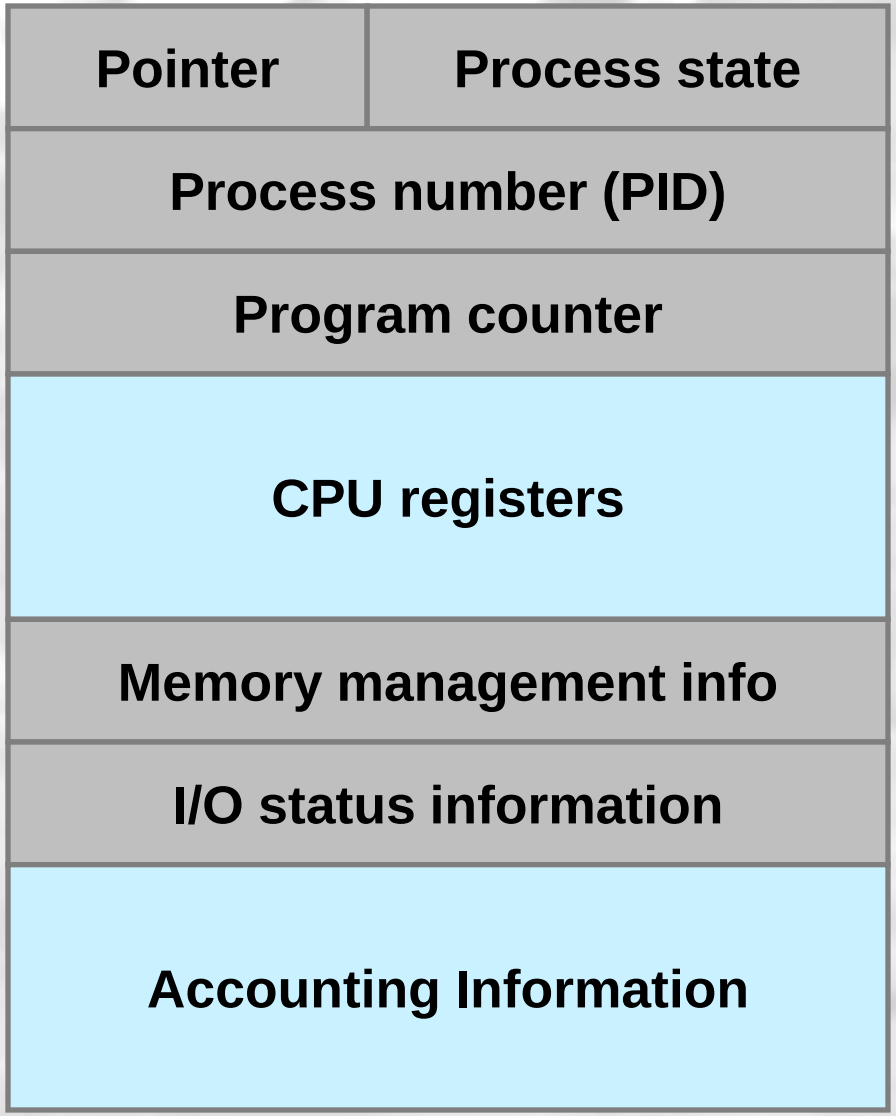


Process Control Block (PCB)

- Information associated with each process:
  - **Process state** (running, waiting/suspended, … etc.)
  - **Program counter** (address of next instruction to execute for the process)
  - **CPU registers** (for context switching, allowing a process to resume exec later)
  - **CPU scheduling information**
  - **Memory-management information** (memory allocation, like base and limit)
  - **Accounting information** (CPU usage, clock time elapsed, time limits, priority)
  - **I/O status information** (process's open files, devices in use, pending I/O requests)
  - **Pointer** (a link to the next PCB in a linked list of PCBs)

Process Control Block (PCB)

*Always remember the **ID card***
(Order is not important)



Process Scheduling

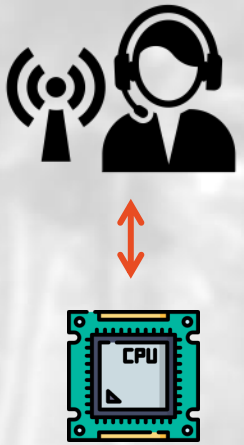
Process Scheduling

- Maximize CPU use, quickly switch processes onto CPU for time sharing.
- Long-term (or job) Scheduler – selects which processes should be brought into the **ready queue**.
- CPU (or Short-term) Scheduler selects from among the processes in **ready queue**, and allocates the CPU to one of them.
- Maintains scheduling queues of processes
  - » **Job queue:** set of all processes in the system
  - » **Ready queue:** set of all processes residing in main memory, ready and waiting to execute
  - » **Device queues:** set of processes waiting for an I/O device

- Processes migrate among the various queues.

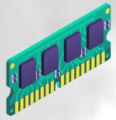
Dispatcher

- Dispatcher module gives control of the CPU to the process selected by the short-term scheduler; this involves.
  - » Switching context
  - » Switching to user mode
  - » Jumping to the proper location in the user program to
  - » Restart that program

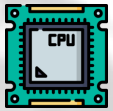


- Dispatch latency is the time it takes for the dispatcher to stop one process and start another running.

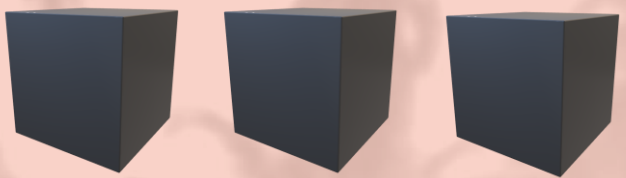
All processes in the system

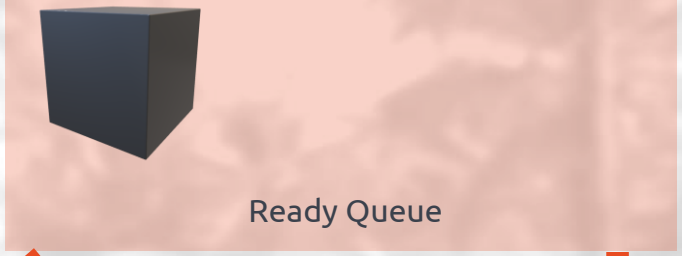


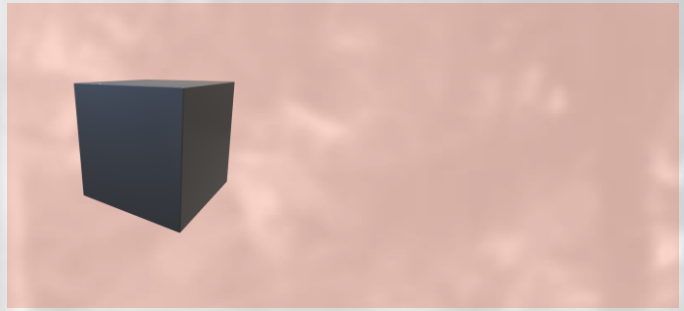
All processes in memory



Processor (CPU)





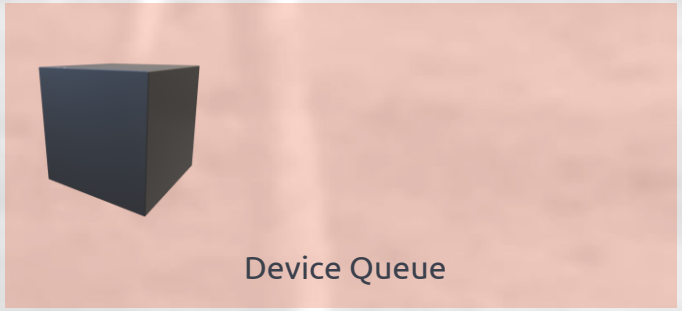


Long-term (or job) Scheduler

CPU (or Short-term) Scheduler

Dispatcher

Processes waiting for an I/O device



Process Scheduling

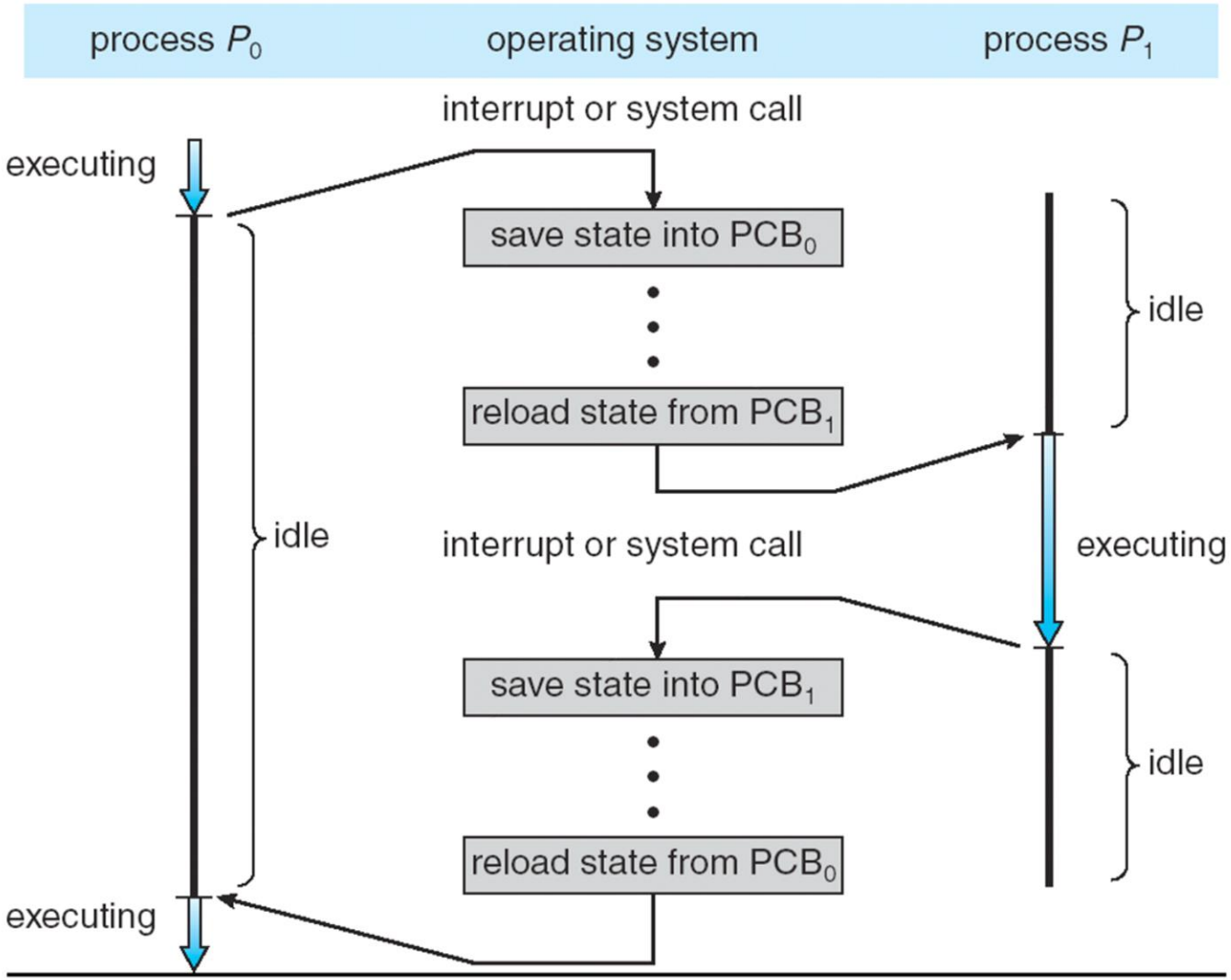
Scheduling Algorithms

- First- Come, First-Served (FCFS) Scheduling
- Shortest-Job-First (SJF) Scheduling
- Priority Scheduling
- Multilevel Queue Scheduling
- Etc.

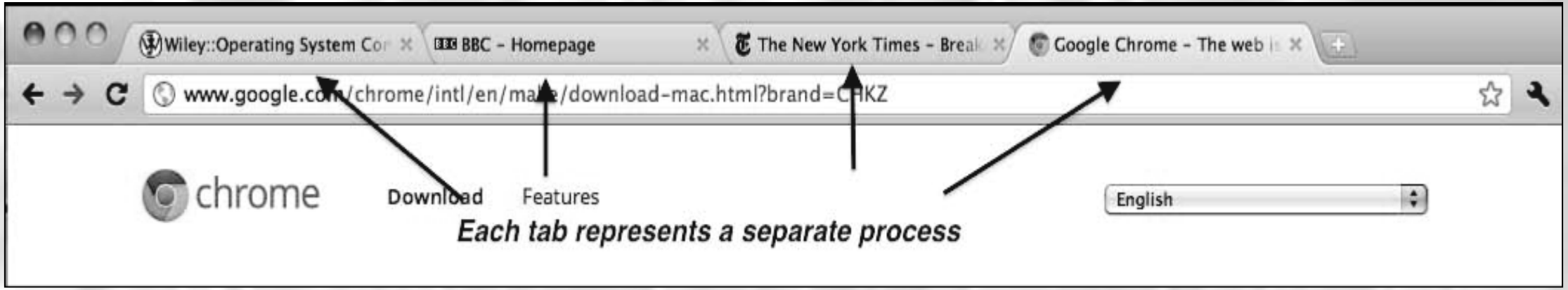
Context Switching

- When CPU *switches to another process*, the system must save the state of the old process and load the saved state for the new process via a context switch.
- Context Switching is for Multitasking.
- Context-switch time is *overhead*; the system does no useful work while switching.
- The more complex the OS and the PCB, the longer the context switch
- Time *dependent on hardware* support

Context Switching (Cont.)



Process Example

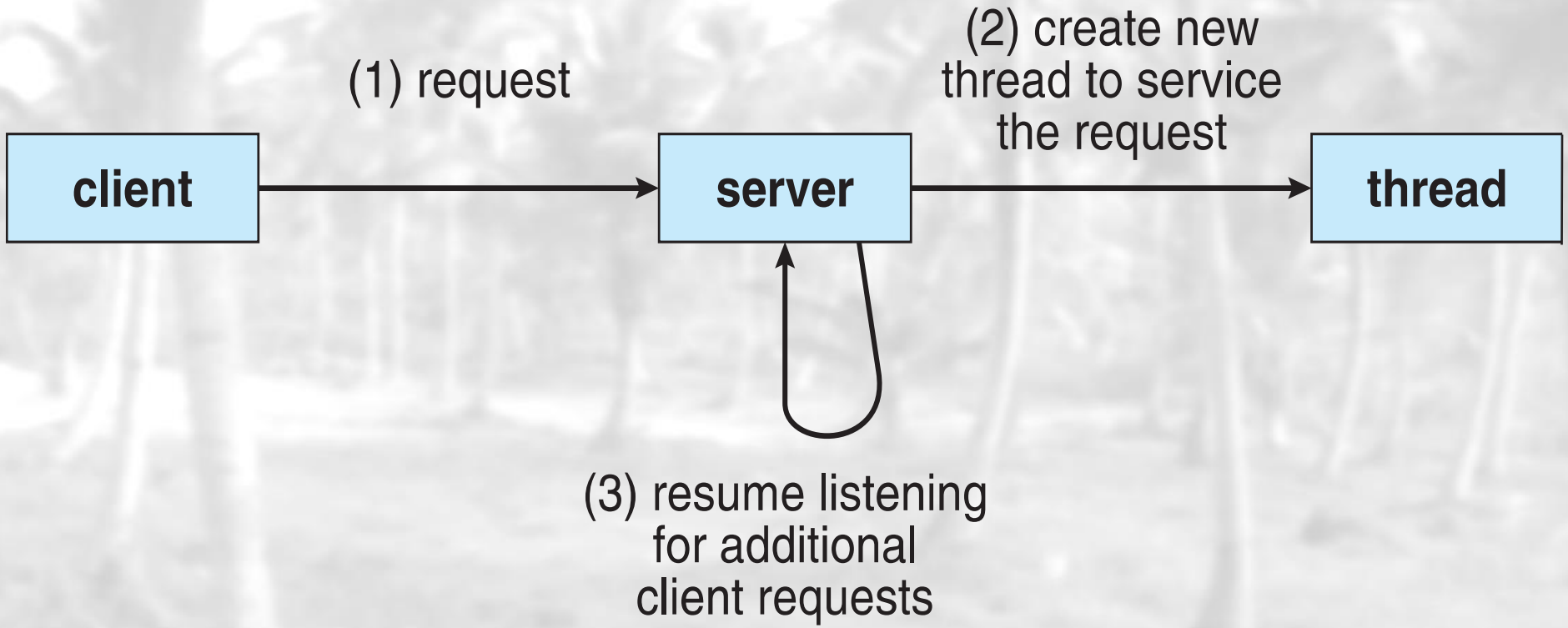


Google Chrome Browser

Question: Why would they do this?

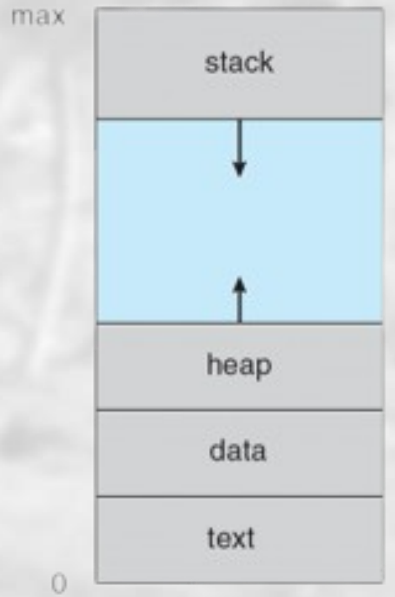
Threads

Multithreaded Server Architecture



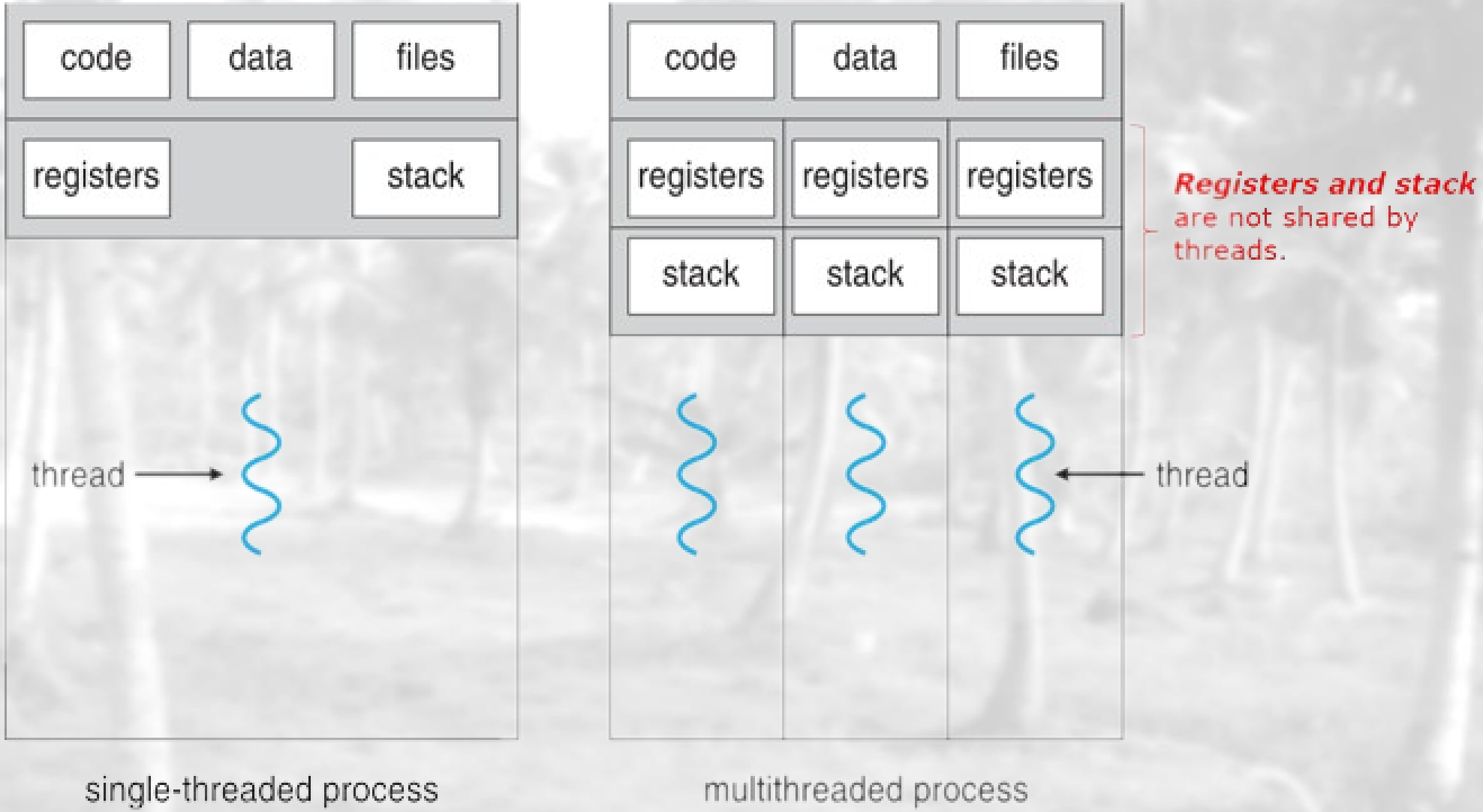
Threads Concept

- Most modern applications are multithreaded.
- Multiple tasks with the application can be implemented by separate threads.
- Process creation is *heavy-weight*.
- Thread creation is *light-weight*.
  - Only Program Counter is needed
- **Threads** simplify code and increase efficiency



+ PCB

Single and Multithreaded Processes

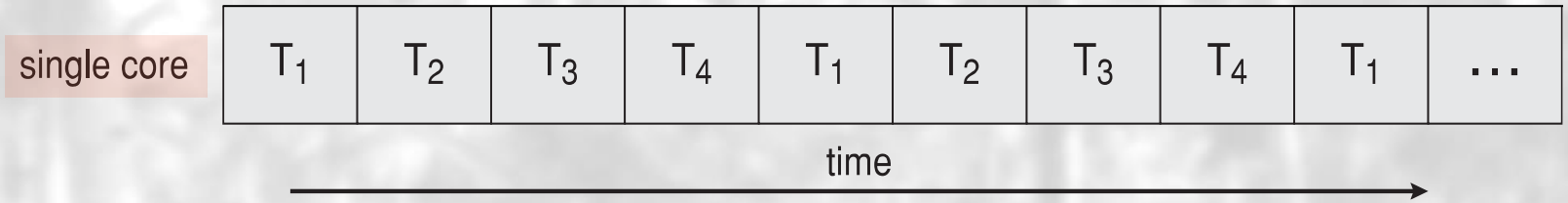


Benefits of using threads

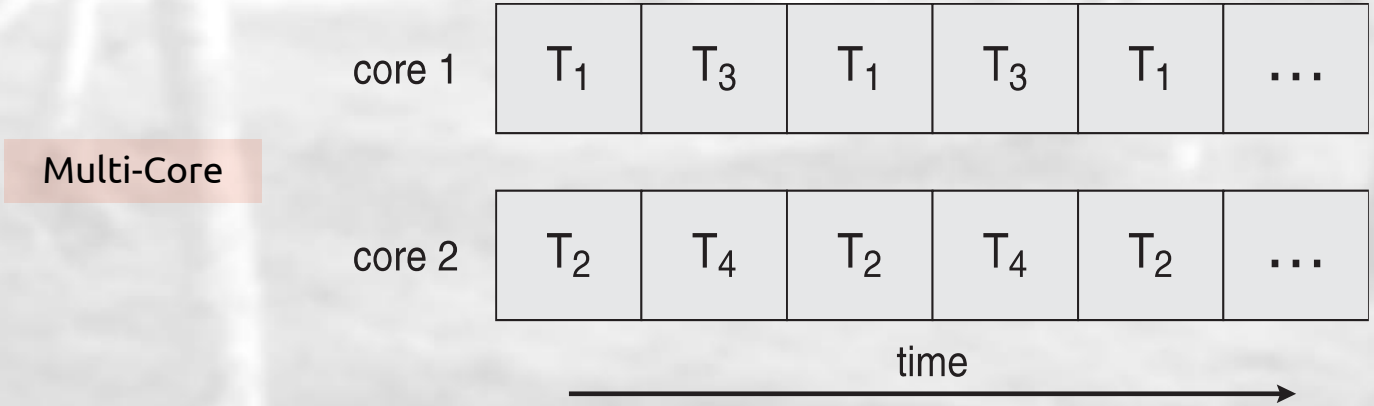
- Responsiveness – may allow continued execution if part of process is blocked
  - *especially important for user interfaces*
- Resource Sharing – threads share resources of process
  - *easier than shared memory or message passing*
- Economy – cheaper than process creation
  - *thread switching lower overhead than context switching*
- Scalability – process can take advantage of multiprocessor architectures

Concurrency vs. Parallelism

- Concurrent execution on single-core system:

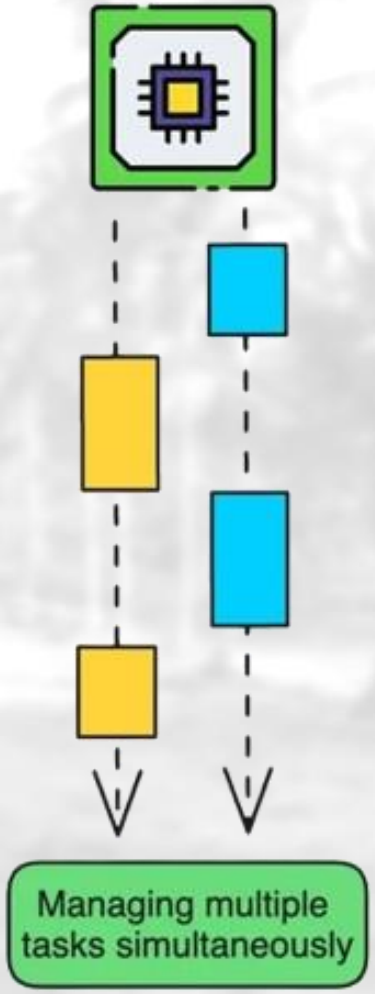


- Parallelism on a multi-core system:



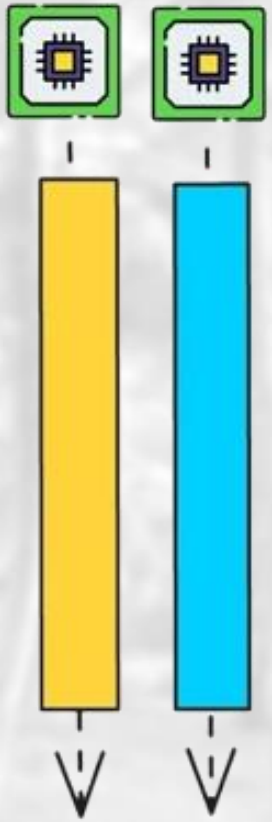
✓ Operating systems (Windows, Linux, macOS) use both concurrency and parallelism to optimize resource utilization.

Concurrency



Managing multiple tasks simultaneously

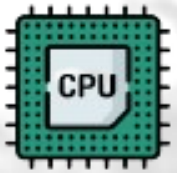
Parallelism



Executing multiple tasks simultaneously

Synchronization

What is Multithreading



CPU/Memory

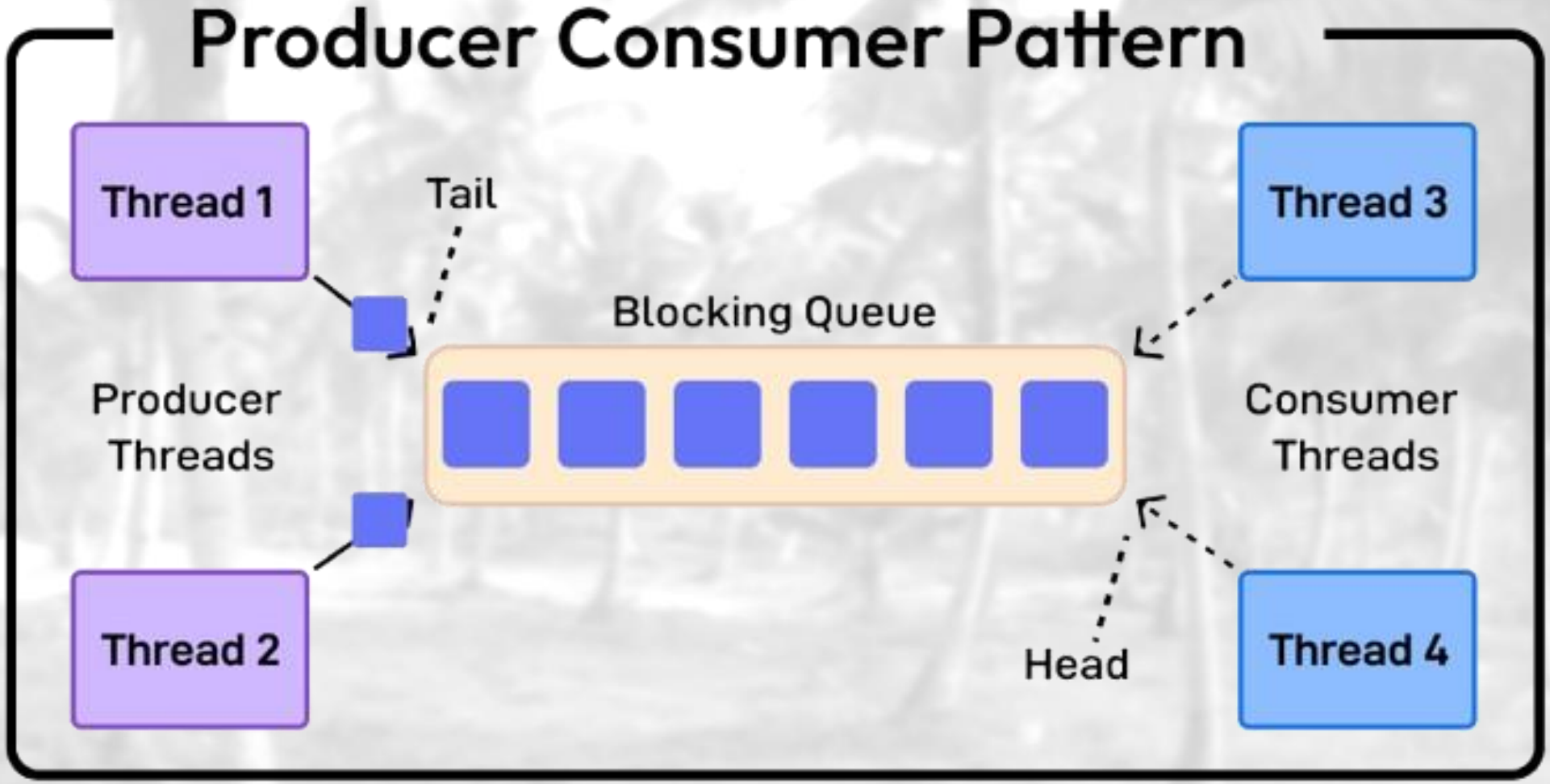
Thread

Thread

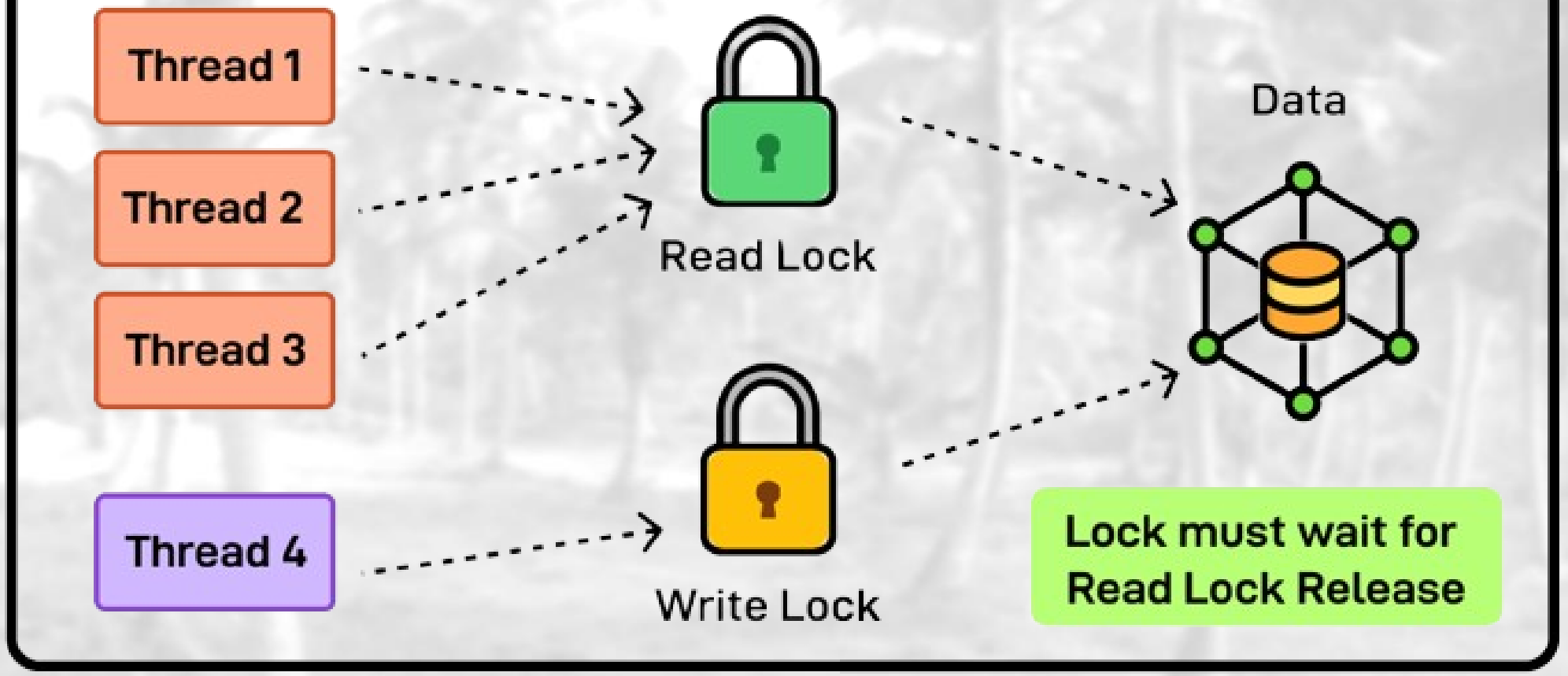
Thread

- Multithreading enables a single program or process to execute multiple tasks concurrently.
- Each task is a thread. Think of threads as lightweight units of execution that share the resources of the process such as memory space.
- Multithreading introduces complexities like synchronization, communication, and potential race conditions.

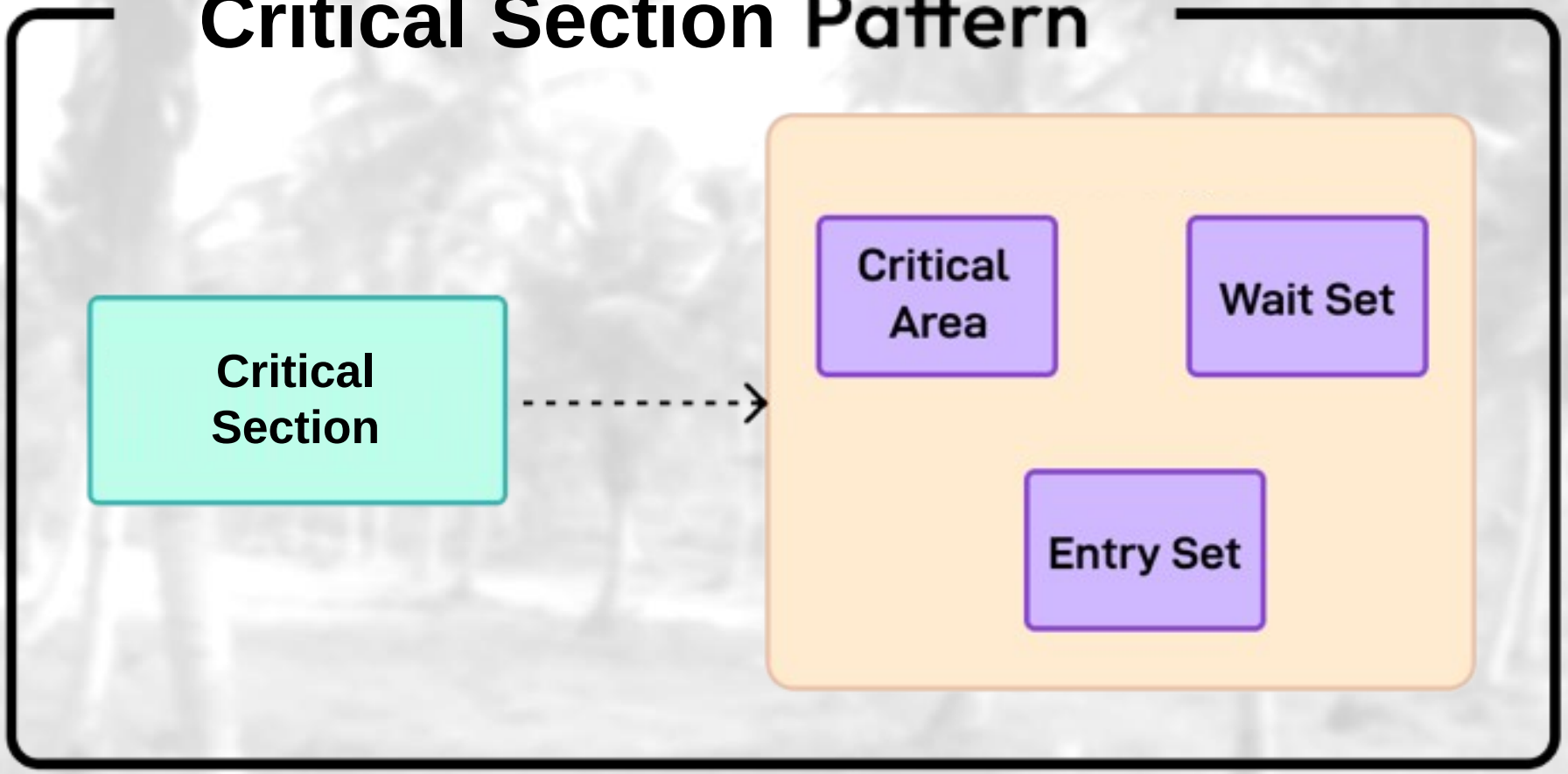
This is where Multithreading Design Patterns Become Useful.

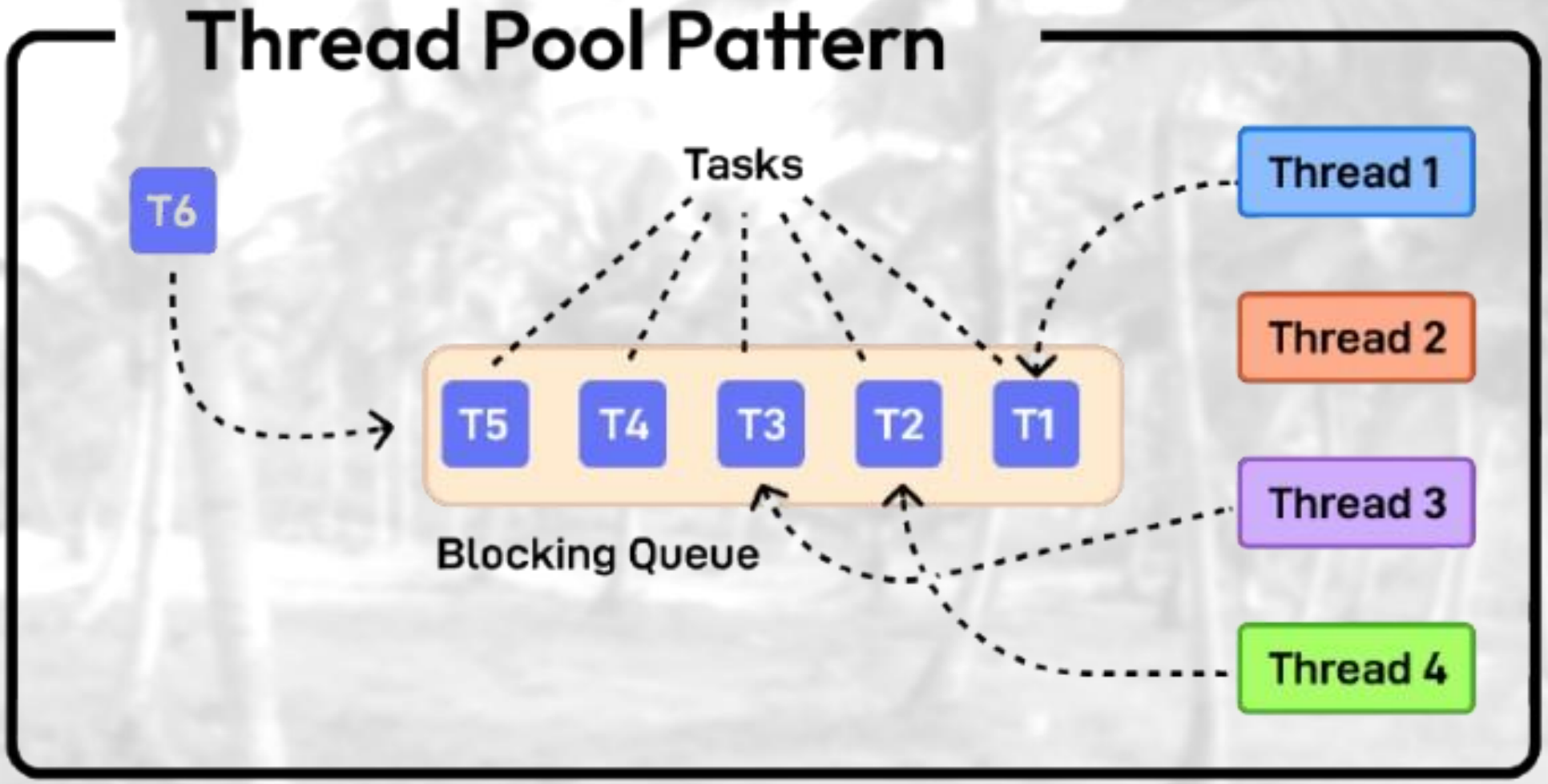

Producer Consumer Pattern
Thread 1
Tail
Thread 3
Blocking Queue
Producer
Threads
Consumer
Threads
Thread 2
Head
Thread 4

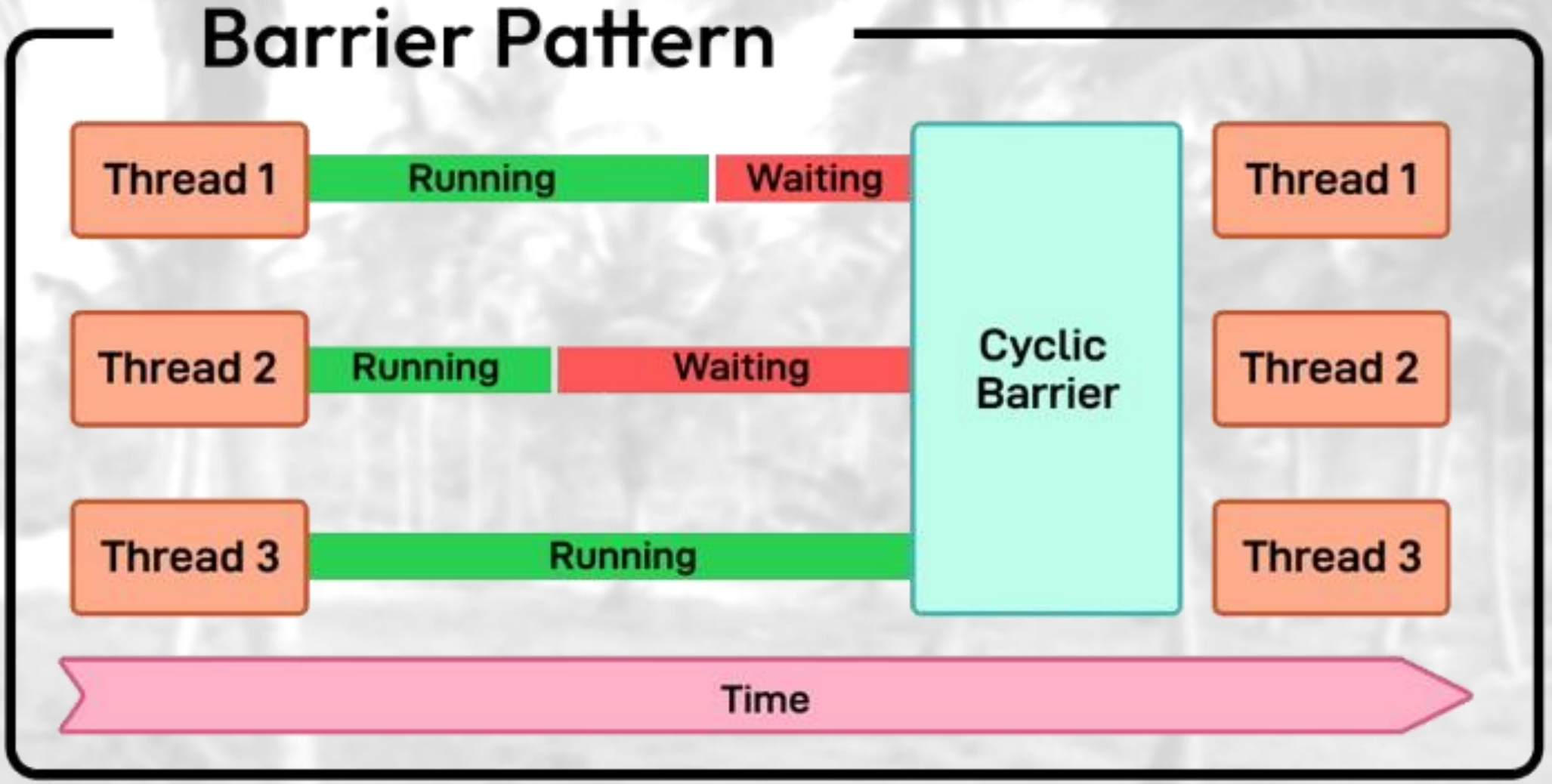
Read/Write Lock Pattern



Critical Section Pattern




Thread Pool Pattern
Tasks
T6
T5
T4
T3
T2
T1
Blocking Queue
Thread 1
Thread 2
Thread 3
Thread 4

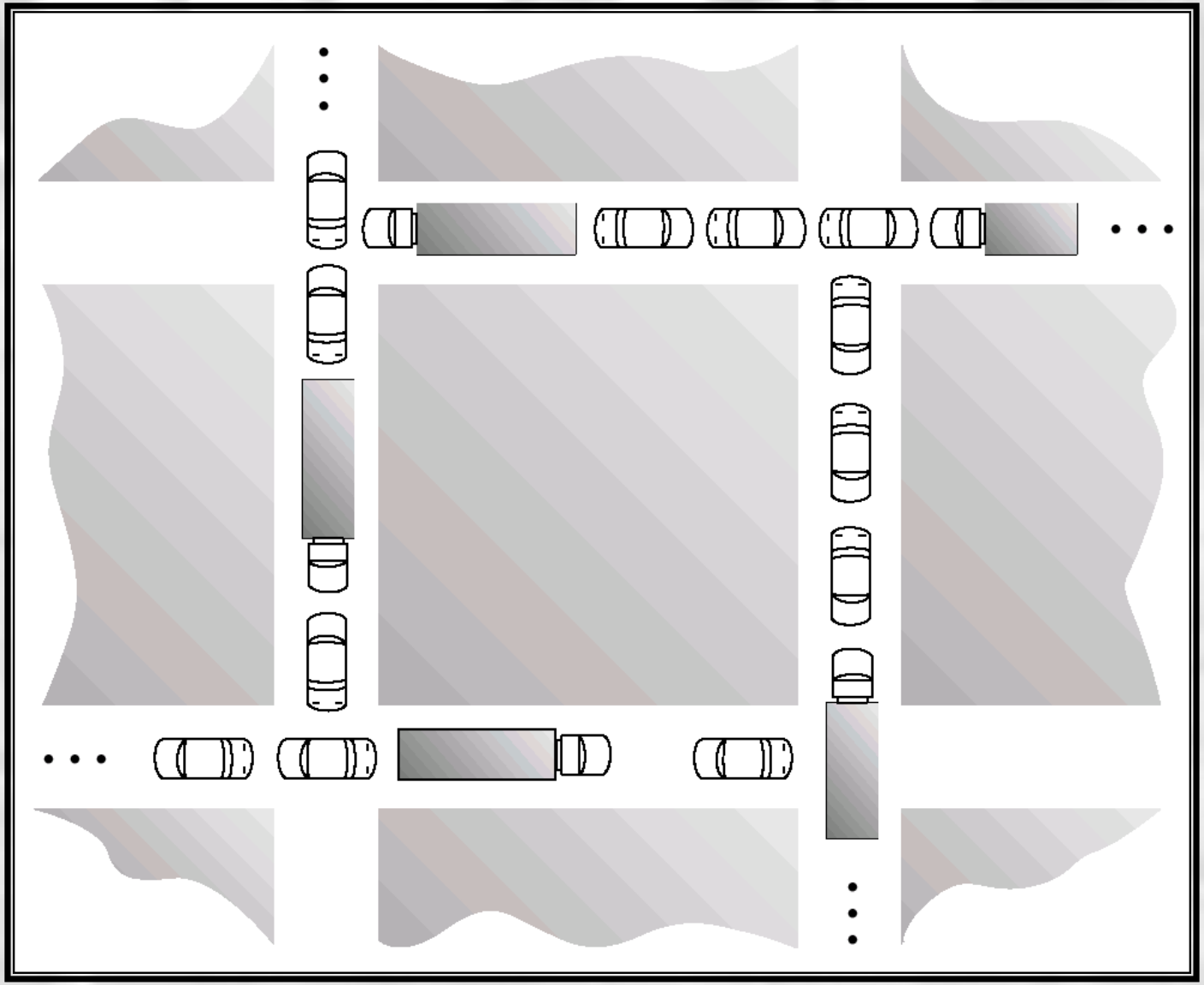

Barrier Pattern
Thread 1
Running
Waiting
Thread 2
Running
Waiting
Thread 3
Running
Cyclic Barrier
Thread 1
Thread 2
Thread 3
Time

Other Solutions

- Synchronization Hardware
- Mutex (via acquire and release)
- Semaphores
- Etc.

Deadlocks

Is this Deadlock?

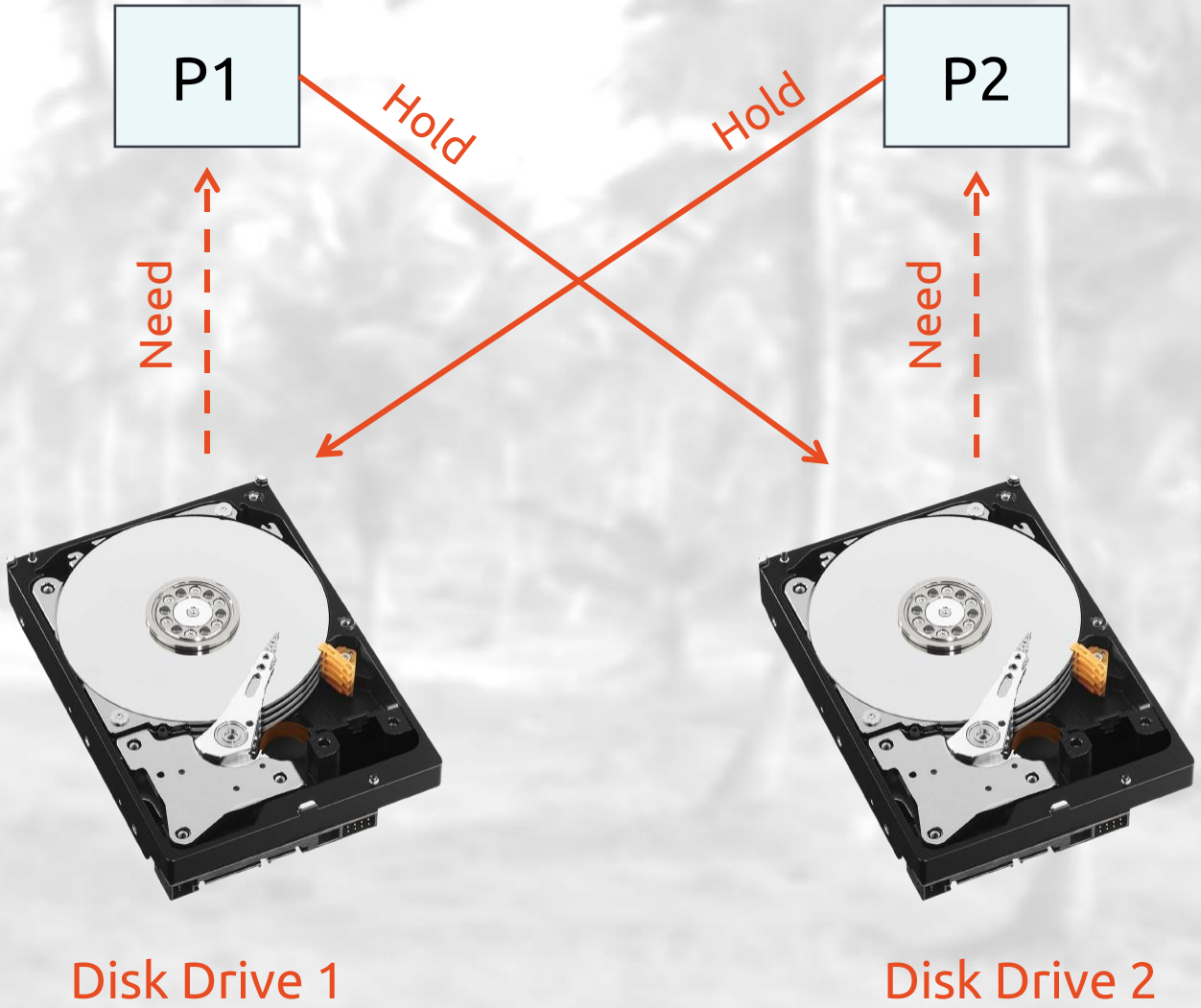


The Deadlock Problem

- A set of blocked processes each holding a resource and waiting to acquire a resource held by another process in the set.
- **Example**
  - System has 2 tape drives.
  - P1 and P2 each hold one tape drive and each needs another one.

P1	P2
wait (A);	wait(B)
wait (B);	wait(A)

Deadlock Example



Methods for Handling Deadlocks

- Ensure that the system will never enter a deadlock state.
- Allow the system to enter a deadlock state and then recover.
- Ignore the problem and pretend that deadlocks never occur in the system
  - used by most operating systems, including UNIX
  - lets the user handle it

- Deadlock prevention is not completely guaranteed and is incredibly hard!

Open-Source OSes

Open-Source Operating Systems

- Operating systems made available in source-code format rather than just binary closed-source (i.e., exe).
- Counter to the copy protection and Digital Rights Management (DRM) movement.
- Started by Free Software Foundation (FSF), which has “copyleft” GNU Public License (GPL).
- Examples include GNU/Linux and BSD UNIX (including core of Mac OS X), and many more.

Command Line

Command Line Interfaces (CLI)

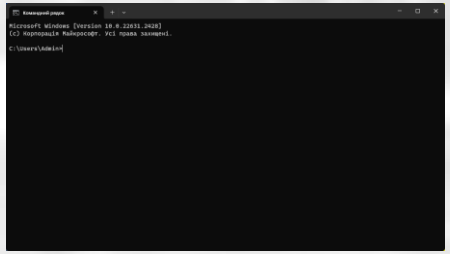
Powershell

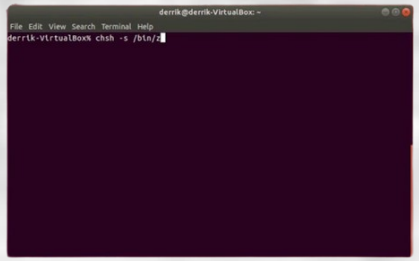
Command Prompt

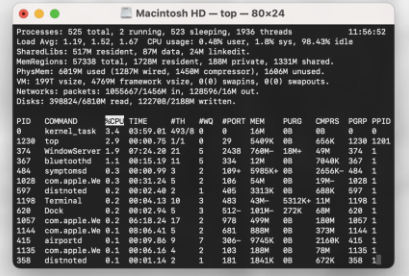
CMDER

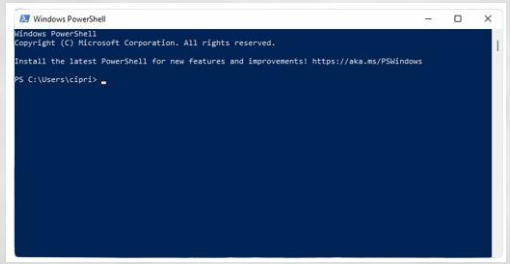
Bash

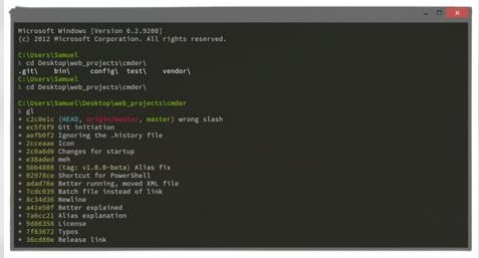
macOS Terminal











Key Reasons to Use the Command Line

- **Efficiency:** Execute tasks quickly and efficiently.
- **Automation:** Script complex or repetitive tasks.
- **Powerful Tools:** Access advanced utilities and features.
- **Resource Light:** Minimal CPU and RAM usage.
- **Remote Management:** Administer systems remotely via SSH.
- **Customization:** Personalize with scripts and commands.
- **Precision:** Gain detailed control and options.
- **Troubleshooting:** Diagnose issues with logs and error messages.
- **Learning & Skills:** Enhance technical proficiency and system understanding.

Thank You!

Hooray! You've made it to the end.